

—— // ZWIRE // ——

zwire

v0.6.40

THE COMPANION

the Blink base · the rebrand · the HUD · the fork

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	4
Chapter 1 — The Pitch	6
Chapter 2 — Why a Real Blink Base	8
Chapter 3 — The Architecture	10
Chapter 4 — The Base	13
Chapter 5 — The Runtime Rebrand	15
Chapter 6 — The New-Tab HUD	17
Chapter 7 — The Internal-HUD Extension: The Injection Model	20
Chapter 8 — The Eight Schemes and the Sync Bus	24
Chapter 9 — Vim on Every Page	30
Chapter 10 — tmux in the Browser	33
Chapter 11 — The Powerline Statusbar	38
Chapter 12 — The Command Palette	40
Chapter 13 — Fuzzy Find	43
Chapter 14 — The HUD Internal Pages	44
Chapter 15 — The Native Host	47
Chapter 16 — The Terminal	50
Chapter 17 — zgui-core, the Shared Toolkit	52
Chapter 18 — zpwrchrome, the Power-Tool	54
Chapter 19 — The Launcher and State Layout	56
Chapter 20 — The CDP Verification Layer	59
Chapter 21 — The Source Fork	61
Chapter 22 — Dev Loop, Testing, and CI	67
Chapter 23 — Packaging and Self-Contained Installs	69
Chapter 24 — Cross-Platform, Updating, and Operations	72

Chapter 25 — The Trust Boundaries and the Threat Model	74
Chapter 26 — The Performance Budget	76
Chapter 27 — Extending the HUD	78
Chapter 28 — The Lineage: The Cyberpunk Idiom and the zpwr Fleet	80
Chapter 29 — Positioning: zwire Among the Custom Browsers	82
Chapter 30 — Failure Modes and Recovery	84
Appendix A — File and Directory Map	87
Appendix B — The State Directory (\$ZWIRE_STATE)	89
Appendix C — Keybinding Reference	90
Appendix D — The Eight-Scheme Palette	91
Appendix E — Launch Flag Reference	93
Appendix F — The Chrome WebUI Token Map	94
Appendix G — The Native-Messaging Protocol	97
Appendix H — The Seeded Keyword Commands	98
Appendix I — The Storage Keys	100
Appendix J — The Rust Host Modules	101
Appendix K — The Launcher and Build Scripts	102
Appendix L — The CDP Verification Probes	103
Appendix M — The zpwrchrome Manifest	104
Appendix N — The Fork Patch Targets	105
Appendix O — The Three Extension Manifests	106
Appendix P — The Content-Script Injection Order	107
Appendix Q — The Fuzzy Matcher	108
Appendix R — Visual Effects and Readline	109
Appendix S — The System-Stats Sample	110
Appendix T — The Palette Destinations	111
Appendix U — The Command Bus	112
Appendix V — The Frecency Model	113
Appendix W — The Extension Messaging Graph	114
Appendix X — The zgui-core Toolkit Surface	115

Appendix Y — The HUD Internal Pages	116
Appendix Z — The Dev-Loop Scripts	117
Appendix AA — Three Paths, End to End	118
Appendix AB — Glossary	119
Appendix AC — The Statusbar Segments	120
Appendix AD — Grounding and Edition	121

Before You Begin

This is the book about **zwire** — the one you read in a chair, front to back, to understand not only *what* **zwire** does but *why* it is shaped the way it is. It is not the README, which enumerates every switch, path, and patch and which you reach for when you need one fact fast. This book is the other thing: the continuous account of the browser — from the unbranded Blink snapshot it starts from to the neon toolbar the source fork compiles — and of the deliberate line it draws between what a runtime rebrand can do and what only a patched Chromium can.

zwire is a **Chromium/Blink browser forked into a tiling cyberpunk HUD**. It is not a WebView wrapper and not a new engine: it is a real Blink base, patched at the seams a compiled fork can reach and extended with a keyboard-driven workspace. The repository states the intent on its own front page — [\[FORKED CHROMIUM · TILING HUD · CYBERPUNK\]](#), with the one-line creed the rest of this book unpacks: “*Not Chrome with a skin — a real Blink engine turned into a keyboard-driven, tiling, cyberpunk workspace.*” It carries a tmux-style tiling overlay, a `⌘K` command palette, durable sessions, a browser-wide audio engine, and the **zpwchrome** power-tool preloaded against a dedicated profile so it never touches your system Chrome.

The reason **zwire** needs a *real* Blink engine, rather than a WebKit shell like Tauri or Servo, is concrete and it drives every decision downstream: the **zpwchrome** extension is Manifest V3 and depends on `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, and a service-worker background, with `minimum_chrome_version: 127`. None of that runs anywhere but a genuine Chromium engine. The base is therefore a plain, **unbranded Chromium snapshot** — no Google logo, no “for automated testing” stripe — which also happens to retain the `--load-extension` switch that branded Chrome removed in version 137. That single retained switch is what lets **zwire** preload its extensions at all; the book returns to it in Chapter 2.

zwire ships two build paths, and the whole shape of the project is the boundary between them. The **runtime rebrand** (the default, no compile) applies HUD colors, the HUD new-tab, the internal-page skin, and your extensions on top of a prebuilt Chromium base — tab shapes stay stock, because a Chrome theme extension can change colors and nothing else. The **source fork** (`fork/`, which compiles Chromium) removes that limit: it restyles the native chrome itself — tab shapes, UI fonts, the neon toolbar, native menus and dialogs, and a browser-wide audio engine — through a series of twenty-four authored patches against a pinned Chromium tag. This book is honest about which capabilities live on which side of that line, because the line *is* the design.

The whole thing is two commands to stand up and one to run:

```
git clone --recurse-submodules https://github.com/MenkeTechnologies/zwire.git
cd zwire
scripts/install.sh          # fetch base + link `zwire` on PATH + rebrand (macOS)
zwire                       # launch with the HUD new tab
```

This is fast-moving software: the edition string on the cover tracks `zwire/package.json (v0.5.58` at the time of writing), and where a feature needs the compiled fork, or a native host must be installed separately, this book marks it rather than rounding up. **zwire** is a MenkeTechnologies project, built on the Chromium project’s engine, and this book is grounded in that code.

How to read this book

The book runs bottom-up. The first chapters are the pitch and the justification — what **zwire** is and why it must be Chromium. Then it descends the stack: the base, the rebrand, the new tab, and the large internal-HUD

extension, which gets a chapter per feature it layers on every page (the eight schemes, vim, the tmux overlay, the powerline statusbar, the command palette, the fuzzy find, the HUD internal pages, the native host, and the terminal). After the internal HUD come the power-tool submodule, the launcher, and the CDP verification probes, then the source fork, the dev-and-CI loop, the packaging, and the operations runbook. The reference appendices at the back cover the file map, the state directory, the full keybinding tables, the eight-scheme palette (dark and light), the launch flags, the Chrome WebUI token map, the native-messaging protocol, the seeded keyword commands, the storage keys, the Rust host modules, the launcher and build scripts, the CDP probes, the power-tool manifest, and the fork patch targets.

Every claim in the chapters is grounded in a file in the [zwire](#) repository, and where the book says “the code comments say,” it is quoting the source. Let’s start.

Chapter 1 – The Pitch

A browser is the least likely thing on the stack to be rewritten, and zwire does not rewrite it. This chapter states what zwire *is* — not “a browser from scratch,” but Chromium **owned** at the seams a rebrand and a patch series can reach: a real Blink engine wearing the cyberpunk HUD, carrying the author’s extensions, on a profile that never collides with system Chrome.

What zwire is – a rebrand and an extension host, not a new engine

The lazy framing is “a custom browser,” which usually means either a WebView wrapper (WebKit, no MV3) or a fresh engine (a decade of work nobody finishes). zwire is neither. It takes a real Chromium and changes the two things that matter for a daily driver — how it looks and what it runs — while inheriting the engine, the security model, and the update cadence for free.

The mechanics are unglamorous on purpose. `bin/zwire` is a shell script. It reads one path (`~/.zwire/base.path`, written when the base was fetched), and it `execs` that binary against a dedicated profile with a fixed set of extensions loaded. There is no daemon, no supervisor, no packaged Electron runtime, no bundled Node. The “browser” is a Chromium snapshot Google itself builds and hosts; zwire’s contribution is the profile, the extension set, the icon, the name, and — in the fork — twenty-four C++ patches. Everything else is Blink, unmodified, doing what Blink does. That is the entire trick, and it is why zwire exists in kilobytes of shell and JavaScript while a from-scratch browser would be millions of lines of C++ nobody finishes.

Two consequences follow immediately, and they run through the whole book. First, zwire tracks upstream Chromium *for free*: a security fix that lands in the Blink tree is one `scripts/fetch-base.sh` away, because the base is a stock snapshot, not a frozen fork of one. (The fork path pays for its native restyle by giving that up — Chapter 21 is honest about the rebase cost.) Second, every capability zwire adds on the runtime path is an *extension* or a *launch flag* — nothing patched, nothing recompiled — so the runtime rebrand degrades gracefully: delete an extension directory and the launcher simply skips it, and you still have a working browser.

What the rebrand buys, and what only the fork buys

A Chrome theme extension can repaint the frame, the toolbar background, and the tab strip — and nothing else. Tab *shape*, the UI *font*, the toolbar *border*, the omnibox field *geometry*: those are native C++ in Chromium’s Views layer, not exposed to any extension API. This is not a zwire limitation; it is the boundary Chromium draws around what a theme is allowed to touch. The runtime rebrand accepts that boundary and does everything it can reach on the right side of it — which turns out to be a great deal, because zwire pushes the color and content surface much further than a theme extension normally does:

- **Colors** it changes through the theme layer and, more aggressively, through the internal-HUD content script that skins `chrome://` pages directly.
- **Content** it owns completely: the new tab is a full HUD page (`chrome_url_overrides`), and several internal pages (`chrome://extensions`, `chrome://history`, `chrome://bookmarks`, `chrome://settings`, `chrome://version`) are redirected to HUD-styled replacements served from the extension.

- **Behavior** it layers on with content scripts: vim keybindings on every page, a tmux-style tiling overlay, a powerline statusbar, a `⌘K` command palette, a fuzzy find bar that replaces Chrome's own, and an xterm terminal over a real PTY.

What stays out of reach on the runtime path is precisely the native Views chrome: the pill-shaped tabs, the system UI font, the rounded omnibox. The `fork/` path compiles a patched Chromium to reach those — trapezoid HUD tabs, a Share Tech Mono UI font, a neon under-toolbar line, HUD-colored frame, and the eight HUD schemes wired into the DevTools Theme dropdown. `zwire` ships *both* paths rather than forcing the ~100 GB Chromium checkout on everyone: the rebrand is the daily driver, and the fork is there for whoever wants the whole browser in the HUD and will pay the build-and-rebase cost. The boundary between them is not a wart to be apologized for — it is the map of what a browser lets you change without becoming its maintainer.

Why it never touches system Chrome

`zwire` runs against `~/.zwire/profile` — a dedicated `--user-data-dir` — and never your system Chrome profile. This is not politeness; it is a hard requirement of loading unpacked extensions with elevated flags. The launcher passes `--load-extension`, `--extensions-on-chrome-urls`, and `--test-type`; you do not want those flags, those specific unpacked extensions, or the developer-mode banner they raise anywhere near the profile that holds your real logins, cookies, and saved passwords. A separate user-data-dir keeps the two worlds from ever sharing state: `zwire`'s bookmarks, history, sessions, and extension metadata live under `~/.zwire/profile`, and Chrome's live where they always did.

The isolation goes one level deeper than the profile. The extensions the launcher loads are staged into a per-user `~/.zwire/ext/` copy at launch, not run in place from the `.app` bundle, because Chromium writes extension `_metadata/` indexes into the extension directory and every user on the machine needs their own writable copy. And the native-messaging host — the bridge that powers the HUD terminal, the system-stats stream, and the scheme sync — is installed *profile-relative*, into `~/.zwire/profile/NativeMessagingHosts/`, so it is wired for `zwire` and invisible to system Chrome. The whole design keeps `zwire` a guest that brought its own everything: its own profile, its own extension copies, its own native host, its own icon and name. Nothing it does can be felt by the browser you already had.

The two tiers, in the launcher's own words

The whole shape of the project is stated at the top of `bin/zwire` itself, and it is worth reading before the chapters take the stack apart: “Chromium, rebranded, with the `zpwrcchrome` power-tool + cyberpunk new-tab + the internal-HUD skin preloaded. Runtime rebrand: a plain Chromium snapshot binary launched against a dedicated profile with a fixed extension set. Nothing to compile (the optional whole-chrome HUD lives in `fork/`).” Two tiers, named in the first four lines of the launcher: a runtime rebrand that compiles nothing and reaches everything an extension and a launch flag can reach, and an optional source fork that recompiles Chromium to reach the native chrome. The parenthetical is the honest boundary the rest of the book draws — “the optional whole-chrome HUD lives in `fork/`” — and it is why the same `bin/zwire` launches both: the launcher reads only `base.path`, so it does not care whether that path points at a downloaded snapshot or a compiled fork. The runtime tier is the daily driver, the fork tier is the whole-browser HUD for whoever will pay the build cost, and the seam between them is a design decision, not a limitation to apologize for.

Chapter 2 – Why a Real Blink Base

This chapter is the technical justification for the whole project: the exact set of Manifest V3 capabilities `zpwchrome` requires, why each one rules out the non-Chromium alternatives, and why the base must be *unbranded* Chromium rather than Google Chrome or Chrome for Testing.

The MV3 capability set `zpwchrome` requires

`zpwchrome` declares `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, `webRequest`, a service-worker background, and `minimum_chrome_version: 127`. Each is load-bearing, and the combination exists only on a genuine Blink engine. Take them one at a time:

- `userScripts` is Manifest V3's supervised replacement for the old arbitrary-code injection that `tabs.executeScript` used to allow. It lets an extension register user-authored scripts that run in page context. It is a Chromium-only API — it does not exist in Safari's WebExtensions surface, and it has no analogue in a WebView, which exposes no extension model at all.
- `declarativeNetRequestWithHostAccess` is the MV3 network-rules engine: the extension ships declarative match/modify rules that the browser applies in its own network stack, rather than round-tripping every request through a JavaScript listener. The `WithHostAccess` variant grants the dynamic, host-scoped rules `zpwchrome` needs. This is the modern content-blocking and request-rewriting substrate, and it is implemented in Chromium's network service — not reachable from a wrapper that only owns a WebView. (zwire's own `hud-internal` extension uses the plain `declarativeNetRequest` form for its frame-busting rule set, discussed in Chapter 10.)
- `nativeMessaging` is the bridge to a local process: the extension launches a registered native binary and speaks a length-prefixed JSON protocol over its stdio. `zpwchrome`'s `pass` integration and segmented downloads need it to talk to a native host; zwire's own HUD host (Chapter 15) uses the same mechanism for the terminal and the system-stats stream. Native messaging requires the browser to read a host manifest from a known directory and spawn the process — something only a full browser (not a WebView) does.
- `webRequest` is the observe/block network API. Even in MV3, where the *blocking* form is constrained in favor of declarativeNetRequest, the observational surface remains, and `zpwchrome` declares it.
- **A service-worker background** is the MV3 background model: no persistent background page, an event-driven worker that the browser wakes on demand and suspends when idle. Every piece of zwire that coordinates across tabs — the scheme bridge, the `⌘K` command relay, the terminal open-state — is written for this suspend-any-time worker, and it only exists inside a Chromium extension runtime.
- `minimum_chrome_version: 127` is the extension declaring, in its own manifest, that it will not load on anything older than Chromium 127. That is not a preference; it is a floor. It rules out not only WebKit and Servo but any Chromium old enough to predate the APIs above.

The point is cumulative. Any *one* of these might be worked around; the *set* can only be satisfied by a current, genuine Chromium/Blink engine running a real extension runtime. Laid against the alternatives:

Capability need	WebView (Tauri)	Servo	Safari WebExtensions	Chromium
userScripts	no ext model	no	no	yes
declarativeNetRequest	no	no	partial	yes
nativeMessaging	no (own IPC)	no	limited	yes
service-worker bg	no ext model	no	yes	yes
--load-extension	n/a	n/a	n/a	yes (unbranded)

That is why zwire is a Chromium rebrand and not a Tauri app, a Servo experiment, or a Safari extension. The engine is not an implementation detail zwire happened to pick — it is the one requirement everything else hangs from.

Why unbranded Chromium, not Chrome or Chrome for Testing

The base is a plain Chromium snapshot from the public snapshot index — no Google logo, no “for automated testing” banner. This narrows the field to exactly one build, and it is worth seeing why the two obvious alternatives are both wrong.

Google Chrome is the branded build, and branded Chrome removed the `--load-extension` command-line switch in version 137. That switch is the entire mechanism by which zwire preloads its extensions on launch — the launcher’s `--load-extension="$LOAD"` line. Without it there is no way to load the HUD new-tab, the internal-HUD skin, and `zpwchrome` unpacked from disk at startup. So branded Chrome is disqualified by a removal that is specific to the *branded* build; the upstream Chromium engine still honors the switch.

Chrome for Testing is Google’s dedicated automation build, and it keeps `--load-extension` — but it wears a permanent “for automated testing” info-bar stripe across the top of every window, because that is its entire purpose: a build you cannot mistake for a user’s real browser. That stripe is exactly what you do *not* want on a daily driver you have gone to the trouble of rebranding with a cyberpunk icon and name. So Chrome for Testing is disqualified by the banner.

Plain unbranded Chromium — the continuous-build snapshot Google hosts at storage.googleapis.com/chromium-browser-snapshots — threads the needle: it is a real Chromium/Blink build with no Google product branding *and* no testing banner, and it retains `--load-extension`. `scripts/fetch-base.sh` resolves the correct platform id (`Mac_Arm`, `Mac`, `Linux_x64`, `Win_x64`) and, by default, the latest snapshot revision from the `LAST_CHANGE` marker; pass an explicit revision as the one argument to pin an exact build. The resolved revision is recorded so a launch is reproducible. Chapter 4 opens the fetch mechanics; the takeaway here is that the choice of base is not aesthetic — it is the only build that satisfies both constraints at once.

The manifest is the evidence

This chapter’s argument is not abstract; it is written down in the power-tool’s `manifest.json`, and it is worth pointing at the exact lines. `zpwchrome` declares `manifest_version: 3`, `minimum_chrome_version: 127`, and nineteen permissions, four of which — `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, and `webRequest` — are the ones that exist only on a current Chromium engine running a real extension runtime. The `minimum_chrome_version` line is not a preference the launcher could talk it out of; it is a hard floor the browser itself enforces, so any base older than Chromium 127, and every non-Chromium engine, refuses to load the extension at all. The internal-HUD manifest adds its own load-bearing declarations — `declarativeNetRequest` for the frame-busting rule, `developerPrivate` and `management` for the extensions page, `settingsPrivate` for the settings page, `system.display` for the worker’s window math — none of which a WebView exposes and none of which Safari’s WebExtensions surface fully implements. The whole of Chapter 2 reduces to a claim you can check by reading two manifests: the *set* of capabilities these extensions require is satisfiable on exactly one engine, and that engine is a genuine, current Chromium. Everything else about zwire is downstream of that single, verifiable fact. (Appendix M is the full manifest.)

Chapter 3 – The Architecture

zwire is seven cooperating layers. This chapter names each one and its single responsibility before the later chapters open them up: **Base**, **Rebrand**, **Theme**, **New tab**, **Internal HUD**, **Power-tool**, and **Launcher**, with the optional **Fork** underneath them all.

The seven layers, top to bottom

Read the stack from the engine up:

- **Base** — a pinned, plain Chromium snapshot downloaded by `scripts/fetch-base.sh` into `~/.zwire/base`, with its resolved binary path recorded in `~/.zwire/base.path` and its revision in `~/.zwire/base.version`. This is the engine, unmodified. Everything else is skin and cargo loaded onto it. On the fork path this same slot holds the compiled patched Chromium instead; the launcher does not care which, because it reads only `base.path`. (Chapter 4.)
- **Rebrand** — `scripts/rebrand-macos.sh`, the in-place bundle patch that makes the Dock, `⌘-Tab` switcher, and menu bar show “zwire” with the cyberpunk icon. It sets `CFBundleName/CFBundleDisplayName`, installs the `.icns`, deletes the key that would override it, re-signs the bundle ad-hoc, and re-registers it with LaunchServices. On Linux there is no bundle to patch — the launcher name *is* the brand — so this layer is macOS-only. (Chapter 5.)
- **Theme** — `theme/`, a Chrome theme extension that maps the HUD palette onto frame, toolbar, and tabs (colors only). It is present in the tree but, deliberately, **not loaded by the launcher** — the reason is the whole point of the palette pipeline and gets its own section below.
- **New tab** — `newtab/`, a `chrome_url_overrides.newtab` extension: the full HUD new tab, with a neon clock, an omnibox, quick-launch tiles, and its fonts vendored locally so it renders offline. (Chapter 6.)
- **Internal HUD** — `extensions/hud-internal`, the MV3 content-script suite that skins Chrome’s own internal pages, redirects several of them to HUD replacements, and layers vim keys, a tmux overlay, a powerline statusbar, the `⌘K` palette, a fuzzy find bar, and a terminal onto every page — plus the floating eight-scheme picker and its native host. This is the biggest layer, and Chapters 7 through 16 open it one feature at a time.
- **Power-tool** — `extensions/zpwrchrome`, the Manifest V3 power-tool, loaded as a **submodule** rather than vendored, so zwire tracks it upstream instead of freezing a copy. (Chapter 18.)
- **Launcher** — `bin/zwire`, the single entry point that assembles the flags, ensures the native host, makes the profile, and `execs` the base. (Chapter 19.)
- **Fork** (optional, underneath) — `fork/`, the source build that compiles a patched Chromium so the native chrome itself lives in the HUD. It is not a layer the runtime path touches; it is an alternate *base*. (Chapter 21.)

The layering has a property worth naming: every layer above the base is optional and independently removable. Delete `theme/` (already unloaded), delete `newtab/`, delete the `zpwrchrome` submodule — the launcher includes only the extension directories that actually contain a `manifest.json`, so a missing layer degrades to a plainer browser rather than a broken one. That graceful-skip rule is in the launcher itself, and it is what lets a fresh clone without submodules still run.

Why the static theme is present but not loaded

The `theme/` extension maps the HUD palette onto frame, toolbar, and tabs — but the launcher does not load it, and the launcher says so in a comment where it builds the extension list. Two things own the chrome palette ahead of it, and a static theme would fight both:

1. **The fork's native color mixer (patch 0002).** On the fork path, the cyberpunk palette is forced onto frame, toolbar, tab background/foreground, and the omnibox directly in Chromium's `chrome_color_mixer.cc`, keyed to the active scheme. That is the authoritative source of the chrome colors when the fork is in use.
2. **The internal-HUD skin.** On the runtime path, the `hud-internal` content scripts drive the scheme, and the scheme is mirrored through a native file so the whole HUD — pages, statusbar, palette — follows one picker.

A Chrome custom theme extension applies **last** in Chromium's color pipeline: it is designed to win, so the user's chosen theme is what they see. If `zwire` loaded `theme/`, it would override both the native mixer and the scheme-driven skin, pinning the chrome to one static palette and defeating the eight-scheme picker. So the theme extension is kept in the tree — it is a correct, working Chrome theme, useful as a reference and loadable by hand — but excluded from the launcher's default set precisely because it is *too* authoritative. Understanding that ordering is understanding the whole palette pipeline: the scheme picker is the source of truth, the native mixer and the content-script skin are its two renderers, and a static theme is deliberately kept out of the way.

One source of truth, four renderers

It is worth stating the palette architecture as a whole here, because it is the shape every later chapter reconnects to. There is exactly one source of truth for the active scheme — the native file `~/.zwire/hud-scheme`, owned by the host — and there are four renderers that all reconcile to it. The internal-HUD content scripts read a mirror of it in `chrome.storage.local` (`zb_scheme`) and style every ordinary page and every skinned `chrome://` page from it. The new-tab extension, a different origin that cannot share that storage, polls the native file directly and styles itself. The power-tool receives it over `runtime.sendMessage` and colors its own surfaces. And on the fork path, the compiled color mixer reads the same native file and colors the native chrome. Four renderers, one file, no primary — pick a scheme anywhere and the file changes, and every renderer follows within its own refresh. The reason the static `theme/` extension is excluded is now precise: it would be a *fifth* renderer that applies last and ignores the file, pinning the chrome to one palette and breaking the picker for the other four. Keeping it in the tree but out of the launcher is not an oversight; it is the one place a working Chrome theme would actively fight the architecture.

The layering has a second property the launcher enforces: every layer above the base is independently removable. The extension list is built by including only directories that contain a `manifest.json`, so a missing `zpwchrome` submodule, a deleted `newtab`, or an uninitialized checkout simply drops that layer and the browser runs with what remains. The native host is ensured but never required. Nothing above the stock engine is load-bearing for launch — which is why a fresh clone without submodules still opens a working browser, just a plainer one.

The layers at a glance

The seven layers, their one file or directory, and the chapter that opens each:

layer	where	responsibility	ch
Base	\$ZWIRE_STATE/base + base.path	the stock (or forked) engine	4
Rebrand	scripts/rebrand-macos.sh	Dock name + icon (macOS only)	5
Theme	theme/ (NOT loaded)	colors-only Chrome theme	3
New tab	newtab/	the HUD new-tab override	6
Internal HUD	extensions/hud-internal/	skin + vim + tmux + palette + find + statusbar + terminal	7-16
Power-tool	extensions/zpwichrome/ (submod)	the MV3 power-tool	18
Launcher	bin/zwire	assemble flags, exec the base	19
Fork	fork/ (optional base)	compiled whole-chrome HUD	21

Every row above the base is optional and independently removable; the launcher reads only `base.path`, so it neither knows nor cares whether the base is a snapshot or a compiled fork, and it loads only the extension layers that actually carry a manifest. That single indifference is what lets one launcher serve both build tiers and lets a partial checkout still run.

Chapter 4 – The Base

The engine layer: how `scripts/fetch-base.sh` resolves and pins a Chromium snapshot, where it lands under `~/.zwire/base`, and how `base.path` / `base.version` record what was resolved so a launch is reproducible.

Fetching and pinning a snapshot

`scripts/fetch-base.sh` is the whole of the base layer, and it is a small, readable script. Its job is to turn “the host I am running on” into “the correct Chromium snapshot binary on disk,” and to record what it resolved.

It begins by mapping the host to a snapshot platform. `uname -s/uname -m` select one of four rows:

```
Darwin/arm64      -> Mac_Arm    chrome-mac.zip  chrome-mac
Darwin/x86_64     -> Mac        chrome-mac.zip  chrome-mac
Linux/x86_64 | amd64 -> Linux_x64  chrome-linux.zip chrome-linux
MINGW/MSYS/CYGWIN -> Win_x64    chrome-win.zip  chrome-win
```

Each row names the snapshot platform id, the zip filename Google publishes for it, and the subdirectory that zip extracts to. Anything else exits with an “unsupported platform” message rather than guessing.

The revision comes next. With no argument, the script fetches the `LAST_CHANGE` marker for the platform — the latest continuous-build revision — and uses that. Pass a revision as the single argument and it pins that exact build instead:

```
scripts/fetch-base.sh      # latest snapshot (LAST_CHANGE)
scripts/fetch-base.sh 1656770 # pin an exact revision
```

This is the pinning story in full. There is no lockfile and no separate manifest: the “pin” is the revision string you pass, and the record of what you got is the `base.version` file the script writes. Pinning matters because Chromium snapshots are continuous — the latest changes hour to hour — and a reproducible `zwire` means knowing exactly which Blink you are running. For a machine you want to keep stable, fetch a revision once and never pass no-argument again until you choose to roll forward.

For total control there is a bypass. `bin/zwire` honors a `ZWIRE_BASE` environment variable pointing at any executable Chromium binary, checked before it consults `base.path`. That lets you point `zwire` at a Chromium you built yourself, a distro package, or a second snapshot, without going through `fetch-base.sh` at all — useful when developing the fork, where you want the launcher’s extension loading but your own compiled base.

What lands on disk

The download itself is a `curl` of `storage.googleapis.com/chromium-browser-snapshots/<PLAT>/<REV>/<ZIP>` into `~/.zwire`, an `unzip` into `~/.zwire/base`, and a delete of the zip. Then the script resolves the actual executable inside the extracted tree, because the path differs per platform:

- On macOS it finds the Chromium binary at `.../Chromium.app/Contents/MacOS/Chromium` inside the extracted `chrome-mac`.
- On Linux the binary is `.../chrome-linux/chrome`.

- On Windows/MSYS it is `.../chrome-win/chrome.exe`.

With the binary located, the script writes exactly two small files and nothing else:

```
~/.zwire/base.path      the absolute path to the resolved binary (bin/zwire reads this)
~/.zwire/base.version   the revision, as "r<REV>"           (reproducibility)
~ record)
```

`base.path` is the contract with the launcher: `bin/zwire` reads that one line to find the engine. `base.version` is the human-and-tooling record of which snapshot is installed. The `base/` directory holds the extracted browser itself — roughly 325 MB, the number the dev-loop scripts cite when they warn about what a `nuke` re-downloads. That is the entire on-disk footprint of the base layer — one directory and two one-line files — and it is deliberately flat so that a base upgrade is just “run `fetch-base.sh` again,” which overwrites the extracted tree and rewrites the two files. (On macOS an upgrade also means re-running the rebrand, because the fetch replaces the freshly-patched bundle with a stock one; Chapter 5 explains why.)

The script, exactly

`fetch-base.sh` is small enough to walk in full. Its own header names the base it targets: “a plain Chromium continuous-build snapshot (Google-hosted): a real Chromium/Blink build with NO ‘Google Chrome’ product branding and NO ‘for automated testing’ banner ... Unbranded Chromium also retains the `--load-extension` switch.” It maps `uname -s/-m` to a `PLAT/ZIP/SUBDIR` triple through the four-row `case` of Chapter 3, then, with no revision argument, `curls` the platform’s `LAST_CHANGE` sentinel from `storage.googleapis.com/chromium-browser-snapshots/<PLAT>` to learn the latest continuous-build number; a revision passed as the single argument pins that build instead. The download is a `curl ... --progress-bar` of `<BASE_URL>/<REV>/<ZIP>` into the state dir, an `unzip -q -o` into `$ZWIRE_STATE/base` (after an `rm -rf` of any prior extraction), and a delete of the zip.

Then it resolves the executable, which differs per platform: on macOS a `find` for the `Chromium` binary inside `*Chromium.app/Contents/MacOS/*`, on Linux `<SUBDIR>/chrome`, on MSYS `<SUBDIR>/chrome.exe`. It writes the two contract files — `printf '%s\n' "$BIN" > base.path` and `printf 'r%s\n' "$REV" > base.version` — and prints where it installed. That `r`-prefix on `base.version` is the marker Chapter 19 reads to tell a snapshot (`r...`) from a compiled fork (`fork-...`). The `~325 MB` figure the dev-loop scripts cite for the extracted browser is not hardcoded in this script; it lives in `localinstall.sh` and `nuke.sh` as the size those operations warn about re-downloading.

Chapter 5 – The Runtime Rebrand

The no-compile path. On macOS `scripts/rebrand-macos.sh` patches the base bundle's Dock name to `zwire`, swaps in the cyberpunk `.icns`, deletes `CFBundleIconName`, and re-signs ad-hoc so macOS honors the change. This chapter explains why each of those steps is required and why the rebrand must be re-run after a base upgrade.

The Dock name, the icon, and the ad-hoc re-sign

The rebrand is an in-place edit of the base bundle under `~/zwire/base` — it never touches a system browser install. The script reads `base.path`, walks up to the `.app` directory (stripping `/Contents/MacOS/*`), and patches the bundle's `Info.plist` and `Resources`. Four operations, each of which is there for a specific macOS reason:

1. **The name.** `CFBundleName` and `CFBundleDisplayName` are set to `zwire` via `PlistBuddy` (Set, falling back to Add if the key is absent). These are what the Dock, the `⌘-Tab` switcher, and the menu bar read. Change them and the app stops calling itself “Chromium.”

2. **The icon, under a new filename.** The cyberpunk `branding/zwire.icns` is copied into the bundle's `Resources` as `zwire.icns`, and `CFBundleIconFile` is pointed at `zwire`. It is installed under a *new* filename rather than overwriting Chromium's own icon so it does not collide with the original icon's LaunchServices association — a fresh name is a fresh association.

3. **Deleting `CFBundleIconName`.** This is the non-obvious step, and the script comments it. Modern Chromium ships its icon as `AppIcon` inside a compiled `Assets.car` and references it through `CFBundleIconName`, which takes **precedence** over the `.icns` named in `CFBundleIconFile`. If you set `CFBundleIconFile` but leave `CFBundleIconName` in place, macOS keeps showing the compiled Chromium icon and your `.icns` is ignored. So the script deletes `CFBundleIconName`, forcing macOS to fall back to `zwire.icns`. Without this deletion the icon swap silently does nothing.

4. **The ad-hoc re-sign.** The base bundle is ad-hoc signed. Editing `Info.plist` and `Resources` invalidates that signature, and once the signature is invalid macOS treats the icon and name change as *tampering* and reverts to the cached original. The fix is to re-seal the bundle: `codesign --force --sign - "$APP"` re-signs it ad-hoc (the nested frameworks and helper apps keep their own valid seals, so only the top level needs re-signing). This is what makes macOS *accept* the change rather than roll it back. The script warns but does not fail if the re-sign fails, because a browser with a stale icon is still a working browser.

Finally it nudges LaunchServices to refresh its icon/name cache for this bundle (`lsregister -f "$APP"`) and *touches* the bundle so the change is noticed. The result: `~/zwire/base/Chromium.app` now presents as “zwire” with the cyberpunk icon, everywhere macOS shows an app's identity, without a single line of Chromium recompiled.

The order matters and the script encodes it: patch the plist and resources **first**, re-sign **second**, re-register **third**. Re-signing before the edits would seal the wrong state; re-registering before re-signing would cache a bundle macOS still considers tampered.

When the icon cache lingers: bust and nuke

macOS caches Dock and `⌘-Tab` icons aggressively by bundle, so occasionally the old Chromium icon or name lingers after a rebrand even when every step above ran. The dev-loop has a dedicated escape hatch for this: `scripts/bust.sh` (“cache-bust”) re-seals and re-registers *both* the base bundle and any `/Applications/zwire.app` wrapper with LaunchServices, forcing the cyberpunk icon and the “zwire” name to refresh immediately. `scripts/nuke.sh` is the heavier option — it removes the wrapper and the ~325 MB base, re-fetches from scratch, rebrands, and reinstalls, while preserving your profile. These exist because the LaunchServices cache is the one part of the rebrand macOS does not always honor on the first try, and a browser whose Dock icon is wrong reads as broken even when it is not.

Why an upgrade forces a re-run, and why Linux needs none

An *upgrade* forces a re-run of the rebrand: a fresh `fetch-base.sh` unzips a stock, Chromium-signed bundle over the top of the patched one, so every one of the four edits above is gone and must be reapplied. `scripts/install.sh` chains the rebrand after the fetch on macOS for exactly this reason, and the README’s update recipe ends with `rebrand-macos.sh` for the same one.

On Linux an application’s identity — the launcher entry, the taskbar name, the window title — comes from how it was invoked and from a `.desktop` file, not from metadata embedded in the binary. So on Linux the launcher *name itself* is the brand: the user runs `zwire`, and the process and window read `zwire` because that is what started them. There is no in-place bundle rewrite because there is no bundle to rewrite, and `install.sh` only invokes `rebrand-macos.sh` when `uname -s` is `Darwin`. “Rebrand” therefore means two different concrete things on the two platforms — a bundle patch on macOS, a launcher name on Linux — and the project does not pretend otherwise.

The four operations, as commands

The rebrand is four `PlistBuddy` and `codesign` operations in a fixed order, and the script’s comments give the reason for each. It resolves the `.app` by stripping `/Contents/MacOS/*` off `base.path`, then a `set_key` helper that tries `Set` and falls back to `Add ... string` writes `CFBundleName` and `CFBundleDisplayName` to `zwire`. It copies `branding/zwire.icns` into the bundle’s `Resources` under a new filename and points `CFBundleIconFile` at it, then deletes `CFBundleIconName` — the non-obvious step whose comment explains that modern Chromium references its icon as `AppIcon` inside a compiled `Assets.car` through that key, which “takes PRECEDENCE over the `.icns` in `CFBundleIconFile`,” so the key must go for the custom icon to win. It re-seals the bundle with `codesign --force --sign - "$APP"` (warning but not failing on error, “nested frameworks/helpers keep their own valid seals”), and finally re-registers with `lsregister -f` and `touches` the bundle. The order is load-bearing and the script encodes it: name and resources first, re-sign second, re-register third — re-signing before the edits would seal the wrong state, and re-registering before re-signing would cache a bundle macOS still considers tampered. `scripts/bust.sh` runs exactly the re-seal-and-re-register half against both the base bundle and any `/Applications/zwire.app` when the LaunchServices cache is stubborn.

Chapter 6 — The New-Tab HUD

`newtab/` is a `chrome_url_overrides.newtab` extension: the full HUD new tab — a neon clock, an omnibox, quick-launch tiles — with its fonts vendored locally so it renders offline and never phones a font CDN. This chapter walks the page, its palette module, and the scheme sync that keeps it in step with the rest of the HUD.

The page and its locally-vendored fonts

The new tab is a small MV3 extension whose whole purpose is one override: `chrome_url_overrides.newtab` points at `newtab.html`. When `zwire` opens a new tab, Chromium serves that page instead of Google's. The manifest pins a stable extension id via its embedded `key` (id `gpoep...gieb`), declares the `nativeMessaging`, `tabs`, and `history` permissions the palette needs, and lists its own files as web-accessible so they load in page context — including from the `tmux` overlay, which frames this page directly.

`newtab.js` is the page's logic, and it is deliberately dependency-free:

- **The clock.** A one-second `setInterval` writes a zero-padded `HH:MM:SS` time and an uppercased `WEEKDAY, MON DD, YYYY` date into the page. No library, just `Date` and `padStart`.
- **The quick-launch tiles.** A default set (GitHub, DuckDuckGo, MDN, crates.io, Hacker News, the docs site) renders as neon tiles, each a two-letter glyph plus a label. The set is overridable at runtime through a `zb.tiles localStorage` key holding an array of `{label, url}` objects, so the tiles are yours to edit without touching the extension.
- **The omnibox.** The page's own search field decides, per input, whether you typed a URL or a query. `looksLikeUrl` is a small heuristic: a scheme (`https://...`) is a URL; whitespace means a search; a bare `host.tld[/path]` or the literal `localhost` is a URL. A URL navigates directly (adding `https://` if you omitted the scheme); anything else goes to the search engine, default DuckDuckGo, overridable via a `zb.engine localStorage` key with a `%s` placeholder.
- **Reclaiming focus.** This is the page's least obvious and most necessary trick. On its own new tab, Chrome parks the text cursor in the `browser` omnibox and defeats the page's `<input autofocus>`, so the page never receives a single keystroke — which would kill both typing and the `⌘K` palette until you clicked in. `newtab.js` calls `q.focus({preventScroll:true})` in JavaScript, which moves `activeElement` to the page field where the attribute could not, and retries it across the first animation frames and again whenever the tab is re-shown. That is why the HUD new tab is typeable the instant it appears.

The fonts are **vendored locally** under `newtab/fonts/`, not linked from a CDN, and this is a design choice with two payoffs. First, the new tab renders correctly **offline** — the display face is present on disk, so a HUD new tab on a plane looks exactly like one on a fiber connection. Second, the page never makes a request to a font host on every new tab, which is both a privacy property (no third party sees a beacon each time you open a tab) and a latency one (no network round-trip in the render path of the most-opened page in the browser). The `fonts/LICENSE` file records the vendored faces' licensing. Vendoring costs a few hundred kilobytes in the extension; it buys an offline-correct, beacon-free, instant new tab, and for the page you open more than any other that trade is obviously right.

Palette, schemes, and scheme-sync

The new tab is a *different extension* from `hud-internal`, which means the global `⌘K` command palette that `hud-internal` injects into ordinary pages cannot reach it — a content script from one extension does not run in another extension's page. So `newtab/palette.js` carries its own copy of the same `ZGui.palette`, built on the shared `zgui-core` fuzzy engine, and because it runs as an extension page with the `tabs` permission it can call `chrome.tabs` directly with no worker round-trip. The palette it builds is substantial: dozens of `chrome://` internal pages (extensions, settings, history, downloads, flags, gpu, net-internals, and a long tail of the `-internals` diagnostics pages), the sub-pages inside `chrome://settings`, one entry per color scheme, plus your frecent history (scored by visit count and recency over the last 90 days) and your open tabs, all fuzzy-filtered in one list.

There is a wrinkle in how the palette opens, and it is the same `⌘K`-reservation problem as focus. On the new tab, Chrome reserves `⌘K` at the browser level, so no page `keydown` listener can catch it. The `hud-internal` background worker owns `⌘K` as a `chrome.commands` shortcut; when the active tab is the new tab it sends the new-tab page a `zwireOpenPalette` message (permitted because the new-tab manifest lists the HUD extension in `externally_connectable`), and `palette.js` toggles the palette in response. On an *ordinary* page `⌘K` and `:` are caught directly. So the palette opens the same way everywhere; the plumbing under it differs because the new tab is a privileged surface Chrome guards more tightly.

The colors are kept in step through `schemes.js`, `palette.js`, and `scheme-sync.js`. `schemes.js` is the canonical table of the eight HUD schemes — its full contents are Chapter 8 and Appendix D. `scheme-sync.js` keeps the new tab following the one scheme the user picked anywhere: because separate extensions do not share `chrome.storage`, the scheme travels through the native file the HUD host writes, and the new tab reads it so switching the scheme in the internal-HUD picker recolors the new tab too. `tmux-pane.js` adds a new-tab surface for the tiling overlay. The through-line of the chapter: the new tab is its own extension, so everything shared with the rest of the HUD — the palette, the eight schemes, the current scheme — has to be *re-carried* here and *re-synced* through the one channel two extensions can share, the native file.

The three support scripts

Behind the page sit three small scripts that make the new tab a full HUD citizen despite being its own extension. `palette.js` is the new tab's copy of the `⌘K` palette, and because it runs as an extension page with the `tabs` permission it calls `chrome.tabs` directly with no worker round-trip — its list carries the same forty-odd `chrome://` destinations, the nineteen settings sub-pages, one entry per scheme, and, appended asynchronously, your open tabs and your frecent history scored by the same 90-day formula the worker uses. It opens the same three ways the ordinary palette does; on the new tab specifically it responds to the worker's `zwireOpenPalette` message, permitted because the new-tab manifest lists the HUD worker in `externally_connectable`.

`scheme-sync.js` is how the colors follow. It cannot share `chrome.storage` with the internal HUD, so instead it polls the native host every 1.5 seconds (`{cmd: 'get'}` to `com.zwire.hud`), applies the returned scheme's variables to `:root`, and applies the returned UI preferences the same way — toggling the `no-scanlines/no-vignette/no-neon-glow/no-anim` classes (an effect is *on* when its class is *absent*, matching the shared effects toggle) and setting light or dark from `ui.light`. `tmux-pane.js` is the forwarder that lets the new tab be a `tmux` pane: when the page is framed as a pane (its window name is `zbtmux`), the overlay lives in the parent frame and this script catches the prefix and command keys and posts them up, mirroring the pane-side half of `zbtmux.js` so the tiling overlay can frame the new tab as reliably as any site.

Why the focus trick needs four attempts

The focus-reclaim is worth one more sentence than Chapter 6 gave it, because the retries are not superstition. Chrome parks the cursor in the *browser* omnibox and overrides the page's `autofocus` after the page

has already run, so a single `focus()` call at load loses a race it cannot see. `newtab.js` therefore calls `q.focus({preventScroll:true})` immediately, again on the first animation frame, and again at 60 and 200 milliseconds, and once more on every `visibilitychange` when the tab becomes visible — covering the load race, the post-layout override, and the “switched back to an already-open new tab” case. It also binds a capture-phase `keydown` directly on the field that swallows `⌘K` (stopping the macOS beep and keeping the field from seeing the key) and calls the palette opener, so the palette works even in the instant before focus has fully settled.

Web-accessible, so the overlay can frame it

One manifest detail earns the new tab its place in the tmux overlay. Fresh tmux panes open the zwire new tab, but `chrome://newtab` cannot be put in an `iframe` — `chrome://` pages are un-embeddable by design — so the overlay frames the new-tab *extension's* page directly, at its stable `chrome-extension://gpoep...gieb/newtab.html` URL. That only works because the new-tab manifest lists its own files as `web_accessible_resources` for `<all_urls>`, which is what lets another page load them in an `iframe`. It is the same page `chrome://newtab` redirects to, reached by its real extension URL instead of the un-framable alias. The manifest's embedded `key` pins that URL to a stable id across installs, so the overlay's hardcoded new-tab address never goes stale — a small manifest choice that is load-bearing for the tiling overlay two chapters later.

Chapter 7 — The Internal-HUD Extension: The Injection Model

`extensions/hud-internal` is the largest layer on the runtime path, and it is worth understanding its *shape* before the feature chapters open each script. The extension is an MV3 background worker plus a carefully ordered stack of content scripts, a set of HUD replacement pages, a native host, and the shared `zgui-core` library. This chapter is the map; Chapters 8–16 are the territory.

The manifest as a wiring diagram

The manifest declares a broad permission set — `storage`, `nativeMessaging`, `management`, `tabs`, `webNavigation`, `history`, `downloads`, `bookmarks`, `sessions`, `alarms`, `notifications`, `developerPrivate`, `settingsPrivate`, `system.display`, and `declarativeNetRequest` — plus `<all_urls>` host access. Each permission earns a feature: `management/developerPrivate` power the HUD extensions page, `history/bookmarks/downloads/sessions` power the HUD replacements for those internal pages, `system.display` feeds window math, and `declarativeNetRequest` carries the `frame_bust` rule that lets the `tmux` overlay iframe any site.

The content scripts are registered in several blocks, and the *order within a block* is load-bearing because later scripts depend on globals earlier ones set. The canonical block loads, in order: `schemes.js` (defines `window.ZWIRE_HUD` with the palette table), the `zgui-core` `util.js/fzf.js/command-palette.js` (defines `window.ZGui`), then `zkeys.js` (the keybinding registry `window.ZWIRE_KEYMAP`), `cmd-defaults.js`, `zpalette.js`, and `zvim.js`. By the time `zvim.js` runs it can read both the keymap and the palette; by the time the palette runs `ZGui` exists. Every script also guards against double-injection with a `window.__zb*Loaded` flag, because content scripts can be injected more than once across frame and navigation events.

Two treatments for `chrome://` pages, one switch that enables both

`chrome://` pages are privileged, and by default no extension content script runs on them — that is why `zwire` launches with `--extensions-on-chrome-urls`, the switch that lets `hud-internal`'s content scripts match `chrome://*/*` at all. With that switch set, the manifest wires two different treatments for internal pages:

- **Redirect.** For a handful of high-traffic pages — `chrome://extensions`, `chrome://settings`, `chrome://history`, `chrome://bookmarks`, `chrome://version` — a `document_start` content script (`redirect.js`) sends you to a full HUD replacement served from the extension's own `pages/` directory. These are not restyles of Chrome's page; they are purpose-built HUD pages that reimplement the surface using the extension's privileged permissions. Chapter 14 is the tour.
- **Skin.** For every *other* `chrome://` page (the ones without a bespoke replacement — flags, discards, dns, password-manager, and the rest), a `document_start` pair — `schemes.js` then `theme.js` — restyles the page in place with the active scheme's palette. Chapter 8 covers what `theme.js` maps.

The excluded set is exactly the redirected pages plus the new-tab URLs (which the new-tab extension owns), so the two treatments never fight over the same page.

Where each feature lives

The rest of the internal HUD is a set of content scripts and their support files, each of which gets its own chapter:

```
schemes.js      the 8-scheme palette table (window.ZWIRE_HUD)      -> Ch 8
theme.js        the chrome:// WebUI-token skin          -> Ch 8
background.js   the service worker: scheme bridge, tab bus, term flag -> Ch 8,12,16
zvim.js         vim keybindings on every page         -> Ch 9
ztmux-config.js the tmux overlay provider (shared ZGui.tmux WM) -> Ch 10
ztmux-pane.js   the tmux pane-side forwarder (all frames) -> Ch 10
zpowerline.js   the powerline statusbar adapter (shared ZGui.powerline) -> Ch 11
zpalette.js     the global %K command palette             -> Ch 12
zkeys.js        the keybinding registry (single source of truth) -> Ch 9,12
cmd-defaults.js the seeded %K keyword commands          -> Ch 12
zfind.js        the fuzzy find bar (replaces Ctrl+%F)    -> Ch 13
pages/          the HUD internal-page replacements      -> Ch 14
native/zwire-host the native host (scheme/ui/sysinfo/PTY/fs/exec/kv) -> Ch 15
lib/term/       the xterm terminal over the native PTY   -> Ch 16
lib/zgui-core/  the shared cyberpunk UI toolkit (submodule) -> Ch 17
```

That is the whole internal HUD in one view: a manifest that wires an ordered content-script stack against every page, two treatments for `chrome://`, a background worker that coordinates across tabs, a native host for the machine, and a shared toolkit under it. The feature chapters take them in that order.

The seven injection blocks, in order

The manifest's `content_scripts` array is seven blocks, and the split is not cosmetic — each block has its own match set, run timing, and frame scope, chosen so the right script runs on the right surface at the right moment. Read top to bottom:

#	matches	run_at	all_frames	scripts
1	chrome://{5 redirected}	document_start	no	redirect.js
2	chrome://{*/} (minus above)	document_start	no	schemes.js, theme.js
3	http/https/file/chrome://	document_idle	no	schemes.js, fzf.js, zfind.js
4	http/https/file/chrome://	document_start	no	schemes.js, zgui util/fzf/ command-palette, zkeys.js, cmd-defaults.js, zpalette.js, zvim.js
5	http/https/file/chrome://	document_idle	no	schemes.js, powerline.js, zpowerline.js (statusbar)
6	http/https/file	document_idle	yes	ztmux-pane.js (pane forwarder; top frame is ZGui.tmux + ztmux-config.js)
7	http/https/file/chrome://	document_idle	no	xterm + terminal + boot

Three things fall out of the table. Block 1 fires first and only on the five redirected `chrome://` pages, at `document_start`, so the redirect happens before Chrome's own WebUI paints; block 2 skins every *other* `chrome://` page, also at `document_start`, and its exclude set is exactly block 1's match set plus the new-tab URLs, so the redirect and the skin never fight. Block 4 — the keyboard and palette stack — runs at `document_start` and in a fixed internal order, because later scripts read globals earlier ones set: `schemes.js` defines the palette table, the `zgui-core` trio defines `ZGui`, `zkeys.js` defines the keymap, and only then do `zpalette.js` and `zvim.js` load able to read both. Block 6 is the only one with `all_frames: true`, because the tmux overlay must run its forwarder inside every framed pane, not just the top document. Every script also carries a `window.__zb...Loaded` guard, because content scripts can be injected more than once across an SPA's frame and navigation events, and a second injection must be a no-op.

The permission set reads as a feature list once you know which API powers which surface: `management` and

`developerPrivate` back the extensions page and the shortcut editors; `history`, `bookmarks`, `downloads`, and `sessions` back the four list pages; `settingsPrivate` backs the settings page; `system.display` feeds the worker's window-tiling math; `notifications` and `alarms` power the CI monitor; `browsingData`, `tabGroups`, `readingList`, and `power` back the automation bus's data-clearing, tab-grouping, reading-list, and keep-awake verbs; `declarativeNetRequest` carries the single `frame_bust` rule that lets the tmux overlay iframe any site; and `<all_urls>` host access is what lets the content-script stack run everywhere. The `externally_connectable` list holds two ids — the power-tool and the new-tab page — the only extensions permitted to message the worker from outside; the new-tab page uses it to bridge its palette into the automation bus described next.

The automation bus: 161 verbs through one executor

The HUD is not only keyboard-driven; it is scriptable. A stryke script — run from a terminal, from a page's ⌘K palette, from the new-tab palette, or from a HUD page — reaches the browser through `zwire-host` and drives it over a semantic action bus. The surface is 161 verbs, of which 99 are `browser.*` verbs that manipulate the browser itself; the rest are the host's own file, process, and key-value commands.

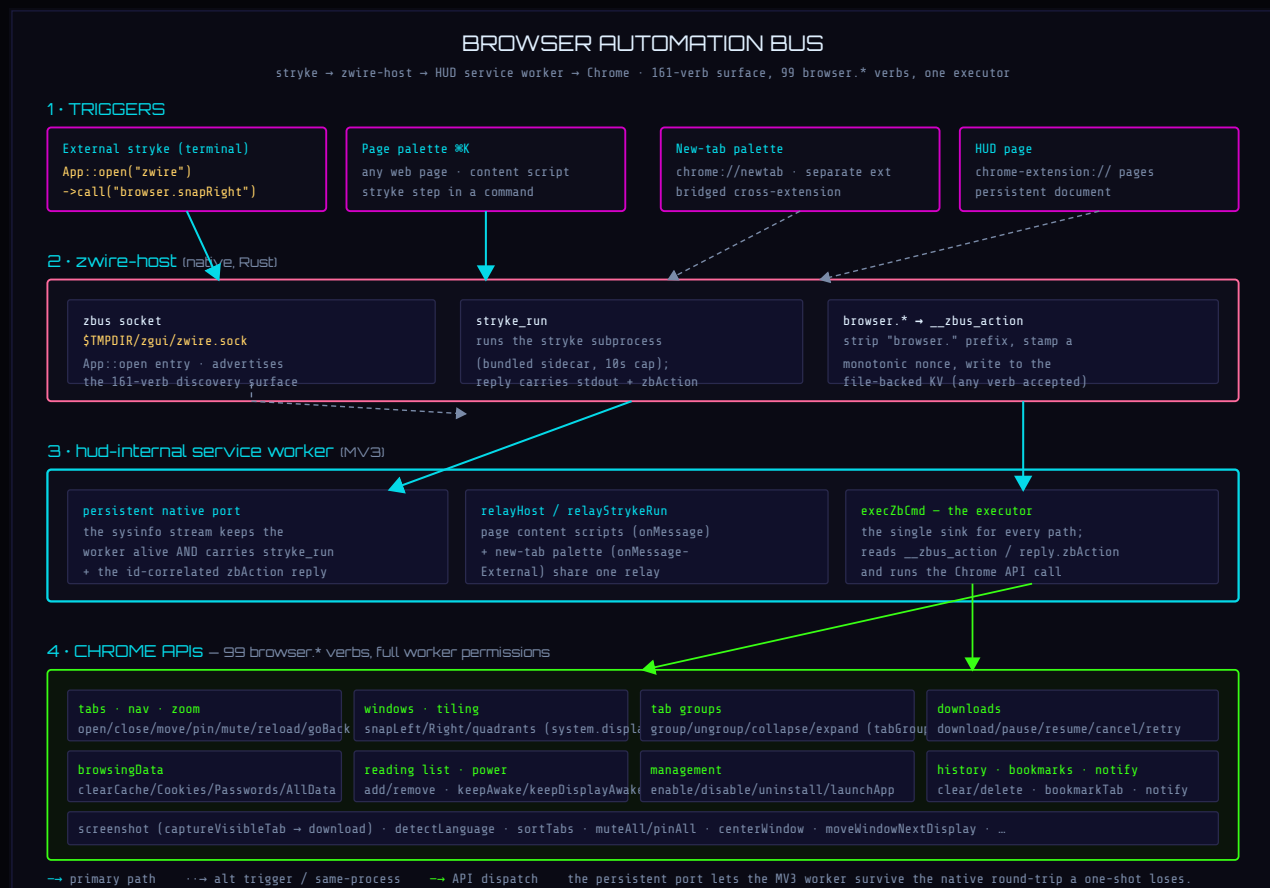


Figure 1: zwire's browser automation bus: a stryke trigger reaches zwire-host, which runs stryke and stamps a browser.* action into a file-backed KV; the MV3 service worker's execZbCmd, kept alive across the native round-trip by the persistent sysinfo port, is the single executor that runs the Chrome API call across the 99-verb browser surface.

The path has four stages. A trigger runs stryke code containing `App::open("zwire")->call("browser.<verb>")`. `zwire-host` runs the stryke subprocess and, for any `browser.*` verb, strips the prefix and stamps `{a:<verb>, ...args, _n:<nonce>}` into a file-backed key-value entry — it accepts

any verb, so the host never needs the verb list: discovery is advisory, execution is open. The MV3 service worker is the single executor. Its `execZbCmd` reads that action — or the `zbAction` piggybacked on a `stryke_run` reply — and issues the matching Chrome API call, with the worker's full `tabs`, `windows`, `downloads`, `bookmarks`, `history`, `tabGroups`, `browsingData`, `readingList`, and `power` permissions behind it.

The hard part is MV3's service-worker lifecycle. A one-shot native message issued from a page's content script loses its reply, because the worker is torn down during the roughly 200-millisecond native round-trip — external pages saw a zero-percent success rate. The fix is the persistent native port that already streams system stats: it keeps the worker alive and delivers the reply on an already-open channel, correlated by a request id so the browser action fires exactly once. The new-tab page — a separate extension whose content scripts are excluded from `chrome://newtab` and whose storage is isolated from the worker's bus — bridges its palette's host-backed steps into the same executor over `externally_connectable`, so `execZbCmd` stays the one place a browser action is ever run, whichever surface asked for it.

The 99 browser verbs span the browser's own surface: tab lifecycle and navigation, page reload and history motion, per-tab zoom, window state and — using `system.display`'s work area — left/right/quadrant tiling and multi-monitor moves; tab grouping; the full download lifecycle; browsing-data clearing; the reading list; keep-awake; extension and app management; and history, bookmark, and notification actions. Because the executor is one function and the host forwards any verb, adding a capability is a single `execZbCmd` branch plus a single discovery-list entry — the surface grows without a new transport.

Chapter 8 – The Eight Schemes and the Sync Bus

The HUD has eight color schemes — cyberpunk, midnight, matrix, ember, arctic, crimson, toxic, vapor — and every surface in the browser follows the one the user picks: the ordinary-page content scripts, the `chrome://skins`, the separate new-tab extension, the `zpw:chrome` power-tool, and (on the fork) the native color mixer. This chapter is the palette table, the token map that applies it to Chrome’s own pages, and the cross-extension bus that keeps them all agreeing.

The palette table

`schemes.js` is the canonical scheme table, `window.ZWIRE_HUD`, and it is the same file loaded by every content script and by the new-tab extension — auto-extracted from the same `hud-theme.js` the wider MenkeTechnologies app suite uses, so the browser cannot drift from the apps. It defines three things: an ordered list of twenty CSS custom-property keys (`SCHEME_VAR_KEYS`), the scheme order (`SCHEME_ORDER`), and the schemes themselves (`COLOR_SCHEMES`). Each scheme has a `label`, a one-line `desc`, a dark `vars` set, and a parallel `lightVars` set:

```
cyberpunk: {
  label: 'Cyberpunk', desc: 'Hot pink + cyan (default)',
  vars: {
    '--accent': '#ff2a6d', '--cyan': '#05d9e8', '--magenta': '#d300c5',
    '--green': '#39ff14', '--red': '#ff073a',
    '--text': '#e0f0ff', '--text-dim': '#7a8ba8',
    '--bg-primary': '#05050a', '--bg-card': '#0d0d1a', '--border': '#1a1a3e',
    /* ...20 keys total, plus the glow/dim variants... */
  },
  lightVars: { /* the same 20 keys, tuned for a light background */ },
}
```

The twenty keys are the HUD’s whole vocabulary: three accent shades, three cyan shades, two magenta, a green pair, a red, three text weights, four background depths, and two border shades. Every HUD surface — the statusbar, the tmux panes, the palette, the find bar, the internal pages — styles itself entirely in terms of these variables, so recoloring the browser is just swapping which scheme’s values are written to `:root`. The eight schemes’ full dark-and-light palettes are Appendix D; the point here is that they are *data*, one table, shared by every consumer, with a matched light variant so the HUD works on a light background too.

Applying the palette to Chrome’s own pages

`theme.js` is the script that maps the active scheme onto Chrome’s internal pages — the `chrome://` pages that have no bespoke HUD replacement. Its comment is careful about what it is *not*: it deliberately drops all the HUD decoration (the grid lattice, CRT scanlines, neon glow, the floating picker) and maps *only* the scheme’s palette onto Chrome’s WebUI color tokens, plus the mono font, so these pages simply inherit the same theme the rest of the browser uses — no HUD chrome on top. The mechanism is a stylesheet that rewrites Chrome’s own CSS custom properties to point at the HUD variables:

```

:root, :host {
  --color-sys-surface: var(--bg-primary) !important;
  --color-sys-on-surface: var(--text) !important;
  --color-sys-primary: var(--cyan) !important;
  --color-sys-secondary: var(--accent) !important;
  --color-sys-outline: var(--border) !important;
  /* ...plus the --cr-fallback-color-* and --cr-*--background-color families... */
}

```

Chrome's WebUI is built on these `--color-sys-*` and `--cr-*` tokens, so overriding them with `!important` recolors the page's own components without touching their markup. The scheme `theme.js` reads the native source of truth (`~/ .zwire/hud-scheme`), fetched via the background worker — a content script cannot call `sendNativeMessage` itself, so it reads the mirrored value the worker maintains.

The sync bus: how five consumers agree on one scheme

The hard part is not styling one surface; it is keeping every surface agreeing on one value across three isolation boundaries — content scripts vs the worker, extension vs extension, and the browser vs the native chrome. `hud-internal` solves it with a two-tier store plus a native file as the ultimate source of truth:

- **Content scripts ↔ worker.** The background worker mirrors the current scheme into `chrome.storage.local` under `zb_scheme`, and the content scripts read and observe that key. The worker uses storage rather than an async native round-trip on purpose, and the code comments say why: an MV3 worker can suspend mid-round-trip, so a `sendResponse` across the native host is unreliable, while a storage write always lands.
- **Extension ↔ extension ↔ native chrome.** The scheme travels through `native/zwire-host` and the scheme file it owns. When a page or the picker sets a scheme, the worker relays it to the host (which atomically writes the file) and writes `zb_scheme`. On startup it seeds `zb_scheme` from the file, which may already hold a scheme chosen in a prior session. The new-tab extension reads the same file (the one channel two extensions can share), and on the fork the native color mixer reads it too.
- **The `zpwchrome` bridge.** Because separate extensions cannot share storage, the worker also pushes the scheme to `zpwchrome` over `runtime.sendMessage`, with a guard (`lastPushed`) so a scheme that came from `zpwchrome` is not echoed straight back into a loop. `zpwchrome` can also pull the current scheme on startup (`zb-scheme-get`) and push a user's pick back (`zb-scheme-set`).

A companion file, `~/ .zwire/hud-ui.json`, carries the shared visual-effect and light-mode preferences (light mode, scanlines, vignette, glow, animation) the same way, since those are per-origin `localStorage` on each page and the native file is the only channel that reaches the other extensions. The whole design is one value with one source of truth (the native file) and several caches that all reconcile to it — which is why picking “matrix” in the floating picker recolors the statusbar, the tmux panes, the find bar, the new tab, the `chrome://` pages, and the power-tool, all at once.

The fleet-wide single source: `zwire-host` and `global.toml`

The per-extension mesh above — `chrome.storage` mirrors plus cross-extension `sendMessage` pushes — is fragile: two surfaces writing at once (a scheme change here, a light toggle there) could desync `newtab`, `zpwchrome`, and the HUD. Newer builds move the *source of truth* off that mesh and onto the native host. The scheme, the light-mode flag, the visual-effect prefs, and any user-defined custom schemes now live in one app-independent file the Rust `zwire-host` owns — `~/ .zwire/global.toml` — read and written by every `zwire-host` client, not just the browser but the wider MenkeTechnologies fleet (Audio-Haxor, `zmax`, `zpw-daw`, and the rest), so a scheme picked in any app repaints all of them. `newtab` and `zpwchrome`

subscribe the host bus directly (a snapshot on connect plus live updates), and the HUD background folds a host theme subscription into its sysinfo port and bridges it to `chrome.storage` with an echo guard — retiring the old push/pull/mirror/seed mesh. This also fixed a real race: two concurrent writes (`{scheme}` and `{ui}`) could clobber each other and silently revert the scheme, so the host now serializes the read-modify-write of `global.toml` behind a cross-process lock (`global.toml.lock`). The legacy `hud-scheme` and `hud-ui.json` files are no longer the bus; the host still writes a plain-text `hud-scheme` projection (and, for light mode, a `hud-light` projection) *beside* `global.toml` purely so the fork’s C++ color mixer — which cannot parse TOML at paint time — has a trivial file to read. `zwire-host` also keeps a shared command log, `~/.zwire/hostlog.jsonl`, recording every message it sends and receives (the sysinfo stream included), which the HOST internal page reads back through a `hostlog` command.

Light and dark are part of the same single source. Every HUD surface can toggle light mode — the `⌘K` palette carries a “Toggle light mode” command, the shared command palette from `zgui-core` adds its own light/dark entries, and the app-shell settings modal has a switch — and each writes the light flag through to `global.toml` exactly the way a scheme change does, so light toggled from any surface flows to the one file and every renderer follows. On the fork, the native chrome follows too: the toolbar, tabstrip, and frame read the `hud-light` projection and repaint on launch, so the whole browser, native chrome included, honors one light/dark choice (Chapter 21).

The background worker’s full job list

The scheme relay is one of a dozen things `background.js` does. It is a single MV3 service worker — no persistent page, woken by events and suspended when idle — and it is the coordinator every content script leans on for anything a page cannot do itself. Its own header comment names only the first two jobs (relay the scheme to the native host, mirror it into `chrome.storage.local`), but reading the file top to bottom the worker turns out to own nine distinct responsibilities, each wired to its own set of Chromium events:

- **The scheme fan-out** (Chapter 8’s subject): `mirror(scheme)` writes `zb_scheme` and calls `pushToZpwr`, which relays to the power-tool over `runtime.sendMessage` with the `lastPushed` echo guard.
- **The native seed on startup**: `seedFromNative` sends `{cmd: 'get'}` to `com.zwire.hud` and mirrors whatever scheme the file already held, so a scheme picked in a previous session survives a browser restart. It is registered on both `onStartup` and `onInstalled`, and called immediately, because the file “may already hold a scheme picked in a prior session or written before launch.”
- **The `⌘K` command owner**: the worker registers `open-palette` as a `chrome.commands` shortcut (`Command+K` on macOS, `Ctrl+K` elsewhere) and routes it to whichever palette matches the active tab — the content-script palette on an ordinary page (`tabs.sendMessage`), the new-tab extension’s palette (cross-ext `sendMessage`), or a HUD page’s own palette (a runtime broadcast). Its comment spells out why the browser command must be the single owner: the new-tab page reserves `⌘K` at the browser level before any page JS sees it, so a page listener could never open the palette there, but a `chrome.commands` shortcut fires regardless of focus — at the cost of consuming `⌘K` on every tab, which is why the routing lives here.
- **The sysinfo stream** (Chapter 11): a persistent `connectNative` port that sends `sysinfo_start`, writes each `{sys: {...}}` frame into `zb_sys`, and reconnects five seconds after any disconnect. The comment notes a second reason the port is held open: it keeps the MV3 worker itself alive.
- **The per-tab PTY relay** (Chapter 16): the `zwire-pty` connection handler, covered in its own section below.
- **Four storage mirrors**: the worker keeps `zb_tabs` (every open tab), `zb_exts` (enabled extensions, via `chrome.management`), `zb_shortcuts` (every extension’s keyboard shortcuts, via `developerPrivate` — `chrome.management` does not expose commands), and `zb_frecent` (a frequency-scored history list) continuously fresh, each wired to the relevant Chromium event (`tabs.onUpdated`, `management.onInstalled`, `history.onVisited`, and so on). These exist because a content-script palette cannot call `chrome.tabs` or `chrome.management` itself, and — the comment is explicit — “an async worker round-trip” from a page is unreliable when the worker may be asleep, so the worker publishes to storage eagerly and the page reads storage directly.

- **The custom-command seed:** `seedCustomCmds` populates the `%%K` keyword-command registry once, guarded by a `zb_cmds_seeded` flag, “merging defaults with any existing user entries” so a user’s edits and deletions are never clobbered. (Chapter 12 and Appendix H.)
- **The CI monitor:** a GitHub-Actions poller on a `chrome.alarms` heartbeat, covered below.
- **A second tiling engine:** a `chrome.windows`-geometry tmux implementation that tiles *real OS windows* as panes, driven from the same `zb_cmd` bus, distinct from the in-page iframe overlay of Chapter 10.

The frequency list is worth pinning down because the same formula appears in three places (the worker, the new-tab palette, and the HUD pages’ `zg-boot.js`): a `chrome.history.search` over the last 90 days, scored $(\text{visitCount} + 2 \cdot \text{typedCount}) / (1 + \text{ageDays} \cdot 0.3)$, `chrome://` URLs filtered out, top 30 kept. A typed visit counts double a clicked one, and the recency divisor decays a site’s score by roughly a third per day of age, so a page you typed yesterday outranks one you clicked ten times last month.

The storage command bus

The recurring problem the worker solves is that an MV3 service worker can be suspended at any instant, and a `runtime.sendMessage` to a sleeping worker is not guaranteed to wake it in time. The HUD’s answer, used by vim, the palette, and the tmux overlay alike, is the **storage command bus**: a content script writes a `zb_cmd` object into `chrome.storage.local`, and `storage.onChanged` is a reliable MV3 wake-up, so the worker always runs the command. The worker’s comment states the contract directly — “content scripts write `zb_cmd` to storage; `storage.onChanged` is a reliable MV3 wakeup (unlike `sendMessage` to a sleeping worker), so palette navigation / tab-switching always executes.”

A `zb_cmd` carries an action field `a` and a monotonic tag `n` (so a repeated identical command still registers as a change), plus any parameters. The worker’s `onChanged` handler dispatches on `c.a` across a broad tab-and-window vocabulary: `open/openTab` (navigate or new tab), `activate` (switch to a tab and focus its window), `newTab/newWindow/duplicateTab/reopenTab`, `pinTab/muteTab`, `closeTab/closeOthers`, `nextTab/prevTab`, `tmux` (handed to the worker’s own tiling engine), and `ping` (a wake-and-refresh that re-runs all four storage mirrors). This is why a vim `x` can close the current tab and a palette selection can switch to another window even though neither the vim script nor the palette script can touch `chrome.tabs`: they write one storage key and the worker, woken by the write, does the privileged work. The same `onChanged` handler also fans a `zb_scheme` write out to the power-tool, so a scheme picked on the Settings page — which writes storage and the native file but never messages the worker — still reaches `zpw:chrome`.

The per-tab PTY relay, keyed by tab

The terminal’s bytes flow through the worker because a content script cannot open a native-messaging port that survives navigation. The worker exposes a `zwire-pty runtime.onConnect` port and keeps a `ptySessions` map **keyed by tab id**, so — the comment says — “the shell SURVIVES page navigation.” Each session holds its own `connectNative` port to `com.zwire.hud`, the content-script port currently attached (or `null`), an output buffer, and a `spawned` flag. Three details make it robust: output produced while no page is attached is buffered (capped at 500 messages) and flushed on reconnect; `pty_spawn` is honored only once per tab (`if (sess.spawned) return`), so a re-injected content script re-attaches to the running shell instead of forking a second one; and a content-script disconnect sets `sess.cs = null` rather than killing the native port, “to keep the shell alive across nav,” while an actual `tabs.onRemoved` tears the native port down. The result is a shell session bound to a browser tab, not to a page — you can click a link, load a new site, and keep typing in the same zsh.

The CI monitor: a poller, an alarm, and a desktop notification

The one worker feature with no page-facing counterpart in the earlier chapters is a continuous-integration monitor. On a `chrome.alarms` heartbeat named `zb-ci-poll` (`periodInMinutes: 10`) the worker runs `pollCi`, which walks your GitHub repositories through the REST API — `/user/repos` with a token, else `/users/<user>/repos` for the default account `MenkeTechnologies` — fetches the latest workflow run per repo, and tallies failing, running, and passing counts into a `zb_ci_status` object the statusbar can show as a dot. The request budget is tuned to GitHub's anonymous cap: the comment “stay under the 60/hr anon cap” sits next to a repo limit that rises from five to twelve only when a token is present. When the failing count rises above the previous poll, and `chrome.notifications` exists, the worker raises a desktop notification titled “CI failing” whose body lists the first four broken repos, drawn with a 16-pixel hot-pink `CI` tile inlined as a base64 SVG “so notifications never depend on an icon file.” This is the only `chrome.alarms` use in the extension, and the only place the HUD reaches out to a network service on its own initiative; the CI page of Chapter 14 is its interactive front end, and the two share the `zb_ci` config key.

The other tmux: tiling real OS windows

There is a second tiling engine in the worker, and it is easy to miss because it shares the `tmux` vocabulary with the in-page overlay of Chapter 10 while doing something entirely different. The overlay tiles *iframes inside one page*; the worker's engine tiles *real Chromium windows* by geometry. Its own comment states the model: “PANES = real browser windows, tiled by `chrome.windows` geometry. WINDOWS (tmux) = named groups of panes; only the active group is visible, the rest are minimized. SESSIONS = the whole thing, persisted to storage.” State lives in a `TMUX` object mirrored to `zb_tmux_state` so it survives a worker eviction, and the tiling area comes from `chrome.system.display`'s primary work area (falling back to 1440×820 when the display API is unavailable).

The engine is driven from the same `zb_cmd` bus (`{a:'tmux', sub:...}`) and its subcommands mirror tmux: `split` (down splits into rows, else columns), `navigate`, `zoom`, `closePane`, `newWindow`, `nextWindow/prevWindow`, `selectLayout` (cycling `cols`, `rows`, `main-v`, `grid`), `syncToggle`, `killWindow`, and `retile`. `ArectsFor(n, layout)` helper computes the pixel rectangles — a 60% main pane for `main-v`, a $\lceil\sqrt{n}\rceil$ -column grid otherwise — and `tile` minimizes every window not in the active group, positions the rest, and focuses the active one. Because real windows can be closed or focused outside the engine's knowledge, a `reconcile` step drops closed panes and prunes empty groups on `windows.onRemoved` and `onFocusChanged`, and an `ensureInit` rebuilds the state from `zb_tmux_state` after an eviction, defaulting to “the current window is the sole pane.” The engine even has its own synchronize-panes: a `zbSync` message fans a keystroke out to the active tab of every sibling window in a syncing group. It is a genuine window manager for your browser windows, living in the service worker, distinct from and parallel to the in-page overlay — two answers to “tmux in the browser,” one tiling pages and one tiling windows.

The color science under the eight names

The eight schemes are not eight arbitrary palettes; they are eight *coordinated* ones, and the coordination is what lets a single content script skin an arbitrary `chrome://` page without auditing it first. Each scheme is a small closed set of roles — a background, a raised surface, a foreground, a dimmed foreground, an accent, and a warning — and the skinning layer paints roles, not pixels. That indirection is the reason the same `theme.js` that colors `chrome://history` also colors `chrome://settings` and every HUD internal page: the page asks for `--zw-accent`, not for `#00e5ff`, and the scheme table decides what the accent is this minute. Change the scheme and every surface that referenced a role moves in lockstep, because there was only ever one place the literal color lived.

The dark schemes share a discipline that keeps neon legible against black, which is the hard case a cyberpunk

palette must survive. A saturated cyan or magenta at full value on a near-black field is the classic mistake — it blooms, it strobes on LCD subpixels, and long-form text set in it is unreadable. The schemes avoid it by never using the accent as body text: the accent lives on borders, the active tab, the statusbar segments, the cursor, and the focus ring, where a thin run of high chroma reads as *signal* rather than as a wall of glow, while the body foreground is a high-value near-white or near-cyan held well below full saturation. The eye reads the accent as an edge and the foreground as a plane, and the two never compete. This is why a scheme can be maximal in name — `matrix`, `synthwave`, `cyberpunk` — and still be a working reading surface: the loud color is rationed to the geometry, not spent on the prose.

The light schemes are the harder engineering, and the book is honest that they came later. Inverting a dark cyberpunk palette does not produce a usable light one — the same cyan that reads as signal on black is nearly invisible on white, and a magenta that glows on a dark field turns muddy on a bright one. The light variants therefore re-derive each role against a light ground rather than flipping a value: the background rises to a warm off-white, the accents drop in value and rise in saturation so they still carry against the paper, and the dimmed foreground — the one role most likely to fail a contrast check — is pinned to a value that clears the readable threshold rather than being computed as a fixed alpha over the background, because a fixed alpha is exactly what fails when the background changes from black to white. The fork's native color mixer reads the same role names from the host's `hud-light` projection, so the eight-times-two matrix stays a single source: eight names, a dark and a light derivation of each, one set of roles, and one table that every surface — content script, new tab, fork chrome, and DevTools dropdown — reads through. The palette is not decoration bolted onto the browser; it is a small, closed color system, and the eight names are its presets.

Chapter 9 – Vim on Every Page

`zvim.js` puts real vim keybindings on every page: normal mode by default, suppressed while you are typing in an editable field, with motions, link hints, marks, tab operations, and a per-tab off switch. This chapter is the whole keyboard model, and it pairs with `zkeys.js`, the shared registry that makes every key remappable.

Normal mode, and never stealing keys

The script installs one capture-phase `keydown` listener on the document and decides, per key, whether it owns it. The first rule is restraint: if the active element is editable (an `input`, `textarea`, `select`, or `contentEditable`), vim does nothing except let `Escape` blur the field — so typing in a search box or a comment box is never intercepted. With any of `Alt/Meta/Ctrl` held (except its own combos) it also bows out. And crucially, an *unknown* key is left alone: the listener only calls `preventDefault` when the key maps to a real action, so keys vim does not claim reach the page normally. When it *does* act, it calls both `preventDefault` and `stopImmediatePropagation`, so a single-key site shortcut (a Gmail `j`, say) does not also fire.

Motions, screens, and the z-family

The core motions are the vim ones, each a small `scrollBy/scrollTo`:

- `j/k` scroll down/up, `h/l` scroll left/right (66px steps), `d/u` half-page down/up.
- `gg` (a two-key sequence via the `g` prefix) jumps to the top; `G` to the bottom.
- `H/M/L` jump to the high/middle/low of the *document* — the vim screen motions, mapped here to top/middle/bottom of the page.
- The `z`-family (`zt/zz/zb`, via the `z` prefix) positions a reference element to the top/center/bottom of the viewport. The reference is the focused element if it is real and on-screen, else whatever is at the center of the window — computed with `getBoundingClientRect` and a `scrollBy`.

Two-key sequences are handled with a small pending-key state and a 900ms timeout: pressing `g`, `z`, `y`, `m`, or ``` arms a prefix, and the next key completes it (or the prefix expires). This is how `gg`, `gt/gT` (tab next/prev), `gi` (focus the first input), `yy` (yank the URL to the clipboard), `m<x>` (set a mark), and ``<x>/'<x>` (jump to a mark) all work from single-character keys without colliding.

Link hints

`f` and `F` enter hint mode. The script collects every clickable element in the viewport — `a[href]`, `button`, form controls, `[role=button]`, `[onclick]`, `[tabindex]` — filters out the invisible and off-screen ones, and overlays a short letter label on each (one letter when there are few targets, two-letter combinations when there are many, drawn from a home-row-first alphabet). You type the label; a unique match activates immediately. `f` clicks in place; `F` opens links in a new tab (routed through the background worker). Editable targets get focused rather than clicked, and `Escape` or a no-match cancels. Highlighting the labels is a fixed-position overlay themed in the HUD yellow, with the active filter shown in cyan.

Tabs, history, marks, and the storage bus

Tab operations cannot be done from a content script directly — a page cannot close or switch its own tab — so they route through the background worker over what the code calls the storage command bus: `zvim.js` writes a `zb_cmd` object into `chrome.storage.local`, which reliably wakes the MV3 worker (more reliable than a `runtime.sendMessage` to a suspended worker), and the worker performs the tab action. `J/K` switch tabs, `x` closes, `t` opens, `r` reloads, `[/]` are back/forward. Marks are per-page scroll positions kept in memory. A small toast in the corner confirms actions like a yank or a mark set.

The keybinding registry and remaps

Every default key above lives in `zkeys.js`, `window.ZWIRE_KEYMAP` — the single source of truth, loaded both as a content script (before `zvim/zpalette/zfind`) and by the Keyboard settings page so both render and dispatch from the same list. It is a categorized table: Scroll, Tabs & history, Jump & hints, Palette & find, and a large Tmux category (every post-prefix key), plus global chorded hotkeys (`⌘K`, `⌘F`) and the fixed native prefix. User remaps live in `chrome.storage.local` under `zb_keys` as `{action: key}`; `zvim` builds its `key` → `action` map by merging those over the defaults and rebuilds it live when the key changes. So rebinding `scrollDown` from `j` to `s` in the settings page immediately changes what `zvim` fires — and, because `ztmux.js` reads the same registry, the same mechanism remaps the tmux post-prefix keys too. `Ctrl/⌘+\` toggles vim off for the tab (and back on), for the moments you want the page's raw keys.

The handler, in the order it decides

The whole of `zvim.js` is one capture-phase `keydown` listener on the document — registered with the `true` third argument so it sees keys before the page does — and reading it is reading a sequence of guards in a deliberate order. First, if vim is disabled for the tab, only `Ctrl/⌘+\` (which re-enables it) is honored and everything else returns. Second, hint mode: while link hints are on screen, the listener routes the key to the hint filter before any other rule, so typing a hint label is never mistaken for a motion. Third, the editable guard: inside an `input`, `textarea`, `select`, or `contentEditable` element only `Escape` acts (it blurs the field) and every other key passes through untouched. Fourth, the modifier bailout: any key held with `Alt`, `Meta`, or `Ctrl` is left alone, because those chords belong to the palette, the find bar, and the vim toggle, not to motions. Only after all four does the key reach the prefix state machine and the keymap. The listener is scrupulous about *not* stealing keys: it calls `preventDefault` only when the key maps to a real action — an unknown key gets no interference — and when it does act it calls both `preventDefault` and `stopImmediatePropagation`, whose comment reads “win over site single-key shortcuts,” so a Gmail `j` fires the vim scroll and not Gmail's own handler.

The exact motion amounts are small and fixed: `j/k/h/l` scroll 66 pixels; `d/u` scroll half the viewport height (`innerHeight · 0.5`, rounded); `G` and `L` both jump to the document bottom (they run identical code); `H` to the top; `M` to the vertical middle of the document. The `z`-family positions a *reference element* — the focused element if it is real and on screen, otherwise whatever sits at the center of the window (`elementFromPoint` at the viewport midpoint) — to the top, center, or bottom of the viewport, computed from its `getBoundingClientRect` with a four-pixel margin.

The two-key prefix machine, marks, and yank

Multi-key commands run through one small state variable and a **900-millisecond** timeout. Pressing `g`, `z`, `y`, `m`, or ``` arms that prefix; the next key completes it or, after 900 ms of silence, the prefix simply clears. Under `g`: `gg` scrolls to the top, `gt/gT` switch to the next/previous tab (through the storage bus), and `gi` focuses the first non-hidden input on the page. Under `y`, `yy` copies the page URL to the clipboard and flashes a toast. Under `m`, `m<x>` records the current scroll position into an in-memory `marks` object keyed by the character you pressed;

under ``` (and `'`, which is aliased to it), ``<x>` scrolls back to that saved position. Marks are per-page and live only in memory — they are scroll bookmarks for the current page, not durable named locations. The toast that confirms a yank or a mark is a single fixed-position element in the corner, themed in the HUD cyan on a translucent black, faded out after 1.2 seconds.

Link hints, precisely

`f` and `F` enter hint mode. The script collects everything plausibly clickable — the selector is `a[href], button, non-hidden input, select, textarea, [role=button], [role=link], [onclick],` and `[tabindex]` — then discards anything smaller than two pixels, anything scrolled off screen, and anything with `visibility:hidden, display:none,` or zero opacity. Each survivor gets a label drawn from a home-row-first alphabet (`asdfghjklqwertyuiop...`): one letter while there are 26 or fewer targets, two-letter combinations beyond that. Labels render as fixed-position yellow chips (cyan once you have typed a matching prefix). Typing narrows the set; the moment exactly one label matches, that target activates. `f` clicks in place; `F` routes a link through the storage bus to open in a new tab; an editable target is *focused* rather than clicked; `Escape` or a dead-end keystroke cancels. The hint overlay sits at z-index 2147483645, one below the find bar and palette, so those always draw over it.

One registry, live remaps

Every default key above is data in `zkeys.js` (`window.ZWIRE_KEYMAP`), and `zvim.js` never hardcodes a key — it builds a `key → action` table by walking the registry's categories and letting a user override win over each default. The overrides live in `chrome.storage.local` under `zb_keys` as `{action: key}`, and the script both reads them at load and re-listens on `storage.onChanged`, so rebinding `scrollDown` from `j` to `s` on the Keyboard settings page changes what fires on every open page the instant you save it, with no reload. The same registry and the same `zb_keys` key drive the tmux overlay's post-prefix keys, which is why one settings page remaps both. The per-tab off switch (`Ctrl/⌘+\\`) flips an in-memory flag rather than a stored one, so it is deliberately transient — a page where you have turned vim off gets its raw keys back, and the next page starts with vim on again.

Chapter 10 – tmux in the Browser

`ztmux.js` is the most ambitious content script in the extension: a real, in-page tiling window manager modeled on tmux and zellij, where a session holds windows which hold panes, the panes are actual webpages rendered in iframes, and you split, zoom, swap, rotate, resize, lay out, copy, paste, and broadcast-type across them — all without a single OS window. At 777 lines it is a small application; this chapter is its model and its command set.

The session/window/pane tree

The whole state is one in-memory object: a session `S` with a list of `windows` and an `active` index. Each window has a binary *tree* of nodes — a `leaf` is a pane (an id, a url, a title), and a `split` has a direction (`row` = side by side, `col` = stacked), a `ratio`, and two children `a/b`. Nesting is arbitrary: a split's child can be another split, so panes tile to any depth. This is the same data model tmux uses internally, rebuilt in JavaScript, and every command is a tree edit:

- `splitPane(dir)` replaces the active leaf, in place, with a new split whose children are the old leaf and a fresh pane.
- `closePane()` finds the active leaf's containing split and promotes its sibling into the split's slot (collapsing the split), killing the window if it was the last pane.
- `swapPane`, `rotatePanes`, and `breakPane` (move a pane out to its own window) are all slot swaps in the tree.
- `resizePane(dir, amt)` walks from the active leaf up to the first ancestor split on the right axis and nudges its `ratio`, clamped to `[0.08, 0.92]`.

A `nodeSlot(root, target)` helper returns the `{parent, key}` such that `parent[key] === target`, and that little abstraction is what makes every structural command a two-line edit rather than a special case.

Panes are iframes, which is why the fork strips X-Frame-Options

Each pane renders a real webpage in an iframe. The catch is that most sites send `X-Frame-Options` or a frame-ancestors CSP that forbids being framed — so `hud-internal` ships a `declarativeNetRequest` rule (`rules/frame_bust.json`) that strips those headers, and on the fork the native `ancestor_throttle.cc` patch (0008) does the same at the engine level. That is the whole reason patch 0008 exists: without it, the tmux overlay could only tile a handful of framing-friendly sites. Fresh panes open the zwire new tab, and because `chrome://newtab` itself cannot be iframed (`chrome://` pages are un-embeddable), the overlay frames the new-tab *extension's* web-accessible page directly — the same page `chrome://newtab` redirects to.

The overlay runs in *all* frames, and the roles differ. The top frame hosts the overlay UI, the tree, and all the commands. A pane iframe (marked with `window.name='zbtmux'`) runs a thin forwarder: it catches the `Ctrl-b` prefix and the following command key and `postMessages` them up to the top frame, so the prefix works even while your focus is inside a framed page. The prefix is `Ctrl-b` (the real tmux one) or, because the pre-rebuild fork's native split patch used to eat `Ctrl-b`, `⌘-b` (Alt-b) as an alternate nothing else intercepts.

The command set

After the prefix, the keys are the tmux ones, and every one is remappable through the shared registry (Chapter 9). The full set from `zkeys.js`:

```
% split right      " split down      o next pane      ; last pane
h/j/k/l select pane ← ↓ ↑ →      H/J/K/L resize pane ← ↓ ↑ →
z zoom pane        x close pane      { } swap ← →      0 rotate
! break to window  q pane numbers    space cycle layout
F/G/I grid 4/8/16
c new window        n/p next/prev win  Tab last window    w window list
, rename window     . move window      ' go to window      & kill window
m mark / swap       e synchronize panes E toggle this pane in sync group
[ copy mode         ] paste buffer    = choose buffer
s sessions          S save session
r reload pane       t clock           : palette          d detach  ? help
0-9 select window  ←→↑↓ focus by direction (Ctrl/Alt + arrows resize)
```

Layouts (`space`) cycle through even-horizontal, even-vertical, main-vertical, main-horizontal, and tiled — each rebuilt by `buildEven/cycleLayout` from the current leaf list. Directional focus (`navDir`) uses the rendered pane rectangles to pick the nearest pane in a direction, so the arrows move focus spatially like tmux's `select-pane -LRUD`.

Copy mode, paste buffers, and synchronize-panes

Because panes are real scrollable documents, copy mode (`[`) is a genuine scroll-and-select mode inside the focused pane: `j/k/d/u/g/G` scroll, you select text, and `y/Enter` yanks the selection up to the overlay's paste-buffer stack (most-recent-first, capped at 30). `]` pastes the most recent buffer into the focused pane's field, and `=` opens a chooser. Any ordinary copy in a pane is also captured into the stack.

Synchronize-panes (`e`) is the broadcast-typing feature, and it is careful about line editing. When sync is on, a pane forwards each editable keystroke up to the top frame, which relays it to every other pane's focused (or last-focused) field. Plain characters, `Enter`, and `Backspace` forward as-is; the readline combos forward as *semantic tokens* so the broadcast stays correct — `C-w` (kill word) and `C-u` (kill to line start), plus their macOS `⌘/⌘-Delete` twins. Each receiving pane applies the token cursor-aware, deleting the right span and firing a real `input` event so the framed page's own handlers see the change. This is how you type one command into eight terminals-in-tabs at once.

Persistence

The whole tree lives in memory, so a reload of the host page would wipe it. `ztmux.js` snapshots the windows, panes, layout, and paste buffers to per-tab `sessionStorage` (`zbtmux-session-v1`) and restores on re-inject, so the tmux session survives a page reload — the tree, the active window and pane, and the buffers all come back. It is themed entirely with the active scheme's CSS variables, so the tabs, pane borders, dividers, and modals match cyberpunk, ember, matrix, and the rest live. The overlay reserves the bottom 22px for the real powerline statusbar, which is the next chapter.

Resize and layout, as tree math

Resizing walks the tree. `resizePane(dir, amt)` picks an axis from the direction (`left/right` → a `row` split, `up/down` → a `col` split), builds the path from the active leaf up to the root recording which side each descent came from, and nudges the ratio of the *first* ancestor split on that axis, clamped to `[0.08, 0.92]`. The nudge size depends on how you asked: a plain `Ctrl+arrow` resizes by 0.03, `Alt+arrow` by 0.08, and

the remappable `tmux-resize-*` keys by 0.05; dragging a divider with the mouse is the exception, clamped instead to `[0.06, 0.94]` from the pointer's position within the split. The clamp is what keeps a pane from ever collapsing to nothing or swallowing its sibling.

The five layouts (`space` cycles them) are each a fresh tree built from the current flat list of leaves. `even-h` and `even-v` recursively split the list at its median so the panes tile evenly across a row or a column. `main-v` gives the first leaf 60% of the width and stacks the rest in a right-hand column; `main-h` gives the first leaf 60% of the height and lays the rest in a row beneath — the classic tmux main- pane layouts. `tilled` groups the leaves into rows of `√n` columns. There is a separate grid path, too: `tmux-grid-4`, `tmux-grid-8`, and `tmux-grid-16` (bound to `F/G/I` after the prefix) pad the window with blank panes up to 4, 8, or 16 and build a square-ish tile — a one-key way to summon an even wall of panes.

Directional focus (the arrow keys, `navDir`) is geometric, not tree-order. It reads the rendered percentage rectangle of each pane, takes the active pane's center, and among the panes strictly on the requested side (with a one-unit epsilon) picks the one whose center is nearest by straight-line distance. There is no wrap: if nothing lies to the left, the left arrow does nothing. That is the same “nearest pane in a direction” rule tmux's `select-pane -LRUD` uses, rebuilt from the panes' on-screen geometry.

The forwarder protocol, and typing into eight terminals

Because each pane is a cross-origin iframe, the overlay cannot listen for keys inside a pane from the top frame — so the pane runs a thin forwarder and the two frames talk by `postMessage`. Every message from a pane up to the top frame carries a `__zbtmux` tag and one of a small set of shapes: `{prefix:1}` (the `Ctrl-b/⌘-b` prefix was pressed inside the pane, so the top frame should arm), `{cmdKey, ctrl, alt}` (the post-prefix command key), `{palette:1}` (`⌘K` inside a pane, so the palette opens in the top frame and drives the active pane rather than opening a stray tab), `{yank:...}` (a copy-mode selection), and `{synckey:...}` (a broadcast keystroke). The top frame replies downward with `{setSync:...}`, `{copyMode:true}`, `{pasteText:...}`, and `{syncapply:...}`. This is why the prefix works while your cursor is inside a framed site: the pane catches it and hands it up.

Synchronize-panes (`e`) is the most careful piece. When sync is on and you are typing in an editable field, the pane forwards each keystroke up as a `synckey` token, and the top frame relays it to every other pane in the sync group. Plain characters, `Enter`, and `Backspace` travel as themselves; the line-editing combos travel as *semantic* tokens — `Ctrl-w` and `Alt-Backspace` both become `C-w` (kill word), `Ctrl-u` and `⌘-Backspace` both become `C-u` (kill to line start). Each receiving pane applies the token cursor-aware against its focused (or last-focused) field: `C-w` walks back over trailing whitespace and then a run of non-whitespace and deletes that span; `C-u` deletes back to the start of the current line (`lastIndexOf('\n')`); a plain character is spliced in at the caret. Every edit dispatches a real `input` event so the framed page's own handlers see the change. That token discipline is what lets one command typed once land correctly in eight different sites' input fields at once, even when their fields disagree about selection state.

Copy mode, paste buffers, and the chooser

Copy mode (`[`) runs inside the focused pane, because the pane is a real scrollable document. A magenta indicator appears, `j/k` scroll by 64 pixels, `d/u` by half the viewport, `Space/b` by nine-tenths of it, `g/G` jump to the ends, you select text with the mouse, and `y` or `Enter` yanks the selection up to the overlay's paste-buffer stack. The stack is most-recent-first, de-duplicated, and capped at 30 entries; any ordinary copy inside a pane is captured into it too. `]` pastes the most recent buffer into the focused pane's field, and `=` opens a chooser — a modal listing each buffer's first 90 characters, selectable by click or by digit. The buffers persist with the rest of the session.

Two kinds of persistence

The overlay has two separate persistence layers, and they answer two different questions. A per-tab `sessionStorage` snapshot under `zbtmux-session-v1` keeps the live tree, the active window and pane, the open flag, and the paste buffers, so a reload of the host page restores exactly where you were — this is the “survive an accidental refresh” layer, and it migrates an older per-window `sync` boolean to the newer `sync-member-id` array on the way in. The second layer is *named* sessions: `chrome.storage.local` under `zb_tmux_sessions` holds durable, user-named layouts (each a list of windows, each a list of panes with their URLs and titles), created with `S`, listed and loaded with `s`, and managed in full on the Sessions page of Chapter 14. Loading a named session writes a `zb_tmux_load` trigger the live overlay watches, rebuilds every window’s tree from its saved pane list, and re-opens. One layer is a crash-recovery snapshot; the other is a library of layouts you keep.

The rest of the command surface

The window and pane operations round out the set, and every one is a tree edit or a small modal. Windows: `c` new, `n/p/Tab` next/previous/last, `,` rename, `.` move (renumber), `'` go-to by prompt, `w` list, `k` kill, and `0–9` jump. Panes: `q` overlays a large number on each pane (type the digit to focus it, auto-dismissed after three seconds), `m` marks the active pane and swaps it with a previously marked one, `!` breaks a pane out into its own window. `r` reloads a pane by re-assigning its `src`; `t` shows a full-screen clock ticking every second; `?` opens a help card built from the same keymap the settings page renders, so the help can never drift from the actual bindings; `d` detaches (hides) the overlay. The overlay writes a small `zb_tmux` record to storage on every change so the real powerline statusbar — a separate content script in the top frame — can show the window/pane counts and light the prefix indicator without the overlay drawing its own bar.

The pane sandbox and the z-index ladder

Each pane iframe is deliberately sandboxed, and the exact token set is the interesting part. The overlay sets `sandbox="allow-scripts allow-same-origin allow-forms allow-popups allow-popups-to-escape-sandbox allow-modals allow-downloads allow-top-navigation-by-user-activation allow-storage-access-by-user-activation allow-presentation"` — a broad grant so framed sites behave like real tabs, with one pointed omission: it does not grant automatic top-navigation, only the user-activation form. That omission is what defeats JavaScript frame-busting scripts that try to break out by reassigning `top.location` on load; combined with the `frame_bust` header-strip of the earlier section, it is what lets the overlay hold an arbitrary site in a pane without the site either refusing to be framed or escaping the frame. Panes also get `allow='clipboard-read; clipboard-write; fullscreen'` so copy, paste, and fullscreen work inside them.

The overlay’s stacking is a fixed z-index ladder, tuned so the right thing always draws on top: the whole overlay sits at 2147483640, a pane’s click-cover at 2, the drag dividers at 6, the pane-number badges at 40, a pane’s autocomplete at 45, the drag shield at 50, and modals at 60; the copy-mode indicator inside a pane climbs to the maximum 2147483647 so it is never hidden by page content. Every surface is themed from the active scheme by injecting the scheme’s variables into an `#zbtmux` CSS prefix — so the tabs, pane borders, dividers, modals, and the marked-pane accent all recolor live when you switch schemes, with the marked pane drawing an accent border and a ♦ and a synced pane a cyan border. It is a small windowing toolkit’s worth of CSS, all keyed to the same twenty variables the rest of the HUD uses.

Now a shared component: `ZGui.tmux`

The overlay described above began as `ztmux.js`, a bespoke content script living only in `hud-internal`. It has since been promoted into the shared `zgui-core` toolkit as `ZGui.tmux` — the same tiling window

manager, now the common one the house Tauri apps consume too — and the HUD consumes it rather than carrying its own copy. The model, the command set, the copy-mode and synchronize-panes behavior, and the persistence are all unchanged; what moved is *where the code lives*. In the manifest the overlay is now two thin scripts around the shared WM: the top frame loads `ZGui.tmux` and its dependencies (the shared split-pane, modal, toast, and button-bar widgets) plus `ztmux-config.js`, the provider that calls `ZGui.tmux.init` with iframe panes (an address bar, `openEmptyPane/renderPane`, and `applyKey/copyMode/paste` bridged over `postMessage`) and reads prefs from `chrome.storage`; and every frame loads `ztmux-pane.js`, the pane-side forwarder extracted from the old script, which relays the prefix, the synced keystrokes, and yanks up to the top frame and runs copy-mode locally over the pane's page. `zgui-core` v1.0.18 added the external-feed API (`prefix/key/relaySync/yank`) precisely so a cross-origin-iframe consumer like this one can feed the shared WM from outside. The retired `ztmux.js` is left in the tree for reference but is no longer injected.

Chapter 11 – The Powerline Statusbar

`zstatus.js` pins a tmux/powerline statusbar to the bottom of every top-level page. The left side is the HUD's own indicators; the right side is a live system monitor fed by the native host. This chapter is its segments, its powerline geometry, and the stream behind the numbers.

Left and right

The bar sits at `position:fixed; bottom:0; height:22px`, top frame only, themed by the active scheme, in the Share Tech Mono face. Reading left to right:

- **Left:** a `ZW` badge, the `C-b` prefix indicator (lit while a tmux prefix is armed), the current scheme name, the `VIM` status, and `⌘K`.
- **Right:** the machine, streamed from the native host into `chrome.storage`'s `zb_sys` key — CPU, MEM, SWAP, DISK, IO, NET, LOAD, UP (uptime), a temperature, BATT, LAN IP, WAN IP, host name, and a clock.

Each right-side segment is a `<span>` with a `data-*` hook the refresh function writes into: `data-cpu`, `data-mem`, `data-swap`, `data-disk`, `data-io`, `data-net`, `data-load`, `data-up`, `data-temp`, `data-batt`, `data-lip` (LAN), `data-host`, `data-clock`. The refresh formats each from the `zb_sys` object — CPU as a percentage, memory and swap as used/total, disk as a percent, IO and NET as per-second rates with `↑/↓` and `R/W`, battery with a charging-vs-on-battery glyph and a percentage, uptime humanized, and the clock ticking every second.

Powerline chevrons

The bar draws true powerline chevrons, not glyph characters, using CSS borders. Each block is a `.plr` with a triangular `:before` pseudo-element formed from transparent top/bottom borders and a solid side border, so the segments interlock as arrows. Left-of-the-spacer segments point right (►) and right-of-the-spacer segments point left (◄), and consecutive blocks alternate between two background shades (`s0/s1`) so the chevrons read as distinct steps. This is exactly how a tmux powerline theme is built in the terminal, rebuilt here in CSS so the browser statusbar looks like the shell statusbar the rest of the stack uses.

The stream behind the numbers

The numbers are real because `zwire` is a desktop app with a native host that can read the machine. `zstatus.js` does not sample anything itself; it renders whatever is in `zb_sys` and re-renders when that key changes. The native host fills it: Chapter 15 covers `zwire-host`'s `sysinfo_start` mode, which samples the machine through the `sysinfo` crate on a background thread and streams the result, which the background worker writes into `zb_sys`. So the statusbar is a genuine system monitor — CPU load, memory pressure, network throughput, battery, temperature — pinned to the bottom of every web page, not a decorative mock. The bar is toggled from the `⌘K` palette, and (like everything HUD) recolors instantly with the scheme.

Every segment, and how it is formatted

Each right-hand segment reads one field of the `zb_sys` object the worker writes and formats it with a small, purpose-built formatter, so the numbers read like a tmux theme rather than raw JSON. CPU is a whole-number percent; MEM and SWAP are used-over-total run through a byte-humanizer that steps `B/K/M/G/T` in powers of 1024 and shows one decimal below 100; DISK is a percent; IO is `R<rate> W<rate>` and NET is `↑<rate> ↓<rate>`, both per-second and both humanized the same way; LOAD is the three load averages joined by spaces; UP humanizes uptime as `Xd Yh Zm`; the temperature is `<n>°C` and, when the machine reports no sensor, that segment hides itself entirely rather than showing a dash; BATT prefixes a charging glyph (a bolt ⚡ when plugged, a battery glyph on battery) before the percent; LAN and WAN are the private and public IPs; HOST is the short hostname; and the clock, updated on its own one-second interval independent of the two-second stats stream, shows `MM-DD HH:MM:SS`. Every segment writes into a `data-*` span (`data-cpu`, `data-mem`, and so on down to `data-clock`), so a stats refresh rewrites text in place without rebuilding the bar.

The left side is the HUD's own state, not the machine: a `ZW` badge, the `C-b` prefix indicator, the current scheme name (filled from the scheme table's label), a per-window tmux readout (`i·paneCount`, the active window emphasized, a `⇄` when any window is syncing), a static `VIM` marker, and a clickable `⌘K` that opens the palette. A flexible spacer between the two halves is what splits the bar into a left group and a right group.

Powerline geometry in CSS

The chevrons are real triangles, not glyphs. The 22-pixel bar height sets up an 11-pixel transparent top and bottom border, and each segment carries a pseudo- element with a solid side border that draws the arrow: left-of-spacer blocks put a `border-left` on their `::after` so they point right, right-of-spacer blocks put a `border-right` on their `::before` so they point left, and consecutive blocks alternate between two background shades (`--bg-card` and `--bg-secondary`) so the interlocking arrows read as distinct steps. The layout function finds the spacer, walks left assigning the point-right class with an alternating shade index, walks right assigning the point-left class, and skips the badge, spacer, and prefix block. This is exactly how a tmux powerline theme is assembled in a terminal, rebuilt in CSS so the browser's bottom line matches the shell's.

The `C-b` prefix block does one extra thing: it lights up. When you press the tmux prefix, the block's background flips from dim to the scheme accent for 1.6 seconds and then fades back — a visible confirmation that the prefix armed. Because the statusbar lives only in the top frame while the prefix can be pressed inside a framed pane, the block listens two ways: directly for `Ctrl-b` keydowns in the top frame, and for the overlay's `zb_tmux.armed` storage write, which reaches it reliably across frames. The bar itself is `position:fixed` at the bottom, 22 pixels tall, top-frame only, with a blurred translucent background, and it re-themes from `zb_scheme` and re-renders from `zb_sys/zb_tmux` on every relevant storage change.

Now a shared component: `ZGui.powerline`

Like the tmux overlay, the statusbar has been promoted out of the HUD and into `zgui-core`. The bespoke `zstatus.js` is retired; the bar is now `ZGui.powerline` (shipped in `zgui-core` v1.0.17 as `powerline.js/powerline.css`) — the exact same powerline bar the house Tauri apps (`zoffice`, `zphoto`, `zpdf`) render, so the browser's bottom line and the apps' now come from one implementation. What `hud-internal` ships is a thin adapter, `zpowerline.js`, that feeds `ZGui.powerline.init` from `chrome.storage:zb_sys` becomes the system-stats segments, `zb_tmux` the window/pane segment and the armed prefix lamp, `zb_scheme/zb_ui` the scheme (injected onto `:root` so the shared bar inherits the live theme), and `zb_status` the show/hide toggle. The manifest's top-frame statusbar content script now loads `powerline.js` plus `zpowerline.js` (and `powerline.css`) in place of `zstatus.js`. Every segment, formatter, chevron, and behavior described above is preserved — it is the same bar, sourced once and shared — with `zstatus.js` left in the tree for reference but no longer injected.

Chapter 12 — The Command Palette

The `⌘K` command palette is the HUD's universal launcher: one fuzzy-filtered list that jumps to any internal or native page, switches scheme, runs a keyword search, and surfaces your tabs and frecent history. It exists in **four** implementations, one per execution context, all sharing the same `zgui-core` engine and a single backend-agnostic item source (`palette-cmds.js`, vendored verbatim into each extension so the four can't drift):

- `zpalette.js` — the content-script palette on ordinary `http(s)/chrome://` pages.
- `pages/zg-boot.js` — the palette on the HUD's own extension pages (Settings, Sessions, Host console, and the rest). A web page gets the content-script palette; a HUD page gets this one — both are `hud-internal`, which is why it reads as one but is two.
- `newtab/palette.js` — the New Tab palette (a separate extension).
- `zpwchrome/lib/zpc-palette.js` — the palette on the `zpwchrome` power-tool's own dashboard pages (a second separate extension).

Every surface lists the same rows, including deep-links into `zpwchrome`'s tool pages (Downloads, Pass, Userscripts, Host console, ...). `hud-internal` owns `⌘K` browser-wide as a `chrome.commands` shortcut — a page keydown can never intercept it — and its worker routes each press to the palette matching the active tab. This chapter is what it lists and how it opens.

What the palette lists

Opened with `⌘K` (or `:` on a page, or `o` in vim mode), the palette registers several sources into one fuzzy list:

- **Internal pages** — dozens of `chrome://` destinations, from the everyday (extensions, settings, history, downloads, bookmarks, flags, gpu) to the deep diagnostics tail (`net-internals`, `serviceworker-internals`, `indexeddb-internals`, `webrtc-internals`, `tracing`, `histograms`, and many more), each opening in a new tab.
- **Settings sub-pages** — the “tabs” inside `chrome://settings` (Appearance, Privacy & security, Performance, Search engine, and the rest), so you can jump straight to one.
- **Schemes** — one entry per color scheme, which sets it through the native bridge.
- **Open tabs** — every tab, with its title and URL, switching to it (and focusing its window) on select.
- **Frecent history** — your history scored by visit count and recency over the last 90 days, top matches first.
- **zwire destinations** — the CI-runs page, the app store, the Chrome Web Store.

The engine is `zgui-core`'s `ZGui.palette` over `ZGui.fzf`, so typing filters fuzzily and the matched characters are highlighted. The styling is inlined into the content script (the `.palette-*` CSS) so it works on arbitrary pages, themed by the active scheme's variables.

Keyword commands (cmd-defaults.js)

Beyond jumping to pages, the palette runs *keyword commands* — a short keyword plus an argument that expands into a URL. The background worker seeds a default set on first run only (`cmd-defaults.js` and the `CUSTOM_CMD_SEED` in `background.js`), stored in a user-editable `zb_custom_cmds` array so your edits and deletions are never clobbered. The seeds are the everyday web tools:

```
cg ChatGPT      cl Claude      pp Perplexity    wa WolframAlpha
gm Gmail        gd Google Drive cal Google Calendar
tr Translate    img Google Images def Define word
hn Hacker News  li LinkedIn    ...
```

Typing `cg how do futexes work` in the palette expands the `{q}` in the ChatGPT template with your query and opens it. Each rule is `{id, icon, label, keyword, type, value}`, where `type` is `url` (a template) or `action`, and the whole set is edited on the HUD Commands page (Chapter 14). This turns the palette into a custom search-engine bar with your own keywords, seeded sensibly and yours to change.

How it opens across surfaces

Opening the palette is the same `⌘K`-reservation dance as focus on the new tab. On an ordinary page, `zpalette.js` catches `⌘K/Ctrl+K` (and `:` when not in a field) directly in a capture-phase listener. On the new tab, Chrome reserves `⌘K` at the browser level, so the `hud-internal` background worker owns it as a `chrome.commands` shortcut and messages the new-tab page to open its own palette. And from *inside a tmux pane*, `⌘K` is forwarded up to the top frame so the palette opens there and navigates the pane rather than opening a stray new tab. Three surfaces, three plumbing paths, one palette — because `⌘K` is guarded differently on each and the HUD routes around each guard.

The full source catalog

The list the palette assembles is larger than the summary above suggests. The static half is three tables in `zpalette.js`: a `PAGES` set of the HUD's own replacement pages (Extensions, Settings, History, Bookmarks, CI runs, Shortcuts, Extension shortcuts, Custom commands, System info); a `CHROME` set of roughly forty-four `chrome://` destinations that runs from the everyday (downloads, flags, gpu) down the entire `-internals` diagnostics tail (`net-internals`, `serviceworker-internals`, `indexeddb-internals`, `quota-internals`, `webrtc-internals`, `media-internals`, `histograms`, `tracing`, `ukm`, `predictors`, `memory-internals`, and more), ending at a “chrome://about” entry that lists them all; and a `SETTINGS` set of the nineteen tabs inside `chrome://settings` (Appearance, Privacy, Security, Cookies, Performance, Search engine, and the rest). On top of the static tables the palette registers one entry per scheme, your open tabs (read from `zb_tabs`), your frecent history (from `zb_frecent`), your extensions (each offering a *Tweak* and a *Manage* action), and two external destinations — the Chrome Web Store and the zwire app store. Every page or tab entry routes through the storage command bus so the sleeping worker actually performs the navigation.

Keyword commands and the search providers

Beyond jumping to a destination, the palette runs *keyword commands*. Two provider functions feed the fuzzy list live as you type. The first is a built-in search table: type `crate serde` and it expands crates.io's search URL with `serde`; a bare keyword opens that site's origin. The second is your own `zb_custom_cmds`, seeded once by the worker and edited on the Commands page, and it is richer than a URL opener — a custom command's `type` may be `url` (substitute the typed argument for `{q}` and open it), `shell` (run the argument in the HUD terminal), `js` (evaluate the argument as the body of a function with the typed text bound to `q`), `action` (a built-in browser action), or `scheme` (switch the color scheme). A generic Google/DuckDuckGo fallback is always registered last, marked so the ranking keeps it beneath every real match. So the same input box is a launcher, a custom search-engine bar, a shell one-liner runner, and a tiny JavaScript scratchpad, disambiguated entirely by the leading keyword. The scheme entries and the `cycle color scheme` action both go through the worker's native bridge, so a scheme picked from the palette lands in the same `~/zwire/hud-scheme` file every other surface reads.

The provider tail: shortcuts, extensions, and a Chrome cheatsheet

Three more providers round out what the palette can surface, each drawing on a storage mirror the worker keeps fresh. A shortcut provider reads `zb_shortcuts` — every extension’s keyboard commands, published by the worker via `developerPrivate` — so you can find a binding by name. An extension provider reads `zb_exts` and, per extension, offers a *Tweak* entry (opening `chrome://extensions/?id=...`) and a *Manage* entry, so the palette doubles as an extension launcher. And a platform-aware Chrome-shortcut provider carries a curated cheatsheet of the browser’s own key bindings, keyed on whether the machine is a Mac, so typing part of a native action surfaces its real chord. None of these are static text: they are the worker’s live `zb_*` mirrors turned into palette rows, which is why the palette is never stale — it is a view over the same shared storage the statusbar, the settings pages, and the new tab all read.

Chapter 13 – Fuzzy Find

`zfind.js` hijacks `Ctrl/⌘+F` on every page and replaces Chrome's native find with the HUD's own fuzzy find bar: substring find-in-page with all occurrences navigable, an `zfz` fallback when there is no substring hit, and highlighting that never mutates the page's DOM. This chapter is that one script.

Substring first, fuzzy second

The find bar is a small fixed-position input themed by the active scheme. As you type, it first does a plain substring find-in-page across all text nodes, collects every occurrence as a navigable match, and lets you cycle them (with a count, like Chrome's own “3/12”). When there is *no* substring hit, it falls back to the same `zfz` fuzzy matcher the palette and the sibling apps use (`ZGui.fzf`), so a misremembered or scattered query still finds the closest text. Substring-first is the right default — exact matches are what you usually want — with fuzzy as the safety net rather than the primary mode.

The CSS Custom Highlight API

The interesting engineering choice is how it highlights. Instead of wrapping matches in `<mark>` elements — which mutates the page's DOM, can break the page's own scripts and layout, and is expensive to undo — `zfind.js` uses the CSS Custom Highlight API: it builds `Range` objects for each match and registers them as named highlights that the browser paints via a `::highlight()` pseudo-element, with zero DOM mutation. The page's DOM is never touched; the highlights are a paint-layer overlay the browser manages. The script guards for the API (`'highlights' in CSS`) and no-ops where it is absent, but on the Chromium base it targets, it is present. This is the modern, correct way to highlight find matches, and it is why the HUD find bar can run on complex web apps without corrupting them the way a `<mark>`-wrapping finder would.

Two highlight layers, and the two ranges

The bar keeps two named highlights, not one. All matches are painted underlined in the scheme cyan under the name `zfind`, and the *current* match is painted in the accent color with a translucent background under a second name, `zfind-cur`, which is deleted and rebuilt every time you cycle. Cycling wraps (`Enter` forward, `Shift+Enter` back) and scrolls the current match to the center of the viewport; the counter reads `current/total` exactly like Chrome's own find. Because a `::highlight()` rule cannot read a `var()` off the page's own elements, the script bakes concrete scheme colors into the highlight CSS rather than referencing the HUD variables — a small compromise the code comments call out.

Two implementation choices are worth naming. The match walk uses a `TreeWalker` over text nodes, skipping `SCRIPT/STYLE/NOSCRIPT/TEXTAREA` parents, hidden nodes, and the find bar's own DOM, and it caps at 6000 nodes and 2000 matches so a pathological page cannot hang the search; the fuzzy fallback additionally skips any single text node longer than 300 characters. And the bar's `position:fixed` CSS is injected *synchronously* before the input is focused — the comment explains that focusing an in-flow element would scroll the page down to it, which is the exact “Cmd+F scrolls the page” bug a naïve find bar exhibits. There is no debounce: the search runs on every keystroke, which is affordable precisely because the highlight layer never touches the DOM.

Chapter 14 – The HUD Internal Pages

Several of Chrome's internal pages are not *skinned* but *replaced*: a `document_start` redirect (`redirect.js`) sends `chrome://extensions`, `chrome://settings`, `chrome://history`, `chrome://bookmarks`, and `chrome://version` to purpose-built HUD pages under the extension's `pages/` directory, which reimplement the surface using the extension's privileged permissions. This chapter is the tour.

Why replace rather than skin

The `chrome://` pages that get replaced are the ones you interact with, not just read — the extension manager, settings, history, bookmarks. Skinning them (the `theme.js` token map) recolors them but leaves Chrome's own UI and its quirks; replacing them lets the HUD present the *data* in its own cyberpunk components with its own fuzzy filtering and keyboard model. The replacements are real, backed by the manifest's permissions: `management` and `developerPrivate` for the extensions page, `history` for history, `bookmarks` for bookmarks, `downloads` for downloads, `sessions` and `system.display` where needed. They are served from the extension, so they are web-accessible pages the redirect can navigate to.

The pages

From the `pages/` directory, the replacements and HUD-native pages are:

<code>extensions.html</code>	the extension manager (enable/disable/reload/remove), HUD-styled, via <code>management</code> + <code>developerPrivate</code>
<code>settings.html</code>	a HUD settings surface (scheme picker, visual-effect toggles, light mode) writing the <code>hud-ui.json</code> prefs
<code>history.html</code>	history, fuzzy-filterable, via the <code>history</code> permission
<code>bookmarks.html</code>	bookmarks, via the <code>bookmarks</code> permission
<code>downloads.html</code>	downloads, via the <code>downloads</code> permission
<code>version.html</code>	a HUD <code>chrome://version</code> replacement
<code>keys.html</code>	the keybinding editor – renders <code>ZWIRE_KEYMAP</code> and writes remaps to <code>zb_keys</code> (Chapter 9), same registry <code>zvim</code> reads
<code>commands.html</code>	the <code>%K</code> keyword-command editor – edits <code>zb_custom_cmds</code> (Chapter 12)
<code>extshortcuts.html</code>	extension keyboard-shortcut management
<code>ci.html</code>	a <code>zwire</code> CI-runs surface (linked from the palette)
<code>terminal.html</code>	the standalone terminal page (Chapter 16)

Two of these are the HUD's own configuration surfaces and close loops opened earlier in the book: `keys.html` renders the shared keybinding registry and writes remaps into `zb_keys`, the exact key the vim and tmux scripts read, so rebinding a key in the page changes what fires on every page; and `commands.html` edits the `zb_custom_cmds` array the palette's keyword commands come from. The pages share a set of HUD stylesheets (`hud-pages.css`, `ext-page.css`) and helper scripts (`zg-boot.js`, `zg-list.js`, `fzf.js`), so they are one consistent surface, not a dozen snowflakes. `redirect.js` runs at `document_start` so the swap happens before Chrome's own page paints, and the redirected set is exactly the set excluded from the `theme.js` skin so the two never collide.

What each replacement actually does

These are not skins with a `chrome://` page underneath; each is a working reimplement on a privileged Chrome API, and the API is what makes the page more than cosmetic:

- **Extensions** (`extensions.js`, the largest page at ~350 lines) is a full extension manager on `chrome.developerPrivate` plus `chrome.management`. It lists every extension including disabled and terminated ones, and it can enable and disable, uninstall (with the confirm dialog), reload an unpacked extension, load a new unpacked directory, pack a directory, check for updates, toggle developer mode, pin to the toolbar, set per-extension site access (`ON_CLICK`, `ON_SPECIFIC_SITES`, `ON_ALL_SITES`), flip the incognito/file-access/error- collection toggles, read an extension's on-disk size, and open its service- worker devtools. A `#shortcuts` sub-view edits every extension's keyboard commands in place, suspending Chrome's own shortcut handling while it captures the chord.
- **Settings** (`settings.js`) is built on `chrome.settingsPrivate`, so it renders every Chrome preference the API exposes — grouped by the first dotted segment of the key, each mapped to the right control (a toggle for booleans, a select where the pref advertises allowed values, a number or text field otherwise, a read-only JSON textarea for list and dictionary prefs) and disabled when the pref is policy- controlled. On top of the raw prefs sit the HUD's own Appearance card (the eight- scheme picker and a light-mode toggle) and an Effects card (scanlines, vignette, glow, animations), whose values are pushed to the native `hud-ui.json` file so the separate new-tab extension can follow them.
- **History, Bookmarks, and Downloads** are thin configurations of a shared filtered-list component (`zg-list.js`) over `chrome.history.search`, `chrome.bookmarks.getTree` (flattened to a folder-path table), and `chrome.downloads.search` respectively, each fuzzy-filterable and each wiring the matching `on*` events for live updates. Downloads opens a completed file on click through `chrome.downloads.open`; the HUD navigation actually points its Downloads entry at native `chrome://downloads`, because the power-tool owns the download manager, which leaves this page a working-but-unlinked alternative.
- **System** (`version.js`) replaces `chrome://version` with a HUD panel that shows the zwire edition, the HUD extension's manifest version, the Chromium version parsed from the user agent, the install and profile paths, and a *live* native- host probe — it sends `{cmd: 'get'}` to `com.zwire.hud` and reports connected with the current scheme and active effects, or “not connected” with the exact error and the setup command to run.

The two configuration surfaces, and two the book had not named

Two replacement pages are the HUD's own control panels and close loops opened earlier: **Keyboard** (`keys.js`) renders the shared `ZWIRE_KEYMAP` registry as editable chips, writes remaps into `zb_keys` (the exact key vim and tmux read), warns when a new binding collides with an existing one, and also edits the tmux prefix chord and arm-timeout that `ztmux.js` obeys; **Commands** (`commands.js`) is the CRUD editor for `zb_custom_cmds`, enforcing unique keywords, supporting all five command types (url/shell/js/action/scheme), and — a nice touch — remembering a deleted default in a `zb_cmds_removed` tombstone list so the worker's seeder never re-adds it.

Two more pages exist that the earlier map did not call out. **Sessions** (`sessions.js`) is a durable manager for the tmux overlay's named layouts: it reads and writes `zb_tmux_sessions`, lets you create, duplicate, rename, and delete sessions and edit their windows and panes, renders a small SVG preview of each layout using the same even-split geometry the overlay uses, exports and imports the whole set as JSON, and “attaches” a session by writing the `zb_tmux_load` trigger the live overlay watches. **CI** (`ci.js`) is a full GitHub Actions dashboard — the interactive front end to the worker's background poller of Chapter 8. It reads a `zb_ci` config (GitHub user, optional token, repo limit), fetches each repository's recent workflow runs through the GitHub REST API with the modern `X-GitHub-API-Version` header, renders them as per-repo accordions with status pills and relative timestamps (repos with a failing or in-flight latest run open by default), and handles rate

limiting by reading the `x-ratelimit-reset` header and backing off until the window clears. A token raises the repo budget and switches the repo query from the public account to your full affiliations.

One shell under all of them

Every page mounts through `zg-boot.js` (`window.ZBHUD`), which is why they read as one surface rather than a dozen. It builds the sticky HUD header and the cross-page navigation row, applies the persisted CRT and effect preferences, and installs the same `%K` command palette the content scripts carry — registered here with the page's own extra commands plus the shared HUD toggles, nav destinations, custom commands, frequent sites, and open tabs. It also owns the pages' half of the scheme bridge: `ZGui.colorscheme.onApply` sends the pick to the native host and mirrors it to `zb_scheme`, and a 1.5-second poll of the native file re-applies an externally chosen scheme so a page left open still follows a scheme picked elsewhere. The three list pages share `zg-list.js`, and the fuzzy matching everywhere is the shared `fzf.js` — so the consistency across a dozen pages is literal shared code, not a dozen parallel implementations.

The custom-command runner, five ways

The palette that `zg-boot.js` installs on every HUD page is the same one the content scripts carry, and it runs custom commands through one dispatcher, `runCustomBoot`, whose five branches are the concrete meaning of the command types the Commands page edits. A `url` command substitutes the typed argument for `{q}` and opens the result in a new tab. A `shell` command hands the argument to `window.zwireTermRun`, which opens the terminal and types it (Chapter 16). A `js` command evaluates the argument as the body of `new Function('q', value)`, with the typed text bound to `q` — a genuine one-line scripting hook, scoped to the page. An `action` command writes a `zb_cmd` onto the storage bus for the worker to perform. And a `scheme` command sets the color scheme through the native host and the storage mirror at once. The same five types resolve identically in the content-script palette, because both call the same runner against the same `zb_custom_cmds` array — so a command you define once behaves the same whether you trigger it from an ordinary page or from a HUD page. It is the last loop the book closes on the palette: one editor, one storage key, one runner, five behaviors, reachable from every surface.

Chapter 15 – The Native Host

`native/zwire-host` is the bridge between the browser and the machine — and, as of this edition, a good deal more than a bridge. It is a single self-contained Rust binary (~500 KB, no Python and no `psutil`, `panic = "abort"` and `opt-level="z"` in release) that began as `zwire`'s Chrome native-messaging host and grew into a **universal local endpoint**: a process any app can ask to read the machine, spawn a shell, crawl the filesystem, run a command, or hold a little state. It ships as its own MIT-licensed submodule under `extensions/hud-internal/native/zwire-host`, with its own README, its own CI, and a 15-module `src/` tree. (Earlier printings of this book described a Python reference host, `hud_host.py`; that file has been removed — the Rust host is now the only one, and where the old text speaks of a “reference,” read it as the historical first cut of the same scheme/stats/PTY path `zwire-host` now owns outright.)

One dispatcher, three transports

The heart of the host is a single **capability router** (`src/session.rs`, whose own doc line reads “the capability router shared by every transport”). Every request is a JSON object dispatched on its `cmd`; every reply is a JSON object; and a request may carry an `id` that is echoed on its reply, so one connection can multiplex many in-flight requests, streams, and terminals at once. Three transports feed that one router (`src/transport.rs`, “the three ways to reach the dispatcher”):

- **Native messaging** (the default, and what `zwire` uses). Chrome spawns the host and speaks length-prefixed JSON on `stdin/stdout`: a 4-byte little-endian `u32` length followed by that many UTF-8 bytes of body. `src/proto.rs` is the whole framing — `write_all(8(data.len() as u32).to_le_bytes())` out, a `read_exact` of four bytes then the body in, and `None` on a short read as the port-closed sentinel.
- **Local-socket daemon** (`zwire-host serve`). The same router, reached over newline-delimited JSON (one object per line) on a **Unix domain socket** (macOS/Linux: `$ZWIRE_HOST_SOCK`, else `$XDG_RUNTIME_DIR/zwire-host.sock`, else `~/.zwire/host.sock`, created `0600` under a `0700` dir because it exposes `exec/fs/pty`) or a **named pipe** on Windows (`\\.\pipe\zwire-host-<user>`). This is what lets `tmux`, `emacs`, shell scripts, and any language talk to the host without a client library — `printf '{"cmd":"sysinfo_once"}\n' | nc -U ~/.zwire/host.sock` is a complete client.
- **In-process library** (`zwire_host` as a crate). The same dispatcher, plus `api::walk` / `api::exec` helpers, so a sibling host can embed it and reuse the `crawl/exec/stream` machinery rather than reimplement it.

Because all three sit behind the one router, every command in the next section works identically over any of them.

The capability surface

Each capability is one small module under `src/`, and the fifteen of them together are the whole of what the host does:

- **Discovery & state** — `hello` feature-tests the host (`{ok,host,version,os, arch,pid,caps:[...]}`), `hostinfo` returns one-shot machine facts, and a per-app key/value store (`src/store.rs`) persists JSON under `~/.<app>/kv/` via `kv_set/kv_get/kv_merge/kv_del/kv_keys`.
- **System stats** (`src/sysmon.rs`) — `sysinfo_once` for a snapshot, `sysinfo_start/sysinfo_stop` to stream one `{sys:{...}}` frame per interval.

- **Filesystem** (`src/fsops.rs`) — `fs_read/fs_write/fs_append/fs_list/fs_stat/fs_mkdir/fs_rm`, plus `fs_walk`, a recursive crawl with `depth/ext/dirs_only/contains` filters.
- **File watching** (`src/watch.rs`) — `fs_watch` streams `created/modified/removed` events and `fs_tail` streams appended lines (`tail -f`, surviving rotation), each keyed by `id` and cancelled with `watch_stop`.
- **Exec & OS** (`src/exec.rs`, `src/osops.rs`) — `exec` runs a program to completion (base64 `stdout/stderr`), and `open/clipboard_get/clipboard_set/notify` reach the desktop.
- **Background jobs** (`src/jobs.rs`) — `job_start` spawns a long-running command that survives the connection and fires a desktop notification on completion; `job_list/job_result/job_poll` track and drain them.
- **Process tools** (`src/procs.rs`) — `ps`, `kill`, and `which`.
- **Pub/sub bus** (`src/bus.rs`) — `sub/unsub/pub` turn the host into a coordination hub; the daemon itself publishes on the `scheme` and `ui` topics whenever they change, so a subscribed HUD gets **live theme sync without polling**.
- **Host-to-host peering** (`src/peer.rs`) — daemons run with TCP peering to federate the bus across machines (`peers/peer_connect/remoted`), token-gated and single-hop so star and mesh topologies never loop.
- **PTY terminals** (`src/pty.rs`) — the terminal backend, below.

The scheme and UI bridge

The host's original job survives unchanged as the “legacy zwire scheme/ui” command group. A `{cmd:'get'}` returns the host `version` plus the current scheme (defaulting to `cyberpunk`) and the UI prefs; a `{scheme:'matrix'}` message — validated against the eight allowed names — and a `{ui:{...}}` merge each rewrite their HUD file with a write-to-`.tmp`-then-atomic-rename so a concurrent reader (the fork's color mixer, the new-tab poller) never sees a half-written file. These files are the cross-extension bus of Chapter 8; the host is what owns them on disk, and — via the `scheme/ui` pub topics above — what now pushes a change to every subscriber the instant it lands.

The system-stats stream

`sysinfo_start` (with an optional `interval_ms`, floored at 250 ms) enters `src/sysmon.rs`, which samples the machine through the cross-platform `sysinfo` crate — no `psutil`, no `pip install` — and sends one `{sys:{...}}` frame per tick:

<code>cpu</code>	<code>overall CPU %</code>	<code>mem/swap</code>	<code>used / total (bytes)</code>
<code>load</code>	<code>1/5/15 load average</code>	<code>uptime</code>	<code>seconds since boot</code>
<code>disk</code>	<code>root filesystem used / total</code>	<code>batt</code>	<code>pmset percent + plugged (macOS)</code>
<code>net</code>	<code>bytes sent/recvd rate (Δ/dt)</code>	<code>io</code>	<code>disk read/write rate (Δ/dt)</code>
<code>temp</code>	<code>hottest Components sensor</code>	<code>host</code>	<code>short hostname</code>
<code>lip</code>	<code>LAN IP</code>	<code>pip</code>	<code>public IP via api.ipify.org</code>

The one structural change from the old reference matters: the stream runs on its **own background thread**, so a persistent connection stays free for other RPCs while it ticks — a HUD can hold a `sysinfo` stream, a PTY, and a file watch on the same pipe at once, which the single-process Python host could not. Rates (`net`, `io`) are deltas over the elapsed wall-clock interval (`dt`, floored at 0.1 s), so they read as true per-second throughput rather than cumulative counters; the public IP is fetched from `api.ipify.org` and refreshed only periodically so the stream never hammers the lookup; and any sensor a platform lacks (no battery off macOS, no temperature) is simply omitted from the frame rather than crashing it. Everything the statusbar shows (Chapter 11) is one field of this sample.

The PTY relay

`pty_spawn` turns the host into a terminal backend (`src/pty.rs`). It opens a pseudo-terminal through the cross-platform `portable-pty` crate — `native_pty_system().openpty(PtySize{...})` — and runs a program in it: `$SHELL` (falling back to `/bin/sh`) with args defaulting to `["-l"]` for a login shell, at optional `rows/cols` (24×80 default), `cwd`, and `env`. A per-session reader thread pumps the shell's output as `{ev:"output", b64:...}` frames; `pty_write` feeds input, `pty_resize` changes the window size, and `pty_kill` ends it with an `{ev:"exit"}`. The important upgrade over the old single-shell reference is **multiplexing**: sessions are keyed by `id`, so `pty_write/pty_resize/pty_kill` route to the right terminal and one connection can hold several at once (an absent `id` addresses a single default session). It is a genuine login shell in a genuine PTY, driven from a web page — the piece with the most moving parts and the one the CDP probes (Chapter 20) spend most of their assertions verifying. Chapter 16 is the browser end of this pipe.

Packaging, security, and the library

`zwire` wires the host up as a standard Chrome native-messaging host: a manifest named `com.zwire.hud` (`type: "stdio"`) points `path` at the built binary and lists the allowed extension origins in `allowed_origins`, and `scripts/localinstall.sh` builds `zwire-host` and drops that manifest into the browser's `NativeMessagingHosts/` directory when it packages the `.app`. Because the binary is a single static Rust artifact, the consuming bundle has **zero runtime dependencies** — no system Python to be missing, no `psutil` to `pip install`, nothing to break on a fresh machine.

The daemon transport takes its exposure seriously, because it hands out `exec`, `fs`, and `pty`: the Unix socket is created `0600` inside a `0700` directory (owner-only), and inbound TCP peering is gated by a shared `--token` (or `$ZWIRE_HOST_TOKEN`) that a connection must present before anything privileged — local socket clients are trusted and skip it. `sysinfo` and `portable-pty` abstract the OS, so the same source builds for macOS, Linux, and Windows, and CI runs the four canonical polish gates (`fmt`, `clippy -D warnings`, `doc`, `test`) on all three. And because the whole thing is also a library (`zwire_host`), sibling hosts — `zpwichrome`'s among them — embed it to reuse the crawl/exec/stream machinery rather than reimplement it. Chapter 16 is the browser end of the PTY pipe.

Chapter 16 – The Terminal

The HUD ships a real terminal: an xterm.js front end (`lib/term/`) whose bytes come from the native host's PTY relay, bridged through the service worker, rendered as an overlay on any page and as a standalone page, and kept open across navigation. This chapter is the browser end of the pipe Chapter 15 opened.

xterm over the service-worker bridge

The terminal is xterm.js (`lib/term/xterm.js` + `terminal.js`) plus a boot script. A content script cannot open a native-messaging port that survives page navigation, and the PTY must outlive any single page, so the terminal's bytes flow through the background service worker: the worker holds the `connectNative` port to `com.zwire.hud`, sends `pty_spawn`, and relays `pty_write/pty_resize` down and base64 `output` up. The content-script terminal decodes the base64 into xterm and sends your keystrokes back as `pty_write`. Toggle it with `Ctrl+`` and it drops in as an overlay pane; there is also a standalone `pages/terminal.html`.

Staying open across navigation

A terminal that vanished on every link click would be useless, so `zterm-boot.js` mirrors the overlay's open state into `chrome.storage.local` under `zb_term_open`. Per-origin `localStorage` does not carry across different sites, so the boot script uses the cross-origin storage key instead: a `MutationObserver` on the pane's `active` class records every open/close, a `pagehide` handler snapshots the state right before navigating, and on the next page the overlay re-injects and re-opens automatically if it was explicitly open — and the worker reconnects the same tab's PTY, so the shell session continues across the navigation.

The open flag is deliberately strict and self-clearing. It re-opens only when `zb_term_open === true` (a real boolean, not a stale truthy value) and the pane is not already showing, and the background worker resets it to `false` on browser startup and on extension reload (Chapter 8's worker also owns this), so a stale “open” flag from a previous session never re-pops the terminal on every page of a fresh one. The result is a terminal that follows you from page to page within a session but does not haunt the next one — the browser-side half of a genuine shell, running the same login zsh you have in a real terminal, over the PTY the native host forked.

The transport shim and the xterm instance

The terminal front end is not zwire-original either; `terminal.js` is the shared `zpwir-embed-terminal` component, extracted verbatim from Audio-Haxor, and its own header says the only zwire-specific change is “a transport shim (auto-detects Tauri vs JUCE).” That shim, `TT`, picks a transport at load: a standalone extension page can `connectNative('com.zwire.hud')` directly, while a content-script overlay — which cannot open a native port — takes the `chrome.runtime.connect({name: 'zwire-pty'})` branch to the worker instead. Both branches speak the same four-message protocol (`pty_spawn`, `pty_write`, `pty_resize`, `pty_kill`) and decode the same base64 `output` frames into a `Uint8Array` that xterm writes directly, so the same component runs identically over the two paths.

The xterm instance itself is configured for the HUD: a blinking block cursor, a 13-pixel Hack Nerd Font (falling back through Share Tech Mono and Menlo), a fully transparent background so the scheme shows through, cyan cursor and selection, a ten-thousand-line scrollbar, and the sixteen ANSI colors set explicitly. There is no fit addon — the component measures the rendered cell size from xterm's own render service and computes

rows and columns by hand, re-fitting on a 50-millisecond- debounced `ResizeObserver` and sending each new size down as `pty_resize`. One piece of the component exists solely for release WebKit: because that engine ignores `setAttribute('style', ...)` on the DOM renderer's rows, a monkey-patch replaces xterm's `_addStyle` to set `el.style.color` and `el.style.backgroundColor` directly, so truecolor output actually paints — the same class of WebKit-release quirk the GUI apps in the suite fight constantly.

Docks, Ctrl-letters, and shell one-liners

The overlay is a draggable, dockable pane, not a fixed panel. It carries four dock positions (the corners), snaps to the nearest one on release using pixel-positioned zones (because “CSS calc() with percentages doesn't resolve in release WebView,” the comment notes), and persists its geometry. Two keyboard details make it usable as a real terminal inside a browser. First, `Ctrl+`` toggles it — deliberately not `Ctrl+T`, which is “left to Chrome for new-tab.” Second, when the terminal has focus a capture-phase handler forwards `Ctrl+<letter>` combinations straight to the PTY as their control codes (`Ctrl-A`, `Ctrl-E`, `Ctrl-K`, `Ctrl-U`, `Ctrl-W`), because macOS WKWebView would otherwise swallow them for its own Emacs bindings — so readline editing works in the browser terminal the way it does in a real one. And `zwireTermRun`, the entry point behind the palette's `shell` command type, opens the terminal and then polls for up to ten seconds until the PTY is alive before writing the command, so a one-liner fired the instant the terminal opens is never dropped before the shell exists.

Chapter 17 — zgui-core, the Shared Toolkit

Under the internal HUD sits `lib/zgui-core`, a git submodule that is the shared cyberpunk GUI toolkit for the whole MenkeTechnologies app suite. `zwire` consumes a slice of it — the fuzzy matcher, the command palette, the effect toggles, the fonts — and this chapter is what that library is and why `zwire` loads it from the submodule rather than copying it.

One toolkit, many consumers

`zgui-core` is, in its own words, the core GUI library shared across the app suite (Audio-Haxor, traderview, ztranslator, zpdf, zemail, zoffice, and more) — one canonical implementation of the shell, settings, dialog, table, and notification chrome so the many apps stop drifting their own divergent copies. Each component was extracted from an app that had already built it. It exposes a single namespace, `window.ZGui`, and its `webui/` directory holds on the order of 246 component scripts plus their CSS — a filterable card grid of buttons, inputs, charts, cyberpunk effects, and overlays that its own demo page renders.

What `zwire` uses

`zwire` does not load the whole library; it loads the handful of modules its content scripts depend on, in the manifest's canonical block:

- `fzf.js` — `ZGui.fzf`, the `fzf`-style fuzzy matcher with configurable scoring weights and a matched-character highlighter (`<mark class="fzf-hl">`). This is the engine behind the command palette (Chapter 12), the fuzzy find fallback (Chapter 13), and the fuzzy filtering on the HUD internal pages (Chapter 14).
- `command-palette.js` — `ZGui.palette`, the palette shell the `%K` list is built on.
- `util.js` — the shared helpers the above need.
- `fonts.css` + `webui/fonts/` — the shared typeface loader: `@font-face` for Orbitron (display) and Share Tech Mono (body/mono), with the `.ttf` files bundled under SIL OFL, so the HUD's font stack resolves the same way it does in every other app in the suite.
- `fx.js` — `ZGui.fx`, the global cyberpunk effect toggles (CRT scanlines, bezel vignette, neon glow, animations), each flipping a `body.no-*` class the theme reacts to — the mechanism behind the visual-effect prefs in `hud-ui.json`.

Submodule, not copy

`zgui-core`'s own guidance is emphatic: consume it as a git submodule and load the modules directly from the submodule path — *never copy* `webui/*`, because copies go stale and partial-sync. `zwire` follows that: `lib/zgui-core` is a submodule (declared in `.gitmodules`), and the manifest references files under `lib/zgui-core/webui/...` directly. This is the same reuse-not-copy discipline the `zpwchrome` power-tool and the native host follow, and it means an improvement to the shared fuzzy matcher or palette reaches `zwire` with a submodule bump rather than a hand-merge. The HUD's palette and find bar are not `zwire`-original UI code; they are the suite's shared components, wired to `zwire`'s data.

Inside the fuzzy matcher

`ZGui.fzf` is not a substring test; it is a scored fzf-style matcher, and its scoring is the reason the palette and find bar rank the way they do. Its own comment records the lineage — “EXTRACTED VERBATIM from Audio-Haxor” and “shared so the apps stop drifting their own copies.” The core, `fzfMatch(needle, haystack)`, is a case-insensitive subsequence match that, for each possible starting position of the first character, greedily takes the earliest subsequent matches and scores the run: each matched character is worth a base amount, adjacency to the previous match earns a consecutive bonus, and a gap between matches costs a start penalty plus a per-character extension penalty. On top of that come *position* bonuses that encode what humans mean by a good match — a character right after a non-word character scores a word-boundary bonus, a capital right after a lowercase scores a camelCase bonus, and the first matched character’s bonus is multiplied. The default weights are concrete (match 16, gap-start -3, gap-extend -1, boundary 9, non-word 8, camel 7, consecutive 4, first-char multiplier 2) and are themselves tunable and persisted, which is how the whole suite can retune ranking in one place. Above `fzfMatch` sits an extended query parser — space-separated AND terms, `|` for OR groups, and the fzf sigils `!` (negate), `'` (exact), `^` (prefix), `$` (suffix) — and a highlighter that wraps the matched characters in `<mark class="fzf-hl">`, which is the cyan emphasis you see on the matched letters in every list.

The palette shell and the shared readline

`ZGui.palette` is the list host the `%K` overlay is built on, and its ranking is a deliberate three tiers: static items and “strong” provider results are fzf-scored together at the top (ties favoring the static entries), “secondary” reference rows sit in the middle, and “fallback” rows — the generic web search — are pinned last no matter what you type, so a real command is never buried under a search suggestion. One more piece of `util.js` reaches every HUD input: on load it installs Emacs/ readline line-editing globally (`^A/^E` to move, `^W/^K/^U` to kill, `M-b/ M-f` by word, and the rest) and disables autocorrect and spellcheck on text fields. That is why the palette input, the find bar, and every settings field share the same terminal-style editing — it is one installed behavior from the shared toolkit, not a per-field reimplementation.

The visual-effects layer

The cyberpunk *look* — the grid lattice, the CRT scanlines, the bezel vignette, the neon glow — is also `zgui-core`, and it is built on one clean convention worth stating: an effect is *on* when its class is *absent*. `ZGui.fx` maps four effects to four body classes (`no-scanlines`, `no-vignette`, `no-neon-glow`, `no-anim`), and turning an effect off adds its class; the default, with no class present, is everything on. Each toggle persists to its own `zg-fx-...` `localStorage` key, and the Settings page bridges the same four toggles up to the native `hud-ui.json` so the new-tab extension follows.

The CSS behind the effects is unglamorous and worth demystifying, because none of it is magic. The grid lattice is two `linear-gradient` hairlines on a 40-pixel fixed-attachment cell, layered under three faint `radial-gradient` color blooms on the body background. The scanlines are a single `repeating-linear-gradient` on `body::after` — two transparent pixels, two barely-visible cyan pixels, repeated — plus, when the animated variant is on, two injected beam divs that sweep across on twelve- and eighteen-second loops. The vignette is one `radial-gradient` ellipse on `body::before` that is transparent to 58% and then darkens to the edges. And the neon glow is not a filter at all; it is pervasive `text-shadow` and `box-shadow` on themed elements. That last point is why the off switch is so blunt: `body.no-neon-glow *` sets `text-shadow` and `box-shadow` to `none !important`, `body.no-anim *` sets `animation` and `transition` to `none !important`, and the two overlay pseudo-elements are simply `display:none`. The effects are cheap layered CSS with a global kill switch, which is exactly why they can be toggled live without a reflow storm — and why the same look renders identically across every HUD surface that includes the one stylesheet.

Chapter 18 — `zpwrcode`, the Power-Tool

`extensions/zpwrcode` is the Manifest V3 power-tool, loaded as a **submodule** — reused, not copied — so `zwire` tracks upstream instead of forking a snapshot of it. This chapter covers what the extension provides, the native host its `pass` and segmented-download features need, and the cosmetic developer-extensions banner that unpacked loading produces.

Loaded as a submodule, not vendored

`zpwrcode` is its own public repository, and `zwire` consumes it through a git submodule at `extensions/zpwrcode` (declared in `.gitmodules`), not by copying its files into the tree. This is the same reuse-not-copy discipline `zgui-core` and the native host follow, and it has a concrete payoff: a fix or feature that lands in `zpwrcode` upstream reaches `zwire` with a submodule bump, and there is exactly one canonical copy of the extension rather than a frozen fork that drifts. `zwire` is a *host* for `zpwrcode`, not a second home for it.

The cost of a submodule is that it can be absent — a clone without `--recurse-submodules`, or a checkout where the submodule was not initialized, leaves `extensions/zpwrcode` empty. `zwire` handles this with the same graceful-skip rule the whole launcher uses: `bin/zwire` includes an extension directory in `--load-extension` only if it contains a `manifest.json`. An empty `zpwrcode` directory has no manifest, so it is silently skipped and `zwire` runs with just the new-tab and internal-HUD extensions. The power-tool is additive, not required; its absence degrades the browser, it does not break it. This is why the README can honestly say a fresh clone still runs.

Because it is a submodule and a public, separately-versioned project, this book does not re-document `zpwrcode`'s own feature surface in depth — that belongs to `zpwrcode`'s own documentation, and duplicating it here would go stale the moment upstream moves. What matters for `zwire` is the *seam*: how the power-tool is loaded (submodule, skipped-if-absent), how it stays color-synced with the rest of the HUD (the background worker pushes the scheme to it over `runtime.sendMessage`, and it can pull the current scheme or push a user pick back through the same bridge, since separate extensions cannot share storage — Chapter 8), and what it needs from the machine.

Native messaging and the developer-mode banner

Two operational notes close the chapter, both from the README's own notes section.

Native messaging. `zpwrcode`'s `pass` integration (the standard Unix password store) and its segmented-download feature both need `zpwrcode`'s native host installed — a separate step, documented in the `zpwrcode` project's setup, not something `zwire`'s launcher does for it. `zwire` wires *its own* HUD host (`com.zwire.hud`, for the terminal, system stats, and scheme file) automatically on launch; `zpwrcode`'s host is `zpwrcode`'s to install. The two hosts are independent, and the `allowed_origins` in the HUD host's manifest list all three extension ids (HUD, `zpwrcode`, new-tab) so the scheme bridge can be reached from any of them.

The developer-mode banner. Because the launcher loads the extensions unpacked via `--load-extension`,

Chromium shows its “disable developer-mode extensions” notice on startup. It is cosmetic. The extensions run fully — the banner is Chromium telling you that unpacked extensions are present, which is precisely what *zwire* intends. There is no way to load unpacked extensions on launch *without* the notice on current Chromium; *zwire* accepts it rather than working around it, and this book flags it so it is expected rather than alarming.

What the power-tool declares

This book does not re-document *zpwchrome*'s feature surface — that belongs to its own project and would go stale the moment upstream moved — but its manifest is worth reading once, because it is the concrete proof of Chapter 2's argument that only a genuine Chromium engine will do. *zpwchrome* is Manifest V3 with `minimum_chrome_version: 127`, and it declares nineteen permissions, among them the four that rule out every non-Chromium base: `userScripts`, `declarativeNetRequestWithHostAccess`, `nativeMessaging`, and `webRequest` (with `webRequestAuthProvider`). Alongside those sit `tabs`, `tabGroups`, `sessions`, `bookmarks`, `history`, `scripting`, `webNavigation`, `cookies`, `downloads`, `contextMenus`, `notifications`, `clipboardWrite`, `alarms`, and `storage`, over `<all_urls>` host access, with a module service-worker background and six content scripts. It binds fifty-four keyboard commands (its action opens on `Alt+T`), which is a scale of its own — and part of why *zwire* loads it as a submodule and lets it own its own shortcuts rather than trying to re-carry any of it.

The feature summary, from the power-tool's own README, is a browser toolkit behind one toolbar icon: a browserpass-compatible Unix `pass` integration (fill, copy, OTP, profile and credit-card autofill, through a vendored Rust native host that walks `~/.password-store`), a segmented multi-connection download manager that takes over Chrome's default via a HEAD probe and parallel range requests, a JetBrains-style most-recently-used tab switcher with scenes and an opener tree and fzf history over thousands of entries, a Tampermonkey-equivalent userscript engine, full-page screenshots, a technology detector, a theme injector, a lights-off dimmer, JSON and XML viewers, a user-agent switcher, and find-in-all-tabs. It needs its own native host for the `pass` integration and the segmented downloads — a separate install, independent of *zwire*'s HUD host — and the `allowed_origins` in *zwire*'s own host manifest lists all three extension ids so the scheme bridge is reachable from any of them. What matters to *zwire* is only the seam of Chapter 18: loaded as a submodule, skipped if absent, color-synced over `runtime.sendMessage`, and speaking to the machine through its own host.

Chapter 19 – The Launcher and State Layout

`bin/zwire` is the single entry point: it starts the base against `~/.zwire/profile` with `newtab`, `zpwchrome`, and `hud-internal` loaded and `--extensions-on-chrome-urls` set, passing any extra Chromium flag straight through. This chapter maps the launcher's flag assembly and the `$ZWIRE_STATE` directory layout.

What the launcher assembles

`bin/zwire` is a short, `set -euo pipefail` shell script, and reading it top to bottom is reading the entire runtime path:

1. **Resolve the base.** It checks `ZWIRE_BASE` (an executable override) first, then the path recorded in `~/.zwire/base.path`. If neither yields an executable binary it prints `zwire: no base browser installed – run ... fetch-base.sh` and exits non-zero. No base, no launch, with a message that names the fix.
2. **Build the extension list.** It iterates the three extension directories — `newtab`, `extensions/zpwchrome`, `extensions/hud-internal` — and includes each only if it contains a `manifest.json`, joining the survivors with commas. This is the graceful-skip rule: a missing submodule or deleted directory drops out of the list instead of breaking the launch. Note what is *not* here: `theme/` is deliberately excluded, for the palette-ordering reason Chapter 3 gave.
3. **Ensure the native host.** It looks for the HUD host manifest at `~/.zwire/profile/NativeMessagingHosts/com.zwire.hud.json` and reads the binary path out of it; if the manifest is missing or its target binary is gone, it runs `scripts/setup-native-host.sh` to build the Rust `zwire-host` and (re)write the manifest. This is what makes a bare `zwire` self-sufficient on Linux and in dev, not just the packaged macOS `.app`. It is never fatal — if the host cannot be built, the browser still launches, with the terminal and system stats disabled and a one-line note on stderr.
4. **exec the base** with a fixed flag set plus your arguments:

```
exec "$BASE" \
  --user-data-dir="$PROFILE" \
  --load-extension="$LOAD" \
  --extensions-on-chrome-urls \
  --test-type \
  --no-first-run \
  --no-default-browser-check \
  --homepage="chrome://newtab" \
  --disable-features=NtpFooter \
  --enable-features=SplitViewHorizontal,SplitViewTabRestore \
  "$@"
```

Each flag earns its place. `--user-data-dir` is the profile isolation the whole design rests on. `--load-extension` is the switch unbranded Chromium keeps and branded Chrome removed — the reason for the whole base choice. `--extensions-on-chrome-urls` lets the internal-HUD content scripts run on `chrome://` pages. `--test-type` suppresses the automation-restriction warnings that would otherwise fire with these flags. `--no-first-run` and `--no-default-browser-check` skip the onboarding

and the default-browser nag. `--homepage=chrome://newtab` points the home button at the HUD new tab. `--disable-features=NtpFooter` removes the new-tab footer that would sit under the HUD page. `--enable-features=SplitViewHorizontal,SplitViewTabRestore` turns on Chromium's side-by-side split view and its restore-on-reopen. And `"$@"` passes *everything you typed* straight through — so `zwire https://github.com` opens a URL, `zwire --incognito` starts incognito, and, relevant to Chapter 20, `zwire --remote-debugging-port=9222` opens the CDP endpoint the verification probes attach to. The launcher owns the HUD-defining flags and gets out of the way of every other Chromium flag.

Reading the launcher line by line

`bin/zwire` is short enough to hold in the head, and a few details below the summary are worth naming. It resolves its state directory by sourcing `state-dir.sh` and honoring `$ZWIRE_STATE`; then it runs the one-time legacy migration described in the next section. Base resolution has an exact precedence — an executable `$ZWIRE_BASE` first, then the path recorded in `$ZWIRE_STATE/base.path`, then nothing — and if nothing resolves to an executable it prints, verbatim, `zwire: no base browser installed - run: .../scripts/fetch-base.sh` and exits non-zero. The extension list is the three-directory loop of Chapter 3, filtered to directories that actually contain a `manifest.json` and joined with commas, with `theme/` conspicuously absent.

The native-host ensure step is more careful than “run the setup script.” The launcher reads the `"path"` field out of the existing `com.zwire.hud.json` with a one-line `sed`, and it re-runs `setup-native-host.sh` only when the manifest is missing or its recorded binary is no longer executable — a dangling manifest, not just an absent one, triggers a rebuild. And it is never fatal: if the host cannot be built, the launcher prints `zwire: native host unavailable (terminal/sysinfo disabled) - see scripts/setup-native-host.sh` to stderr and launches the browser anyway, minus the terminal and the system stats. That single property — the browser runs even when the host, a submodule, or a whole extension is missing — is the graceful-degradation rule made concrete in the one script every launch goes through.

The `$ZWIRE_STATE` directory

All of `zwire`'s state lives under `$ZWIRE_STATE`, and — a correction the earlier printings of this book owe the reader — its *default* is not a dotdir in `$HOME`. `scripts/state-dir.sh` resolves the OS's regular application-data location: `~/Library/Application Support/com.menketechologies.zwire` on macOS — named by the reverse-DNS bundle id, matching the `.app`'s `CFBundleIdentifier` and the platform convention that Application Support folders carry the bundle id — and `${XDG_CONFIG_HOME:-~/.config}/zwire` on Linux, with `$ZWIRE_STATE` overriding on every platform. There are two *legacy* locations earlier builds used: the original `~/.zwire` dotdir, and a brief intermediate macOS build that used the bare `~/Library/Application Support/zwire` name before it moved to the bundle id. `bin/zwire` does a one-time `mv` of whichever leftover it finds into the new state dir when no explicit `$ZWIRE_STATE` is set, so an upgrade keeps your profile, base, and scheme intact and then never touches the old dir again. The theme is the one exception to this per-install dir: the shared colour scheme and light/fx prefs live in a *fleet-wide* location, `~/.zwire` (the host's `global.toml`, overridable via `$ZWIRE_GLOBAL_DIR`), read and written by every MenkeTechnologies app so they all follow one look, and the C++ color patch (fork 0002) resolves that same `~/.zwire` natively — so the scheme file the host writes is the file the browser reads. `~/.zwire` is therefore a real, distinct directory now, not merely shorthand for `$ZWIRE_STATE`. The per-install tree, rooted at `$ZWIRE_STATE`, is a small, legible directory:

```

$ZWIRE_STATE/      (macOS: ~/Library/Application Support/com.menketotechnologies.zwire .
└─ Linux: ~/.config/zwire)
  base/             the extracted Chromium (or the packaged fork)
  base.path         absolute path to the resolved binary (the launcher reads this)
  base.version      the pinned revision, e.g. "r1656770" or "fork-150.0.7871.46"
  profile/          the dedicated --user-data-dir (bookmarks, history, sessions)
    NativeMessagingHosts/com.zwire.hud.json  the HUD native host manifest
    Default/NativeMessagingHosts/com.zwire.hud.json  second location Chromium checks
  ext/              per-user copy of the loaded extensions, staged at launch

```

The theme is deliberately *not* in this tree. The shared scheme, light/fx prefs, and any custom schemes live in the fleet-wide `~/.zwire/global.toml` (with the host's plain-text `hud-scheme/hud-light` projections beside it for the fork's C++ mixer), so every zwire-host client on the machine — the browser, the new tab, `zpwchrome`, and the wider app suite — reconciles to one theme file rather than a per-install copy.

Two of the entries above repay a second look. `profile/` is the isolation boundary — a complete, separate browser profile that never shares a byte with system Chrome, holding your zwire bookmarks, history, sessions, and the extensions' own stored state (`zb_scheme`, `zb_keys`, `zb_custom_cmds`, `zb_term_open`, and the rest). And `ext/` exists because Chromium writes extension `_metadata/` indexes *into the extension directory*, which means loading extensions straight out of a read-only `.app` bundle fails and, on a multi-user machine, two users writing the same directory would collide. So the loaded extensions are staged into a per-user writable copy under `ext/` at launch, and each user gets their own. The `base.version` file is the one place that records whether you are on a snapshot (`r...`) or a compiled fork (`fork-...`), which is the single bit that distinguishes the two build paths on a running machine.

`ZWIRE_STATE` being an environment variable means the whole tree relocates: point it at a different directory and you get a second, independent zwire — a different profile, a different base, a different native host manifest — without touching the first. That is the same isolation principle one level up: as zwire keeps its state out of system Chrome, `ZWIRE_STATE` lets you keep one zwire's state out of another's.

Chapter 20 – The CDP Verification Layer

The top-level `v*.mjs` scripts — `vkeys`, `vovl`, `vpl`, `vpop`, `vpos`, `vsys`, `vterm`, `vwin` — are **verification probes**. Each connects to a running zwire's Chrome DevTools Protocol endpoint on `127.0.0.1:9222`, attaches to a live page or extension page, and asserts that a specific HUD feature is actually wired and rendering. They are not the overlay itself — the vim keys, the tmux tiling, the statusbar, the palette, and the terminal all live in the `hud-internal` content scripts. The `v*.mjs` scripts are how you *check*, from the outside, that those content scripts did their job in a real browser.

Talking to a live browser over CDP

Every probe shares the same eight-line bootstrap, and understanding it once explains all of them. The launcher does **not** open a debugging port by default; you opt in by passing it through:

```
zwire --remote-debugging-port=9222
node vterm.mjs
```

With the port open, a probe does this: `fetch http://127.0.0.1:9222/json/version` to get the browser's `websocketDebuggerUrl`, open a `WebSocket` to it, and speak CDP over that socket. Commands are JSON objects with a monotonic `id`, a `method`, `params`, and (once attached to a target) a `sessionId`; responses come back tagged with the same `id`, and a small map of pending resolvers matches each reply to its request. To reach a page the probe calls `Target.attachToTarget` with `flatten:true` for a session id, enables `Runtime`, and then evaluates expressions in the page with `Runtime.evaluate (returnByValue:true)`, reading back whatever the HUD put in the DOM. Some probes open a fresh target first (`Target.createTarget` at an extension page URL) and some find an existing page in `/json/list` (matching a stable site like `github/amazon/wikipedia`) to attach to. `9222` is the wire because that is the conventional CDP port and what the probes hardcode; the browser only listens on it when you passed the flag.

That these are external CDP probes rather than in-page code is the point: they verify the HUD the way a black-box test would, driving a genuinely running browser and asserting on its real DOM, which is the only way to prove the multi-hop paths (page → content script → service worker → native host → PTY → shell) actually connect end to end.

The modules and what each verifies

Each probe targets one HUD surface and prints a short list of assertions:

- `vkeys.mjs` opens the Keyboard settings page (`pages/keys.html`), confirms `ZWIRE_KEYMAP` is present, counts the editable key chips and category headers, then *exercises a remap*: it writes `zb_keys.scrollDown='s'` into `chrome.storage.local` and reads it back, proving the remap round-trips through storage where `zvim` would pick it up.
- `vterm.mjs` opens the standalone terminal page (`pages/terminal.html`), checks that xterm's `Terminal` global loaded and the pane rendered, waits several seconds for a heavy zsh login to boot over the native-host PTY, and reports whether rows of real shell text appeared.

- `vovl.mjs` and `vpop.mjs` attach to an ordinary web page, confirm the terminal content script injected a (hidden) `#terminalPane`, synthesize a `Ctrl+`` keystroke via `Input.dispatchEvent`, and verify the overlay becomes `active` and xterm renders — `vpop` waits longer and reads back the booted shell's visible text.
- `vwin.mjs` triggers the terminal from a page's `Ctrl+``, then looks in `/json/list` for the terminal popup window that opened, attaches to it, and confirms xterm rendered and `chrome.runtime.connectNative` is reachable from it.
- `vsys.mjs` waits for the native host's system-stats stream to fill `zb_sys`, then reads each statusbar segment — CPU, MEM, NET, HOST, LAN, UP, BATT, the powerline chevron count, and the clock — proving the real-machine stats path is live.
- `vpos.mjs` and `vpl.mjs` verify statusbar *layout*: `vpos` reads the left-to-right segment order and checks the `C-b` prefix sits left of the scheme segment; `vpl` inspects the powerline chevron pseudo-elements (the `::after` border that draws each chevron) and the prefix block's background, confirming the powerline geometry renders as intended.

Taken together the eight probes cover the runtime HUD's riskiest surfaces — the remap round-trip, the terminal's full native-PTY path, the overlay toggle, the system-stats stream, and the statusbar's powerline geometry — from outside the browser, over the same protocol any external tool would use. They are the runtime path's answer to the fork path's `git apply --check`: a concrete, re-runnable proof that the thing is actually wired, not merely written.

Two probe shapes, and no test framework

The probes come in two shapes that reflect what they need to reach. `vkeys` and `vterm` *create* a fresh target at an extension page URL (`chrome-extension://<hud-id>/pages/keys.html` or `pages/terminal.html`) and assert against it, because the keymap page and the standalone terminal are extension pages you can open in isolation. The other six *attach to an existing tab* — they scan `/json/list` for a real page whose URL matches `github`, `amazon`, or `wikipedia`, because the overlay, statusbar, and terminal content scripts only inject into ordinary web pages, and a probe needs a genuine one to inject into. When no such tab is open, those probes exit cleanly rather than failing, since “no stable page” is a setup gap, not a broken feature.

There is no assertion framework: each probe evaluates an expression in the page and `console.logs` the boolean or the count, and pass or fail is read off the printed lines by a human or a CI grep. The overlay probes synthesize the real toggle by dispatching a `Ctrl+`` key event through `Input.dispatchEvent` (with the Ctrl modifier) and then wait — five seconds for `vovl`, seven for `vpop`, six for `vwin` — for a heavy zsh login to boot over the full page → content-script → worker → native host → PTY path before reading back the shell's visible text, which is the only way to prove that multi-hop chain end to end. Two probe lines are honestly dead weight, and this book flags them rather than pretend the suite is airtight: `vterm` reaches for a `window.__zbTermInstance` global that does not exist (the real instance is module-scoped), and `vovl` prints a placeholder string for the `zb_term_open` check instead of actually reading storage. They do not weaken the probes that matter, but they are there, and a book grounded in the code says so.

Chapter 21 – The Source Fork

`fork/` compiles a patched Chromium so the *whole* browser — tab shapes, neon toolbar, HUD-colored frame, native menus and dialogs, and a browser-wide audio engine — lives in the HUD, at the cost of a ~100 GB checkout and a one-to-four-hour first build. All twenty-four HUD patches are authored against the pinned tag `150.0.7871.46` and verified apply-clean (58 hunks across 31 Chromium files). This chapter is the pipeline and the patch series, with one real hunk quoted so the level of the work is clear.

The pipeline: fetch, apply, build, package

The fork is four scripts run in order, and each is a thin, honest wrapper around the Chromium build tools:

1. `fork/fetch.sh` installs `depot_tools` if missing, `fetch --no-history chromium` into `$ZWIRE_SRC` (default `~/zwire-chromium`), checks out the exact tag from `fork/CHROMIUM_VERSION` onto a `zwire-<tag>` branch, and `gclient syncs` the deps for that tag. `--no-history` keeps the checkout to “only” ~100 GB rather than far more. This is the long, network-bound step.
2. `fork/apply-patches.sh SRC [--reverse]` reads `fork/patches/series` (an ordered list with in-line comments stripped) and applies each patch with `git apply --3way` from the Chromium `src/` dir; `--reverse` unapplies them in reverse order, which is how you back the patches out before rebasing onto a new Chromium tag.
3. `fork/build.sh SRC [OUT]` copies `fork/args.gn` into `out/<OUT>` (default `zwire`), runs `gn gen`, and then `autoninja -C out/<OUT> chrome`. That last command is the long compile — one to four hours the first time, minutes for an incremental rebuild after a single patch.
4. `fork/package.sh OUTDIR` installs the freshly built app as the `zwire` base: on macOS it finds the built `.app` (named `zwire.app` if the branding patch ran, else `Chromium.app`), copies it into `~/zwire/base` under its own name, derives the real executable from the bundle’s `CFBundleExecutable`, writes `base.path` and a `fork-<tag>base.version`, and then runs the same `scripts/rebrand-macos.sh` the runtime path uses. After this the same `bin/zwire` launches the HUD-chromed fork with the extensions preloaded — same launcher, forked base.

`fork/args.gn` configures the build: an *unbranded* Chromium (`is_chrome_branded=false`), an official (fully optimized) release build (`is_official_build=true`), with the slow parts turned off for a workstation build time — `symbol_level=0` and `blink_symbol_level=0` (no debug symbols, faster link), `chrome_pgo_phase=0` (skip profile-guided optimization), `enable_nacl=false` (drop legacy Native Client), and `use_thin_lto=false` (ThinLTO is slow for iteration). You set `target_cpu` to `arm64` or `x64` for your host. These are the knobs that turn a multi-hour Chromium build into merely a long one.

What “apply-clean” proves, and what it does not

The fork’s own docs state the boundary plainly, and the book repeats it because it is the honest status of the fork. Each patch was authored by reading its target file at the exact tag on the Chromium GitHub mirror (`raw.githubusercontent.com/chromium/chromium/150.0.7871.46/...`) and is verified with `git apply --check` against that tree. That proves the diff *context matches the pinned source* — the patch will apply without a reject. It does **not** prove the C++ *compiles*; `fork/build.sh` is that gate. So the series’ status is “all twenty-four authored and apply-clean against `150.0.7871.46`,” which is a real, checkable claim, and not the

stronger “compiled and shipped” claim, which the fork does not make. A CI check, `scripts/check-patch-series.sh`, keeps the `series` file and the patch files on disk consistent so the two never drift.

The twenty-four HUD patches

The patches live in `fork/patches/`, are listed in `series`, and each targets one native surface — a look a theme extension cannot restyle, or a behavior it cannot override. They fall into six groups: **nine styling** patches (0001–0009), **six behavior** patches that force zwire’s defaults over Chromium’s (0010–0015), **two native-color** patches that bind the native Views surface to the HUD palette (0020–0021), **three extension-compat** patches that keep the HUD’s own pages and the Chrome Web Store working (0016, 0018, 0019), **one quit** patch (0017), and the **three audio** patches that compile a browser-wide DSP engine into the audio service (0022–0024). Read against the target-file map in `fork/patches/README.md`:

0001 — Tab shape (`chrome/browser/ui/views/tabs/tab_style_views.cc`, in `TabStyleViewsImpl::GetPath`). The patch is more than a corner-radius tweak: it replaces the rounded-rect tab path with a *hard trapezoid* — the classic-Chrome tab silhouette with straight diagonal sides — while keeping the interior clip and hit-test paths rectangular so the favicon and title are never clipped by the sloped edges. The real hunk:

```
- path.lineTo(tab_left, tab_top + top_left_corner_radius);
- path.arcTo(SkVector(top_left_corner_radius, top_left_corner_radius), 0,
-           SkPathBuilder::kSmall_ArcSize, SkPathDirection::kCW,
-           SkPoint(tab_left + top_left_corner_radius, tab_top));
+ // zwire HUD: diagonal left side up to the inset top-left corner.
+ path.lineTo(tab_left + hud_slope, tab_top);
```

The slope is $(\text{tab_bottom} - \text{tab_top}) * 0.28$, applied on each side, so the tab is a trapezoid whose top edge is inset from the bottom. The patch also nudges the favicon and title 8px to the right so they sit clear of the hard trapezoid’s sloped left edge rather than crowding into it. This is a real Views-layer restyle, not a color change — exactly the kind of thing the runtime path cannot do.

0002 — UI colors (`chrome/browser/ui/color/chrome_color_mixer.cc`, in `AddChromeColorMixer`, before the high-contrast early-return). Forces the cyberpunk palette onto frame, toolbar, toolbar text and icons, tab background and foreground, and omnibox background and border, and recolors the under-toolbar separator neon cyan, before the high-contrast early-return. The patch has since grown to seed the eight HUD schemes as Sys-token palettes and to end with a dead-last `AddZwireHudFinalMixer` that re-asserts the tab colors so the Material/native tab-strip mixers can’t leave the active tab a scheme-independent default. This is the patch that reaches everything the theme extension cannot, and the one the unloaded `theme/` extension would fight if it were loaded. Two later refinements live in the same patch. First, the native chrome now follows **light mode**: the mixer reads the `$ZWIRE_STATE/hud-light` projection the host writes when `[theme.ui].light` flips (Chapter 8), swaps the dark scheme values for the light variant, and repaints at startup so the choice applies on launch rather than only after a toggle. Second, the **focus ring follows the scheme** — the patch overrides Chromium’s stock GM3 red focus ring with the active scheme’s accent, so the ring matches the HUD instead of clashing with it.

0003 — UI font. The target is `ui/base/resource/resource_bundle.cc`, in the `InitDefaultFontList` method. It sets `gfx::FontList::SetDefaultFontDescription("Share Tech Mono, Monaco, 13px")` so all native chrome UI text renders in the HUD mono face, with Monaco as a guaranteed-present fallback before Share Tech Mono is installed.

0004 — Toolbar neon border (`chrome/browser/ui/views/toolbar/toolbar_view.cc`, in `Init`, the `kNormal` branch). Adds a 2px cyan bottom border via `views::CreateSolidSidedBorder(..., kColorToolbarContentAreaSeparator)`; the separator color itself is set neon cyan back in 0002, so the two patches together make the under-toolbar line glow.

0005 — Omnibox field. The target is `chrome/browser/ui/views/location_bar/location_bar_view.cc`, in the `ComputeBorderRadius` method. It clamps the field radius to 2px so the omnibox is the

sharp HUD field instead of the rounded pill; its background and border colors come from 0002.

0006 — Product strings (`chrome/app/theme/chromium/BRANDING`). Rewrites the product names `Chromium` → `zwire`, the company to `MenkeTechnologies`, and the bundle id to `com.menketechologies.zwire`. This is the file the unbranded (`is_chrome_branded=false`) build reads, which is why the fork's `package.sh` looks for a `zwire.app` and only falls back to `Chromium.app`.

0007 — DevTools theme (three DevTools front-end files: `design_system_tokens.css` appended, `main-meta.ts` for the `ui-theme` setting, and `ThemeSupport.ts` for `applyTheme`). Adds the eight HUD schemes to the DevTools Theme dropdown, default `zbhud-cyberpunk`. `ThemeSupport` maps `zbhud-<scheme>` onto the `dark` base and stamps `[data-zbhud=<scheme>]` on `<html>`; the appended CSS maps the `--sys-color-*` tokens onto per-scheme `--zb-*` key colors. Without `data-zbhud`, stock Light/Dark/Auto are untouched — the patch adds schemes rather than replacing the existing ones.

0008 — Allow framing (`content/browser/renderer_host/ancestor_throttle.cc`). Bypasses `X-Frame-Options` and frame-ancestors CSP so the tmux overlay (`ztmux.js`) can iframe any site into its N-pane tiling. This patch *replaces* an earlier `0008-tmux-split-prefix` patch, whose native 2-pane split was superseded by the JS overlay — the old patch consumed the `Ctrl-b` prefix and the next key before the page saw them (the “C-b %%” double-tap bug), so the native split was removed and the real N-pane tiling moved entirely into the content script, which just needs framing allowed.

0009 — Ext-command focus (`chrome/browser/ui/views/extensions/extension_keybinding_registry_views.cc`). When an extension command fires while a chrome UI surface (the omnibox or search bar) has focus, this hands focus to the page web contents so the `⌘K` palette overlay is typeable from the omnibox — closing the same focus gap the new-tab page solves in JavaScript, but at the engine level for ordinary pages.

0010 — Force zwire new-tab (`chrome/browser/search/search.cc`, in `HandleNewTabURLRewrite`). When the zwire new-tab extension is loaded, `chrome://newtab` always resolves to it instead of the default Google NTP, so the new tab is always the HUD — even the startup tab. This is the engine-level partner to the extension's own `chrome_url_overrides.newtab`: the override alone loses races on the very first tab, so the rewrite pins it.

0011 — Pinned extensions always show (`chrome/browser/ui/views/toolbar/toolbar_view.cc`). Holds the extensions container at its preferred width so pinned actions never collapse into the puzzle overflow when an extension is re-pinned at runtime — the zwire toolbar keeps its pinned tools visible instead of hiding them behind the generic extensions menu.

0012 — Allow frame-src (`content/browser/renderer_host/navigation_request.cc`). A host page's `frame-src` / `child-src` CSP never blocks a sub-frame navigation, so the tmux overlay can embed any site into a pane even when the top page's CSP would forbid it. This complements 0008: 0008 bypasses the *framed* site's frame-ancestors defense, and 0012 bypasses the *framing* page's frame-src restriction — both directions of the CSP handshake have to yield for arbitrary panes to work.

0013 — Restore last session (`chrome/browser/prefs/session_startup_pref.cc`). Sets the default startup preference to restore the last session, so tabs auto-restore on launch rather than opening a single fresh new-tab.

0014 — No session-crashed bubble (`chrome/browser/ui/views/session_crashed_bubble_view.cc`). Never shows the “Restore pages?” session-crashed bubble — the restore-last-session default (0013) already brings the tabs back, so the prompt is redundant noise the HUD suppresses.

0015 — No navigation block (`extensions/browser/extension_navigation_throttle.cc`). Never blocks a `chrome-extension://` navigation to a not-yet-registered extension, so the new-tab override page always loads at startup even in the window before its extension has finished registering — the engine-side race fix that makes 0010's forced new-tab reliable on the very first paint.

0016 — Ext private-API allowlist (`chrome/common/extensions/api/_permission_features.json`). Allowlists the `hud-internal` extension (by hash) for `developerPrivate` — widening the API's `extension_`

`types` to include `"extension"` — and `settingsPrivate`, so the HUD's own Extensions and Settings pages can drive `chrome.developerPrivate` and `chrome.settingsPrivate` the way the built-in `chrome://extensions` and `chrome://settings` do. Without it the reimplemented pages have no engine to talk to.

0017 — No hold-to-quit (`chrome/browser/app_controller_mac.mm`, in `runConfirmQuitPanel`). Forces the hold-⌘Q-to-quit gesture *off* — immediate quit — regardless of the `kConfirmToQuitEnabled` pref, which a synced profile can otherwise flip back on. A daily driver quits when you press quit.

0018 — Null policy-container crash fix (`extensions/browser/extension_function_dispatcher.cc`). Gates the `is_sandboxed` UMA probe on `RenderFrameHost::IsActive()` so an extension-function IPC arriving from a non-active frame (with a null `policy_container_host_`) cannot reach the `active_sandbox_flags()` `NOTREACHED`. This fixes a real SIGTRAP: a content script calling a `chrome.*` API on `chromewebstore.google.com` mid-navigation would crash the browser without it.

0019 — Web Store scripting (`chrome/common/extensions/chrome_extensions_client.cc`, in `IsScriptableURL`). Drops the gallery special-case (`kCannotScriptGallery`) so extension content scripts and `tabs.executeScript` run on the Chrome Web Store domains like any other page — store-privileged JavaScript bindings stay gated by `IsWebstoreOrigin` in `permissions_data.cc`, so this widens *scripting*, not *privilege*.

0020 — Native menu colors (`chrome/browser/ui/color/chrome_color_mixer.cc`, in `AddZwireHudFinalMixer`). Binds the `ui::kColorMenu*` family — the app/three-dot menu, context menus, and dropdowns — to the HUD Sys tokens (`kColorSysBaseContainer` and friends) so native Views menus follow the scheme and the light/dark toggle instead of the `NativeTheme` default that left light text on a light background. References, not literals: it depends on 0002's Sys-token block.

0021 — Native primitive colors (same file and mixer). Generalizes 0020 to the whole native surface. HUD light mode recolors the Sys tokens but never flips `NativeTheme`, so `core_default_color_mixer` still builds every native-Views *primitive* (`kColorPrimary/Secondary`, the background and midground endpoints) from the OS color mode — which inverts dialogs, bubbles, textfields, and tooltips when the HUD and OS modes disagree. Rebinding those primitives to the HUD-mode-aware Sys tokens makes the entire native tree track HUD light/dark + scheme in one place.

0022 — Output audio EQ (`services/audio/output_controller.{cc,h}` plus `chrome/browser/chrome_content_browser_client.cc`). The first of three patches that give zwire a **browser-wide equalizer**. A preamp + RBJ-biquad cascade (`ZwireAudioEq`) is applied in `OutputController::OnMoreData()` — the audio service's universal chokepoint that every output stream passes through before the OS device — so all tab audio (media element, MSE/YouTube, Web Audio, WebRTC) is equalized, not just the subset the renderer mixer sees. The chain is preamp → EQ cascade → gain → `tanh` soft-clip drive per channel, then an equal-power pan / mono-fold across the stereo pair. It is always on: an empty config is unity gain (audio still routed through it), and only `"off"` bypasses. Because the re-signed macOS audio utility is sandboxed and does not inherit the environment, `ChromeContentBrowserClient::AppendExtraCommandLineSwitches` forwards the config as the `--zwire-audio-eq` switch (switches survive the sandbox; env does not).

0023 — Audio tab-capture (`chrome/browser/extensions/api/tab_capture/tab_capture_api.cc` plus `chrome/common/extensions/api/_api_features.json`). Lets the HUD Audio page pull a playing tab's *real* output into its analyzer: it exposes `tabCapture` to `hud-internal` without the manifest permission (a second allowlisted feature variant, mirroring 0016) and defaults the allowlisted extension id so `getMediaStreamId` bypasses the `activeTab` picker and prompt. The grant is scoped to that one id.

0024 — Live audio config + meters (`services/audio/public/mojom/audio_service.mojom`, `services/audio/service.{cc,h}`, `content/browser/audio/audio_service.cc`). Makes the 0022 EQ **live** with nothing open and no relaunch. The audio sandbox cannot read the config file, so the unsandboxed browser process polls it off-thread and, on change, pushes the spec over a new `mojom.AudioService::SetZwireEqConfig`, which atomically swaps the process-global config the cascade reads each block. The same patch carries the **meters** back-channel: the service computes a compact post-DSP

frame (a Goertzel spectrum, peak/RMS per channel, phase correlation, a decimated stereo scope) of the real output and pumps it — on a dedicated ThreadPool sequence, off the UI thread so trackpad gestures cannot starve it — to `$ZWIRE_STATE/meters`, which the HUD Audio page reads over the native host. Because the page reads the engine's true output rather than a `tabCapture` of the source, watching the meters never mutes or drops the audio.

The rebase cost

The rebase story is the fork's ongoing cost and the docs do not hide it: the `radius-calc`, `color-mixer`, `Init`, and `ComputeBorderRadius` anchors shift between Chromium milestones, so most releases you re-anchor most patches. The procedure is in `fork/patches/README.md` — bump `CHROMIUM_VERSION`, reverse the series off the old tree, fetch the new tag, re-apply and fix any `.rej`, and re-build. You can re-verify an anchor without a full checkout by reading the target file at the new tag from `raw.githubusercontent.com/chromium/chromium/<TAG>/<path>` and `git apply --check`-ing against it. That is the solo-fork commitment, and it is the price of putting the native chrome in the HUD.

The build config, and the series discipline

`fork/args.gn` is the whole build configuration, and every knob is a deliberate trade between build quality and build time on a single workstation:

```
is_chrome_branded      = false    # ships as unbranded Chromium, not Google Chrome
is_official_build       = true     # full release-quality optimization
proprietary_codecs      = true     # H.264 / AAC / MPEG in <video>/MSE
ffmpeg_branding         = "Chrome" # bundle the ffmpeg build that carries them
enable_platform_hevc    = true     # allow HEVC where the OS provides it
enable_platform_hevc_decoding = true
is_debug                = false
is_component_build      = false
symbol_level            = 0        # no debug symbols -> much faster link, smaller
blink_symbol_level      = 0
chrome_pgo_phase        = 0        # skip profile-guided optimization (a large time sink)
enable_nacl              = false    # drop legacy Native Client
use_thin_lto            = false     # ThinLTO is slow; off for iteration builds
target_cpu              = "arm64"  # set to your host: "arm64" | "x64"
dcheck_always_on        = false
```

The first two lines are the identity — unbranded so there is no Google logo and the `BRANDING` strings of patch 0006 take effect, official so the result is a real release build; the last of the trailing block are the knobs that turn a multi-hour Chromium compile into merely a long one by dropping symbols, PGO, NaCl, and ThinLTO.

The codec block is the one place the fork's *behavior* differs from a stock Chromium build. Stock Chromium ships `ffmpeg_branding="Chromium"` — open codecs only (VP8/VP9, AV1, Opus, Vorbis) and no H.264 or AAC — so an H.264 stream that plays in Google Chrome plays in the zwire fork's stock base only as silence and a black frame. Setting `proprietary_codecs=true` with `ffmpeg_branding="Chrome"` bundles the ffmpeg build that carries H.264/AAC/MPEG, so the fork decodes what Chrome does: Synology SurveillanceStation web live-view, most IP cameras, and ordinary MP4/AAC media. HEVC (H.265) is *not* software-decoded even here — Chromium routes it to the platform hardware decoder (VideoToolbox on macOS), which the two `enable_platform_hevc[_decoding]` flags wire up. This is a fork-only capability by construction: the runtime rebrand runs the prebuilt open-codec snapshot and cannot add codecs without recompiling.

The patch series has its own discipline, enforced in CI. `fork/patches/series` is the ordered list `apply-patches.sh` reads, and `scripts/check-patch-series.sh` asserts a strict one-to-one correspondence between the entries in that file and the `*.patch` files on disk — a patch on disk but missing from `series` would silently never apply, and a `series` entry with no file would fail to apply, and the check catches both drifts

before they ship. Each series entry carries an inline note recording that it was verified apply-clean against `150.0.7871.46`, and one entry is instructive by its absence: an earlier `0008-tmux-split-prefix.patch` is commented out with an explanation, because its native two-pane split “consumed the prefix + the next key before the page saw them (the ‘C-b %%’ double-tap)” and was superseded by the JS overlay — its `0008` number reused by the framing patch. The series file is not just a manifest; it is a small changelog of what the fork tried and backed out.

The economics of the rebase

The runtime rebrand tracks upstream Chromium for free; the fork does not, and the difference is the single most important thing to understand before choosing the compiled path. A stock snapshot is a security fix one `fetch-base.sh` away because it is not a fork of anything — it *is* the upstream artifact, re-signed and renamed. The fork, by contrast, is twenty-four patches pinned to one tag, and every time Chromium moves, those patches must be carried forward across a codebase that changes under them without warning. That carrying-forward is the recurring cost, and it is worth being precise about where it falls, because it does not fall evenly across the series.

The patches sort into three risk tiers by *what* they touch. The cheapest are the string and resource patches — the `BRANDING` file, the product name, the icon — which live in files that change slowly and rarely conflict; a rebase moves them with a trivial context shift or none at all. The middle tier is the CSS-and-color work: the trapezoid tab shapes, the neon under-toolbar line, the Share Tech Mono UI font, the frame color wired to the scheme. These live in the Views layer, which Google refactors on its own schedule, so they conflict *sometimes* — a renamed method here, a moved constant there — but the conflicts are mechanical, because the patch’s intent is legible from its diff and the new upstream shape is usually obvious. The expensive tier is the behavioral C++: the `is_sandboxed` gate in the function dispatcher, the three audio-service patches that reach into `OutputController::OnMoreData` and add a new `mojom` method, the codec branding. These touch code that carries real invariants, and a rebase there is not “fix the context” but “re-understand the surrounding code well enough to know the patch is still correct” — because upstream may have changed the very assumption the patch depended on. The audio EQ that hooks the universal mixing callback is the canonical example: if Chromium reworks how the audio service routes output, the patch does not fail to apply — it applies and is *wrong*, which is the worst outcome a rebase can produce.

This asymmetry is why the fork is framed throughout the book as the tier for whoever will *pay the build-and-rebase cost*, not merely the build cost. The build is a one-time multi-hour compile made merely long by the `symbol_level=0` and no-PGO knobs; the rebase is a recurring tax that scales with how far the pin has fallen behind. The disciplines that make it survivable are exactly the ones this chapter described: the `series` file as a manifest CI keeps honest, the inline apply-clean notes that record which tag each patch was last verified against, and the willingness — visible in the commented-out `0008` — to back a patch out rather than carry a fragile one forward. The honest reading is that the fork is a maintained thing, not a finished one, and the runtime rebrand exists precisely so that the daily driver never has to be. The line between the two tiers is not a wart; it is the boundary between what upstream will keep working for you and what you have chosen to keep working yourself.

Chapter 22 – Dev Loop, Testing, and CI

zwire is a browser you assemble, not compile, so its “build” tooling is a set of small shell scripts that fetch, rebrand, cache-bust, and reset the base, plus a JS syntax gate and a CI that lints every script and validates every JSON. This chapter is that loop.

The script set

The `scripts/` directory is a family of dev-loop commands, all sharing one cyberpunk banner wrapper (`cyberpunk.sh`, which just sets ANSI color variables and a banner from `APP_TITLE/APP_SUB`). Reading them as a set:

```
dev.sh      launch the browser from the checkout (extensions unpacked,
            foreground; edits to content scripts hot-reload)
build.sh    "build" == make sure a rebranded base bundle exists + is current
            (fetch the snapshot if missing, then rebrand) – no compile step
clean.sh    remove the /Applications wrapper + staged dist/, keep base + profile
rebuild.sh  clean + build
bust.sh     re-seal + re-register the bundle with LaunchServices (Chapter 5)
nuke.sh     wipe base + wrapper, re-fetch from scratch, rebrand, reinstall
            (preserves the profile)
test.sh     a JS syntax gate over the front-end + each extension's own tests
```

The design point is that none of these compile the browser — the base is a prebuilt snapshot — so “build” and “rebuild” are really “make sure a current, rebranded base exists,” and the destructive ones are careful about what they keep: `clean` and `nuke` never touch your profile, and `clean` does not even touch the 325 MB base (that is `nuke`’s job). The npm `scripts` in `package.json` alias these (`pnpm dev`, `pnpm build`, `pnpm nuke`, and so on) for anyone who reaches for the package manager.

Testing

`scripts/test.sh` is a JS syntax gate over the repo’s own front-end (the new-tab page, the theme, the root `v*.mjs` scripts) plus a delegation to each bundled extension’s own test script. There is no Rust or compile step at the browser level, because the base is a prebuilt snapshot; the tests that matter for behavior are the CDP verification probes of Chapter 20, which drive a running browser. The syntax gate catches the cheap failures (a broken content script would silently not load in Chromium) before they reach a launch.

CI

The GitHub Actions `ci.yml` runs three checks on every push and PR, on a plain Ubuntu runner, with the `zpwchrome` submodule deliberately *not* checked out (nothing CI checks needs it):

- `shellcheck` over `bin/zwire`, `scripts/*.sh`, and `fork/*.sh` at warning level — every shell script the project ships is linted.

- **JSON validation** over every tracked `*.json` outside the `zpwchrome` submodule, discovered with `git ls-files` (never hardcoded), each parsed with `python3 -c json.load`.
- **patch-series consistency** via `scripts/check-patch-series.sh`, which asserts the `fork/patches/series` list and the patch files on disk agree.

A second workflow, `pages.yml`, publishes `docs/` to GitHub Pages — the site the README's “docs · online” badge points at — on any push that touches `docs/`, with a single-deployment concurrency guard so an in-flight publish is never cancelled. The CI is intentionally lightweight: lint, validate, check consistency. It does not build the browser (there is nothing to build) or compile the fork (that needs the 100 GB checkout); those are local, deliberate operations.

Inside the script family

The dev-loop scripts share one banner library (`cyberpunk.sh`, sourced for the ANSI banner keyed off `APP_TITLE`) and each is careful about what it keeps. `dev.sh` runs the browser from the checkout in the foreground — its comment notes that edits to the content scripts hot-reload while a `background.js` change needs the reload glyph on `chrome://extensions`. `build.sh` states outright that “‘build’ == ‘make sure a rebranded base bundle exists and is current’”: it fetches the snapshot if missing and runs the rebrand on Darwin only, with no compile step because the base is prebuilt. `nuke.sh` is the heavy reset — it removes the `/Applications` wrapper and the 325 MB base, re-fetches, rebrands, and reinstalls, and its comment is emphatic that “your profile ... is preserved.” `bust.sh` is the macOS-only icon-cache buster and exits early on any other platform.

`test.sh` is a JS syntax gate, not a behavior suite, and its comment says why: “there is no Rust/compile step at the browser level — the base is a prebuilt snapshot.” It runs `node --check` over the repo's own front-end (the new-tab page, the theme, the root scripts), deliberately excluding `node_modules` and the vendored `lib/zgui-core`, and then delegates to each bundled extension's own `pnpm test`. A broken content script would fail silently in Chromium rather than throw, so catching the parse errors cheaply, before a launch, is the point; the behavioral proofs are the CDP probes of Chapter 20, which drive a running browser rather than lint a file.

Chapter 23 — Packaging and Self-Contained Installs

`scripts/install.sh` is the developer install — a PATH symlink to the checkout. But `zwire` also ships three *self-contained* installers that bundle everything the browser needs into one location, so you can delete the repo and the base and the app still runs. This chapter is those three, and the one substitution they all make.

The developer install vs the self-contained install

`scripts/install.sh` (Chapter 5) is the light path: it fetches the base, symlinks `bin/zwire` into `~/.local/bin`, and rebrands on macOS. The launcher it links still runs *from the checkout* — delete the repo and `zwire` breaks. That is right for development, where the checkout is where you are editing.

The self-contained installers are the opposite: they copy everything into one owned location so the checkout becomes disposable. On macOS, `localinstall.sh` builds `/Applications/zwire.app` containing the Chromium base (`Contents/Resources/browser/`), the three extensions (`Contents/Resources/ext/`), the native host (`Contents/Resources/native/`), and a bundle-relative launcher (`Contents/MacOS/zwire`). Delete the repo and `~/.zwire/base` afterward and the app still runs, with no system dependencies. Only the per-user profile (`~/.zwire/profile`) and the per-user extension copy (`~/.zwire/ext`, staged from the bundle at launch) live outside it.

The Linux and Windows installers mirror this shape:

- `localinstall-linux.sh` assembles everything under `~/.local/opt/zwire/` (browser + extensions + host), drops a `zwire` launcher on PATH and a `.desktop` entry for the app menu and icon, and needs no root — a pure user install. Delete the repo afterward and it still runs.
- `localinstall-windows.ps1` assembles everything under `%LOCALAPPDATA%\zwire` (the `Win_x64` snapshot, the extensions, `zwire-host.exe`), and wires the native host through the *registry* — because Windows, unlike macOS and Linux, does not read native-messaging host manifests from the profile directory. It drops a `zwire.cmd` launcher.

The native host in the packaged install

The one component that cannot ship as source is the native host, because it is a compiled binary. `scripts/setup-native-host.sh` builds `zwire-host` with `cargo build --release` (its own submodule, `extensions/hud-internal/native/zwire-host`) and writes the native-messaging manifest pointing at it; the launcher calls that on first run. Because the binary is a single static Rust artifact with no runtime dependencies, it drops straight into the app bundle — there is nothing for the deployed machine to provide, no interpreter and no `psutil` to be missing. On the scheme-bridge / sysinfo / PTY path `zwire`'s browser actually drives, the wire format is exactly Chapter 15's: the 4-byte-length framing, the eight scheme names, the atomic tmp-and-rename writes to the HUD scheme/ui files, the base64 `output/exit` PTY frames. On Windows the manifest becomes a `.exe` path and lives in the registry; on macOS and Linux it is the profile-relative JSON of Chapter 15.

What the host really is: a full local agent

zwire's browser uses only a small corner of `zwire-host`. The scheme bridge, the sysinfo stream, and the PTY relay are three of roughly fifty commands across fifteen modules, reachable over two transports — the host is a local-machine agent that the HUD happens to be one client of.

The extra surface, module by module: a namespaced key-value store (`kv_get/kv_set/kv_merge/kv_del/kv_keys`, each an atomic JSON file under `~/.<app>/kv/`); a filesystem API (`fs_read/fs_write/fs_append/fs_list/fs_walk/fs_stat/fs_mkdir/fs_rm`, with a 64-MiB read cap and a 50k-entry bounded recursive walk); file observers (`fs_watch` and a rotation-aware `fs_tail`); an arbitrary command runner (`exec`, returning base64 stdout/stderr and an exit code); background jobs (`job_start/job_poll/job_list/job_result`, which fire a desktop notification on completion); process tools (`ps/kill/which`); a process- global publish/subscribe bus (`sub/unsub/pub`); OS integration (`open/clipboard_get/clipboard_set/notify/hostinfo`); and a TCP peer mesh (`auth/peer_hello/peer_connect/peers/remotes`) that federates the bus and runs a request on another host. On the shared capabilities it is also richer: PTY sessions are multiplexed by id (one connection can drive many terminals) and take a shell, args, cwd, and env; the sysinfo interval is configurable and there are one-shot (`sysinfo_once`) and explicit `sysinfo_stop` variants, so a client tunes the cadence or halts the stream without closing the port. A `hello` reply advertises exactly which of these capabilities were compiled in, so a client can feature-test rather than guess.

The second transport is the reason the Rust host exists as its own project rather than a script. Besides Chrome native-messaging over stdio, it runs a line-delimited JSON daemon over a Unix-domain socket (a named pipe on Windows), created owner-only under an owner-only directory precisely because it exposes `exec`, the filesystem, and PTYs — a local agent any app on the machine can call, not only the browser. That is the whole point of the README's framing: one self-contained binary means the consuming bundle has “zero runtime dependencies,” and an embedder that only wants the light capabilities can compile it without the system-stats and PTY features entirely. “No runtime dependencies” is about the deployed artifact needing no Python and no `psutil`, not about the crate graph — it builds on `serde_json`, `base64`, and the optional `sysinfo` and `portable-pty`.

Two sysinfo details worth pinning

Two fields of the sysinfo frame carry more care than the rest, because a naive implementation would drop or mangle them. First, the disk-I/O rate (`io: {r, w}`, read and write bytes per second): the host keeps a `Disks` handle alive across ticks and reads `Disk::usage()` — bytes since the last refresh — summed over every disk and divided by the elapsed interval, so the streaming rate is real rather than a lifetime total. (The one-shot `sysinfo_once` has only a single instant to work with, so it double-refreshes to report a near-zero delta rather than a lifetime total mislabelled as a rate — an honest “no rate from one sample.”) Second, battery, which has no single cross-platform source: the host reads it natively on all three platforms — `pmset` on macOS, `/sys/class/power_supply` on Linux (capacity plus a charging/`Mains`-online check), and `GetSystemPowerStatus` on Windows — so the statusbar renders identically everywhere. The only remaining absence is honest and physical: machines with no battery — desktops and VMs — omit the segment, exactly as they should.

The whole packaging story is one idea: make the app own everything it needs so the checkout is disposable, and the native host is part of that — a single self-contained Rust binary bundled into the app with nothing for the deployed machine to provide. What the user gets is a normal app — an `.app`, a `~/.local` install, a `%LOCALAPPDATA%` install — that happens to be Chromium wearing the HUD, with that host doing the scheme, stats, and terminal work.

What the self-contained installer actually assembles

The macOS `localinstall.sh` copies four things into `/Applications/zwire.app`: the whole Chromium base bundle into `Contents/Resources/browser/`, the three extensions into `Contents/Resources/ext/` (via `rsync` excluding `node_modules`, `.git`, `tests`, and `target`), the freshly built `zwire-host` into `Contents/Resources/native/`, and a bundle-relative launcher at `Contents/MacOS/zwire`. That launcher is a quoted heredoc, so every `$var` is resolved at *runtime*, not install time; it defaults its state to `~/Library/Application Support/com.menketotechnologies.zwire`, resolves the browser executable from the bundle's own `CFBundleExecutable` via `PlistBuddy`, and passes the same nine flags `bin/zwire` does. The Info.plist it writes sets the bundle id to `com.menketotechnologies.zwire`, and the whole app is ad-hoc signed and LaunchServices-registered at the end.

The one subtlety that repays reading is the per-user extension staging, and its comment spells out the failure it avoids: Chromium writes each unpacked extension's compiled declarativeNetRequest ruleset and content hashes into `<ext>/_metadata/`, so the browser needs *write* access to the extension directory, but a shared `/Applications` bundle has a single owner — so a second user would fail to load the extensions with an “Internal error while parsing rules.” The launcher therefore `rsyncs` the bundled extensions into a per-user `$ZWIRE_STATE/ext` (excluding `_metadata`) at every launch and loads from there. The Linux installer mirrors the shape into `~/local/opt/zwire` with a `.desktop` entry, and its launcher adds a tenth flag the macOS one omits — `--no-sandbox`. The Windows installer assembles under `%LOCALAPPDATA%\zwire` and, because Windows does not read host manifests from the profile directory, wires the native host through the registry under both the Chromium and Chrome hives so it resolves regardless of the snapshot's product name.

Cutting a macOS release: `release-macos.sh`

Past macOS releases were built and uploaded by hand, so each cut risked drifting from the last; `scripts/release-macos.sh` captures the recipe as a repeatable script that turns a `v<version>` tag into published artifacts. It reuses `localinstall.sh`'s self-contained bundle assembly — the *same* `/Applications/zwire.app` of the section above — but redirects it to a writable staging dir through a `$ZWIRE_DEST` override (default `/Applications`), so the release build needs no `sudo`: `localinstall.sh` learned to honor `$ZWIRE_DEST` precisely so CI and this script can assemble the self-contained app into a directory they own. It then packages the staged app two ways and uploads both to the GitHub release for the tag: a `zwire-<v>.pkg` (a `pkgbuild` component, identifier `com.menketotechnologies.zwire`, install-location `/Applications`, forced non-relocatable to match prior pkgs) and a `zwire-<v>-macos.zip` (a `ditto --keepParent` archive with `zwire.app` at its root). The guards are what make it safe to run repeatedly: it refuses to overwrite an existing release, requires the `v<version>` tag (pushing it to origin if missing), and warns on a dirty tree; `--dry-run` builds the artifacts without publishing and `--notes-file` overrides the auto-generated changelog (commit subjects since the previous `v*` tag). Signing is unchanged — the bundle stays ad-hoc signed with no notarization, exactly as every prior release, so first launch still needs a right-click → Open. Artifacts land in `dist/` (gitignored), and the whole thing is wired as the `release-macos package.json` script.

Chapter 24 – Cross-Platform, Updating, and Operations

The operational chapter: what works on macOS (aarch64/x64) versus Linux (x86_64) versus Windows, how to roll the base forward, and the day-to-day notes — native messaging setup, the cosmetic developer banner, and re-running the rebrand after an upgrade.

Platform matrix

The `zwire` launcher itself is portable — it is POSIX shell that reads a base path, builds an extension list, ensures a native host, and `execs` Chromium — and it runs on macOS (aarch64 and x64) and Linux (x86_64), with the PowerShell installer covering Windows. What is shared and what differs, at a glance:

	macOS	Linux	Windows
snapshot platform	Mac_Arm / Mac	Linux_x64	Win_x64
state dir default	~/Library/App...	~/.config/zwire	%APPDATA%\zwire
identity rebrand	bundle patch	launcher + .desktop	launcher name
native host wiring	profile dir	profile dir	registry (both hives)
extra launch flag	–	--no-sandbox	–
self-contained to	/Applications	~/.local/opt	%LOCALAPPDATA%

The parts that differ per platform are three:

- **The base download.** `fetch-base.sh` maps the host to the right snapshot: `Mac_Arm` and `Mac` for the two macOS architectures, `Linux_x64` for Linux, and `Win_x64` for Git-Bash/MSYS on Windows. The launcher and the native host are cross-platform; the base is per-platform because Chromium's snapshots are.
- **The rebrand.** As Chapter 5 established, the in-place Dock/icon rebrand is macOS-only, because only macOS carries app identity as bundle metadata. On Linux the launcher name is the brand, and a `.desktop` entry gives it a menu icon; on Windows the installer places a named launcher. `install.sh` only calls `rebrand-macos.sh` under `Darwin`.
- **The native host install.** On macOS and Linux the HUD host is installed *profile-relative* into `<user-data-dir>/NativeMessagingHosts/`, which Chromium reads on every platform, so there is no per-OS `~/Library` vs `~/.config` split — `setup-native-host.sh` writes the same manifest in the same place. On Windows the manifest is wired through the registry instead, because Windows Chromium does not read host manifests from the profile dir. `bin/zwire` invokes the setup on launch when the manifest is missing or dangling, which is what makes a bare `zwire` self-sufficient in dev on macOS and Linux.

The host binary is `zwire-host`, a cross-platform Rust program built with `cargo build --release` on first need (or when `FORCE=1`) — the sole native host, detailed in Chapter 15. On Windows/WSL the manifest points at a `.exe` because the browser wants a Windows path even though the host source is portable Rust.

Updating the base and re-applying the rebrand

Rolling the base forward is two commands on Linux and three on macOS:

```

scripts/fetch-base.sh           # latest Chromium snapshot (or a pinned revision)
scripts/fetch-base.sh 1656770  # pin an exact revision instead
scripts/rebrand-macos.sh       # macOS only: re-apply the rebrand after the swap

```

The rebrand must be re-run after every base swap for the mechanical reason Chapter 5 gave: `fetch-base.sh` unzips a fresh, stock, Chromium-signed bundle over the patched one, so the name, the icon, the deleted `CFBundleIconName`, and the ad-hoc re-sign are all gone and must be reapplied. `install.sh` chains the two on macOS so a first install is one command; an in-place upgrade is where you run them separately and must not forget the rebrand, or the Dock reverts to “Chromium.” If the Dock icon lingers even after the rebrand, `scripts/bust.sh` forces the LaunchServices cache to refresh (Chapter 5).

The runbook notes

Two operational notes carry over from earlier chapters and belong in the runbook:

- **Native messaging setup.** `zwire`’s own HUD host (`com.zwire.hud`) is wired automatically on launch. `zpwrcrome`’s native host — needed for its `pass` integration and segmented downloads — is a separate install documented by `zpwrcrome` itself; `zwire` does not install it for you.
- **The developer-mode banner.** Loading unpacked extensions via `--load-extension` raises Chromium’s developer-extensions notice on startup. It is cosmetic; the extensions run fully. It is expected, not a fault.

That is the whole operational surface: fetch (and pin) the base, rebrand on macOS, let the launcher wire the native host, and expect one cosmetic banner. Everything else — the profile, the extension staging, the scheme file, the terminal PTY — the launcher and the host manage on their own, which is the point of a browser that is mostly a shell script wrapped around a stock engine.

First run, migration, and the degradation ladder

A first launch on a clean machine does more than open a window, and knowing the order helps when something is missing. The launcher resolves its state directory (the OS app-data location unless `$ZWIRE_STATE` says otherwise), performs the one-time migration of any leftover `~/zwire` into it, and then, if no base is recorded, exits with the exact `run: .../scripts/fetch-base.sh` message — so a fresh clone’s first `zwire` tells you the single missing step rather than failing opaquely. Once the base is present, the launcher ensures the native host by building the Rust binary if its manifest is missing or its target is gone, and only then execs the browser.

The value of that sequence is a *degradation ladder* rather than a set of hard requirements. No base is the only fatal condition. A missing native host is not fatal — the browser launches with the terminal and system stats disabled and one stderr line. A missing submodule (`zpwrcrome`, `zgui-core`) is not fatal — its extension directory has no `manifest.json`, so the launcher skips it and loads what remains. A stale LaunchServices icon is a cosmetic annoyance with a dedicated fix (`bust.sh`), not a functional fault. A base upgrade on macOS silently reverts the rebrand, which is why the update recipe ends with `rebrand-macos.sh` — the one manual step the launcher cannot infer. Read as a whole, the operational model is that the browser tries to run with whatever is present and tells you, tersely and on stderr, only about the one gap that actually stops it: no base. Everything above the base is best-effort, and the launcher’s job is to assemble as much of the HUD as the machine currently has.

Chapter 25 – The Trust Boundaries and the Threat Model

Every capability the earlier chapters celebrated is also a boundary widened. `zwire` loads unpacked extensions with elevated flags, runs a native host that hands out `exec`, `fs`, and `pty`, injects content scripts into privileged `chrome://` pages, and — on the fork — bypasses `X-Frame-Options` and frame-ancestors CSP so the tmux overlay can pane any site. Each of those is a hole in a wall Chromium built on purpose, and a browser that opens holes without accounting for them is a liability, not a tool. This chapter is the accounting: who can do what inside a running `zwire`, where the widenings actually land, and why the ones that look alarming are contained by the ones that do not.

The profile is the blast radius

The first and most important boundary is the one Chapter 1 opened with: `zwire` runs against `~/.zwire/profile`, a dedicated `--user-data-dir`, and never the profile that holds your real logins. This is the containment on which everything else rests. The elevated flags, the unpacked extensions, the developer-mode banner, the native host wired into `NativeMessagingHosts/` — all of it is scoped to a profile that started empty and holds only what you did *inside* `zwire`. If every extension in the load set were malicious, the worst it could reach through the browser is the cookies, history, and saved passwords of the `zwire` profile, which is to say the ones you chose to create there. Your system Chrome, its logins, and its saved payment methods live in a different `--user-data-dir` the `zwire` process never names and cannot see. The isolation is not a courtesy to Chrome; it is the design's primary security property, and it is why loading unpacked, elevated-flag extensions is a defensible thing to do at all. You would never point those flags at the profile with your bank in it — so `zwire` never does.

The launch flags and what they actually relax

Three flags in `bin/zwire` change Chromium's posture, and it is worth being exact about what each one loosens, because the reputational weight of the flag names exceeds what they do. `--load-extension` loads the unpacked extension directories; its real cost is that unpacked extensions are not Web Store-reviewed, so the trust you extend is trust in *this author's* code, staged from a copy you can read on disk. `--extensions-on-chrome-urls` is the one that lets the internal-HUD content scripts run on `chrome://` pages at all — the single relaxation that makes the skinned internal pages possible — and it is narrow: it widens *where a content script may run*, not *what the script may then do*, which stays governed by the manifest's declared permissions. `--test-type` suppresses the automation-restriction warnings Chromium raises when it detects it is being driven, which is cosmetic in effect but carries a real caveat: it also relaxes a handful of automation guardrails, and it is the reason the developer-mode banner is merely annoying rather than blocking. On Linux a fourth flag, `--no-sandbox`, appears where the macOS launcher omits it — a genuine reduction in the renderer sandbox, forced by the way the snapshot ships on that platform, and the one flag in the set the book does not wave away: it is the place where `zwire` on Linux is measurably less defended than `zwire` on macOS, and a reader who runs untrusted pages should know it.

The native host is the largest boundary, and its gates

The native host is the widest hole zwire opens, by a wide margin: `exec` runs arbitrary programs, `fs_*` reads and writes the filesystem, and `pty_spawn` forks a login shell. Anything that can send that host a command owns the machine to the extent your user account does. So the gates around *who can send it a command* are the load-bearing security in the whole system, and they are worth stating plainly. On the native-messaging path, Chromium itself is the gate: the host's manifest lists `allowed_origins`, and only the extension origins named there can `connectNative` to it — a page you visit cannot, because a page is not an extension origin. On the daemon path, where the same router is reachable over a socket, the gate is the filesystem: the Unix socket is created `0600` inside a `0700` directory, so only your own user account can open it, and a different user on the same machine simply cannot see it. The one path that crosses a machine boundary — TCP peering — is the one path that demands a shared `--token` before anything privileged runs, because it is the only path where the “same user, same machine” assumption the local gates rely on no longer holds. The design is coherent: local callers are trusted because the OS already authenticated them by letting them open an owner-only socket, and remote callers are trusted only after presenting a secret. The place to be careful is exactly the place the host is careful — the moment a capability leaves the machine.

What the fork widens, and what it deliberately does not

The fork's patches include two that a security reader should look at directly, because both loosen a Chromium invariant. The first is the `is_sandboxed` gate in `extension_function_dispatcher.cc` (patch 0018), which fixes a real SIGTRAP by letting a content script's extension function run in a context Chromium's assertion would otherwise reject. The important detail is the one the chapter on the fork already drew: the patch widens *scripting*, not *privilege*. The store-privileged JavaScript bindings — the APIs Chromium reserves for Web Store extensions — stay gated by `IsWebstoreOrigin` in `permissions_data.cc`, untouched by the patch. zwire's extensions gain the ability to run where the assertion used to fire; they do not gain the ability to call APIs they were never granted. The second is the CSP and `X-Frame-Options` bypass that lets the tmux overlay embed a site whose headers forbid framing. This is a genuine relaxation of a defense-in-depth mechanism, and it is scoped as tightly as the goal allows: it yields for *sub-frame navigation* so a pane can load an otherwise-unframeable site, not for the top-level page, and the sandboxed pane iframe (Chapter 10's exact token set) is the compensating control — the embedded site runs, but in a frame whose sandbox attributes the overlay chose, not with the ambient authority of the host page. The pattern in both patches is the same, and it is the pattern to hold the fork to: widen the narrowest thing that unblocks the feature, and leave the adjacent invariant standing.

The honest residual risk

No account of a threat model is complete if it only lists the mitigations, so here is what remains. You are running unpacked extensions you did not review line by line; your trust is in the author, backed by the fact that the code is readable on disk and staged into a copy you can inspect. You are running a native host that can own your account; your protection is that only your own extensions and your own user can reach it. On Linux you are running with the renderer sandbox off; your protection is the dedicated profile and the fact that you, not an adversary, chose the pages. And you are, on the fork, running with two Chromium invariants relaxed; your protection is that each relaxation is scoped to the exact surface the feature needs. None of these is zero risk, and the book does not pretend otherwise. The design's claim is narrower and defensible: every widening is contained by the profile boundary, and the profile boundary is the one wall zwire never touches. Keep your real life in your real browser, keep zwire's power scoped to zwire's profile, and the residual risk is the risk you can actually reason about.

Chapter 26 – The Performance Budget

A browser that injects a suite of content scripts on every page, streams system stats continuously, holds live PTYs, and repaints every `chrome://` page is spending your machine's time whether you asked it to or not. If that spending is not accounted for, the HUD becomes a tax on every navigation and every idle minute, and the neon is not worth a browser that stutters. This chapter is the budget: where the cycles and bytes go, what keeps the always-on machinery cheap, and where the cost could bite if the disciplines slipped.

The per-page injection cost

The largest recurring expense is the one that runs on literally every page load: the internal-HUD content-script suite injects vim keys, the tmux overlay, the powerline statusbar, the palette, and the fuzzy find into each document. The suite is kept cheap by three habits. First, the scripts run in a fixed order the manifest declares, so shared state (the keybinding registry, the scheme) is resolved once and read by the rest rather than each script recomputing it. Second, every injectable is idempotent and guards its own re-entry — a content script that has already installed its listeners on a document detects that and returns, which matters because same-document navigations and the overlay's own re-injection would otherwise stack duplicate handlers. Third, the heavy surfaces are lazy: the terminal's xterm bundle, the palette's list machinery, and the fuzzy matcher's index are not built at injection time but on first use, so a page you open and immediately leave paid only for the key listeners, not for the UI behind them. The statusbar is the one visible always-on surface, and it reads its data from a storage key the worker fills, so painting it is a read and a layout, not a computation.

The always-on streams

Two things run when no page asked them to, and both are deliberately throttled. The system-stats stream samples the machine on its own background thread in the native host, at an interval floored at 250 ms — fast enough that the statusbar's CPU and memory readouts feel live, slow enough that the sampling itself never shows up in the number it reports. Running it on a dedicated thread is what keeps a held stream from blocking the connection's other RPCs, so a HUD can carry a stats stream, a PTY, and a file watch on one pipe without any of them starving the others. The rates in the sample — network and disk throughput — are computed as deltas over the elapsed wall-clock interval, which means the host does no work to *maintain* a counter between ticks; it subtracts two readings when a tick fires and is otherwise idle. The second always-on cost is the CI poll, and it is deliberately slow: a `chrome.alarms` heartbeat every ten minutes, tuned to stay under GitHub's anonymous request cap. Ten minutes is not a monitoring interval anyone would choose for urgency; it is the interval chosen so the feature is free — a handful of requests an hour against a public API, invisible in the budget.

The native host's footprint

The bridge that does the most is the smallest thing in the system. `zwire-host` is a single static Rust binary around half a megabyte, built with `opt-level="z"` for size and `panic = "abort"` to drop the unwinding machinery, with no Python interpreter, no `psutil`, and no runtime dependencies to load. This is not incidental thrift; it is what makes the host affordable to start on demand. Chromium spawns it per connection on the native-messaging path, so its startup time is paid every time the browser opens a port to it, and a half-megabyte static binary starts in the noise. The alternative the book explicitly rejected — a Python reference host — would

have paid an interpreter startup and a dependency import on every spawn, and would have failed outright on a machine without the right Python and packages. The Rust host's budget is: near-zero to ship (it is packaged inside the bundle), near-zero to start, and, while running, the cost of whatever it was asked to do — a sysinfo sample, a PTY read loop, a filesystem crawl — and nothing while idle.

The base and the staging

The one genuinely large number is the base itself: the extracted Chromium snapshot, or the packaged fork, is around 325 MB on disk. That is the price of a real Blink engine and it is non-negotiable — it is the same disk a Chrome install spends, and it is exactly what a WebView wrapper would have saved by giving up the capabilities Chapter 2 said zwire cannot give up. The one place zwire adds to it is the per-user extension staging: at launch the loaded extensions are copied into a writable `~/.zwire/ext/` tree rather than run in place, because Chromium writes `_metadata/` indexes into the extension directory and every user needs a writable copy. That copy is small — the extensions are JavaScript and CSS, not the engine — and it is paid once per launch, fast enough to be invisible against the browser's own startup. The honest summary of the disk budget is that zwire costs one Chromium plus a rounding error, and the rounding error is the price of never running an extension in place.

Where it could bite, and the mitigations

The failure modes are the mirror image of the disciplines. If a content script dropped its idempotence guard, the overlay's re-injection would stack handlers and a long-lived tab would slow with every same-document navigation — which is precisely why the guards exist and why the CDP probes assert single-injection. If the stats interval floor were removed, a HUD asking for a tighter stream could spin the host's sample thread hot enough to perturb its own CPU reading — which is why the floor is in the host, not left to the caller. If the terminal built its xterm bundle at injection instead of first use, every page load would pay for a terminal most pages never open — which is why it is lazy. The pattern across all of them is that the budget is enforced at the cheapest layer that can see the whole picture: the host floors the stream, the manifest orders the injection, the guards live in the scripts themselves. A HUD is only worth running if it is nearly free when you are not looking at it, and the whole point of the accounting is that, when you are reading a page and not touching the HUD, almost nothing in this chapter is spending anything at all.

Chapter 27 — Extending the HUD

zwire is built to be extended by its author, and the seams it exposes to him are the same seams open to you. This chapter is the practical one: how to add a ninth scheme, a new HUD internal page, a new content script, a palette command, and a native-host capability — each as a walk down the exact files the earlier chapters mapped, so that extending zwire is a matter of adding a row to a table the system already reads, rather than inventing a new mechanism.

Adding a ninth scheme

A scheme is a set of role colors under a name, and adding one means writing that name into every table that enumerates the eight. There are more of those than there look to be, and forgetting one is the classic way a new scheme half-works. The color table lives in `schemes.js`, and it lives there *twice* — once for the new tab and once for the internal HUD — because the two run in different worlds and each carries its own copy; both must gain the new entry, dark and light. The native host validates incoming scheme names against its list of eight allowed names before it will write the scheme file, so a ninth name the host does not know will be rejected at the bridge and never persist — the allowed-names list in the host is the third place to edit. If you want the scheme reachable from the palette, it wants a row in the palette's scheme commands; if you run the fork, the DevTools Theme dropdown that Chapter 21 wired enumerates the same names and wants the ninth too. The rule to internalize is that the eight is not a constant in one place; it is a consensus across the content scripts, the new tab, the host's validator, and the fork's native mixer, and a new scheme exists only once all of them agree it does.

Adding a HUD internal page

The internal pages are the most self-similar surface to extend, because they are all built from the same shell. A new page is four moves. First, add the page under `pages/` and boot it with `zg-boot.js`, the shared shell that gives every internal page the scheme, the fonts, the fuzzy-filter behavior, and the HUD chrome for free — you write the page's content, not its frame. If the page is a list of things, build it on `zg-list.js`, the list factory the history, bookmarks, and downloads pages already share, and you inherit the fuzzy filter and the row rendering rather than re-implementing them. Second, if the page is meant to *replace* a `chrome://` page, add its redirect to `redirect.js`, which is what turns a navigation to the native page into a load of your HUD replacement. Third, if the page draws on a privileged Chrome API — the settings page's `settingsPrivate`, the extensions page's `developerPrivate` — declare that permission in the manifest, because the redirect gives you the surface but not the authority. Fourth, if the page should be reachable by keyword, seed a palette command for it. The whole design of the internal pages is that they are the same page with different content, so the cost of a new one is the content and the wiring, never the chrome.

Adding a content script

A new always-on behavior — the class vim, the overlay, and the statusbar belong to — is a content script, and the two things that make it a good citizen are the two the suite already enforces. First, place it in the manifest's injection order relative to the scripts it depends on: if it reads the keybinding registry or the scheme, it runs after the scripts that establish them, because the order is the contract. Second, make it idempotent — guard the moment it installs its listeners so that a second injection into the same document detects the first and returns

— because the overlay’s re-injection across same-document navigations *will* run it twice and a script that stacks its handlers is a slow leak. If the behavior needs a key, register it in the `zkeys.js` keybinding registry rather than attaching a raw listener, so it appears in the keybinding editor, respects the user’s remaps in `zb_keys`, and does not collide silently with an existing binding. A content script that reads the registry, honors the injection order, and guards its own re-entry is indistinguishable from the ones that shipped — which is the point of having a suite rather than a pile.

Adding a palette command

The command palette is a registry, not a switch statement, so a new command is a new entry, not a new branch. Seed it in `cmd-defaults.js`, which is where the built-in keyword commands live, and give it a command *type* the palette already knows how to execute — a URL to open, a `shell` one-liner to run in the terminal, an internal page to navigate to. The type is what matters: because the palette dispatches on type, a new command that reuses an existing type needs no new execution code at all, and a command that genuinely needs a new behavior adds one type handler that every future command of that kind reuses. Commands that must reach across into the service worker — anything touching tabs or windows — go through the `zb_cmd` storage bus rather than calling into the worker directly, because a content script cannot call the worker’s privileged APIs and the bus is the seam that wakes the worker to do the privileged work. User-defined commands land in `zb_custom_cmds` through the command editor; the seeded ones in `cmd-defaults.js` are the same shape, which is why the editor can show and edit both.

Adding a host capability

The native host is the most disciplined thing to extend, because its whole architecture is one router in front of many small modules. A new capability is a new module under `src/` and a new `cmd` the router in `session.rs` dispatches to — and because all three transports sit behind that one router, the capability is immediately reachable over native messaging, over the local socket, and from the in-process library, without touching any of them. The module’s shape is set by the fifteen that precede it: it takes the parsed request, does its work, and returns a JSON reply, echoing the request’s `id` if it carries one so it can be multiplexed against other in-flight work. If the capability streams — like `sysinfo_start` or `fs_watch` — it runs on its own thread and emits keyed frames the caller cancels by `id`, following the pattern the stats stream and the file watcher already set. And if the capability is privileged in a way the socket’s owner-only gate does not cover, it inherits the token requirement on the TCP path for free, because that gate lives in the transport, not in the module. The router is the extension point; the modules are leaves; and a new leaf is a new file and a new line in the dispatch, which is exactly as much ceremony as a capability of the machine deserves.

Chapter 28 – The Lineage: The Cyber-punk Idiom and the zpwr Fleet

zwire did not arrive from nowhere, and it is not a browser that happened to be painted with neon. It is the browser-shaped member of a larger family of tools that share an idiom — a keyboard grammar borrowed from tmux and vim, a powerline aesthetic, a neon palette — and, more concretely, share code. Understanding the lineage explains things the feature chapters can only assert: why the terminal is “extracted verbatim from Audio-Haxor,” why the fuzzy matcher is the same one in six other apps, and why a browser was, in a sense, the family’s missing member.

The idiom: tmux, vim, and powerline as a grammar

The HUD’s controls are not invented from scratch; they are a dialect of a grammar the author already spoke everywhere else. The tmux overlay uses tmux’s prefix and tmux’s split, navigate, and zoom verbs, because a person who lives in tmux should not have to learn a second tiling vocabulary to tile browser panes. Vim on every page uses vim’s motions for the same reason: the cost of the feature is near zero for someone who already has the motions in their fingers, and the whole value of putting them in the browser is that the browser stops being the one place the grammar does not work. The powerline statusbar is the same aesthetic that terminal prompts and editor status lines in this world already wear — the segmented, arrow-jointed bar that encodes state as colored runs. None of these is a novelty; each is a *continuation*, and that is the point. The HUD is legible on first sight to anyone who came from tmux, vim, and a powerline prompt, because it is those three grammars wearing a browser instead of a terminal.

The fleet: shared submodules across the house

The lineage is not only aesthetic; it is literal shared code, tracked as git submodules. `zgui-core` is the shared toolkit — the fuzzy matcher, the palette, the visual effects, the fonts — consumed by Audio-Haxor, `zmax`, `zpwr-daw`, `traderview`, `ztranslator`, `zpdf`, `zemail`, `zoffice`, and more, and by `zwire`’s internal HUD among them. `zpwr-i18n` is a shared translation catalog. The native `zwire-host` is itself a submodule with its own CI and README, written to be embedded by sibling hosts. Because these are submodules rather than copied snapshots, a fix to the fuzzy matcher or a new palette behavior propagates to every consumer through the same pointer bump, which is why the book can describe a feature as behaving “the way it does in the other apps” and mean it exactly: it is the same code. The fleet is not a set of apps that resemble each other; it is a set of apps built from a shared kit, and `zwire` draws on that kit the way the others do.

Verbatim extraction as a design value

The most distinctive habit in the lineage is the refusal to re-implement. The terminal front end is not `zwire`-original — `terminal.js` is the shared `zpwr-embed-terminal` component, extracted verbatim from Audio-Haxor, with its header recording that the only `zwire`-specific change is a transport shim. The palette and the find bar are not `zwire`-original UI. The fuzzy matcher is the shared one. This is a value, not an accident: when a component is *extracted verbatim* rather than forked and edited, every app that carries it benefits from every fix, and no app accumulates a private divergence that has to be maintained alone. The transport shim pattern is

how this survives real differences between hosts — the terminal component auto-detects whether it is talking to a Tauri backend, a JUCE backend, or, in *zwire*, a native messaging port through a service worker, and the same component runs over all of them because the *difference* is isolated to a shim while the *component* stays shared. The discipline is the same one the native host follows with its library crate: write the capability once, embed it many times, and let the seam absorb the platform.

Why a browser was the missing member

Seen against the fleet, *zwire* fills an obvious gap. The family had an audio workstation, an editor, a translator, a PDF tool, a mail client, an office suite — tools that each put the *tmux/vim/powerline* grammar on a specific task. The one application everyone uses all day and nobody in the family had reshaped was the browser, and the browser is the hardest one, because you cannot reshape it with a Tauri window: as Chapter 2 argued at length, the power the fleet's idiom wants — the extensions, the native chrome — exists only inside a real Blink engine. So the browser waited until the approach was clear: not a from-scratch engine and not a WebView shell, but a real Chromium owned at the seams a rebrand and a patch series can reach. *zwire* is the fleet's grammar finally applied to the application that resisted it longest, which is why it reads as continuous with the others despite being, mechanically, the least like them.

The neon palette as house style

The eight schemes are house style, not *zwire* style. The neon-on-near-black palette, the rationing of high chroma to edges and signal, the Share Tech Mono and Orbitron display faces — these are the fleet's look, and *zwire* wears them so that a person moving from the audio workstation to the browser to the editor sees one continuous world rather than three apps that happen to be dark. This is why the palette is a closed color *system* (Chapter 8) rather than a set of pretty colors: a shared look across many apps has to be defined as roles and derivations, so that a new app can adopt it correctly and an old one can gain a light mode without every surface being hand-tuned. The cyberpunk idiom is the family's signature, and *zwire*'s contribution to it is the hardest surface the family had ever had to paint — a browser's chrome, reached on the runtime path through content scripts and on the fork path through C++ — proving the house style survives all the way down to the native frame.

Chapter 29 – Positioning: zwire Among the Custom Browsers

“Custom browser” is a crowded and imprecise category, and zwire is easy to misfile in it. This chapter draws the map of the neighborhood — qutebrowser, Vivaldi, Arc, plain theme extensions, Electron wrappers, and the from-scratch engine — and places zwire precisely against each, because the value of the project is clearest when you can see exactly which trade every neighbor made that zwire refused, and which one it made that they did not.

The five other answers

There are five well-trodden ways to get a browser that is not stock Chrome, and each is a coherent answer to a different question. **qutebrowser** is keyboard-first and vim-modal like zwire’s page layer, but it is built on Qt WebEngine and its own Python application, so it is a different application wearing a Chromium *render* engine rather than a Chromium *browser* — the extension ecosystem is not there. **Vivaldi** is a heavily customized Chromium with tiling, sessions, and deep UI configuration, shipped as a closed product on top of the engine; it is the closest in *features* and the furthest in *model*, because it is a company’s fork you consume, not a rebrand you own. **Arc** reshaped the browser’s information architecture — spaces, a sidebar, a different notion of a tab — on a Chromium base, as a designed product. **Theme extensions** are the floor: they repaint the frame and nothing else, and Chapter 1 spent its length on exactly why that floor is where a rebrand that only themed would stop. **Electron and Tauri wrappers** are the other pole: they give you total control of a window’s contents at the cost of not being a browser at all — no address bar you did not build, no extension host, and, for Tauri, not even a Blink engine.

What zwire shares with each, and where it diverges

zwire shares qutebrowser’s conviction that the browser should be driven from the keyboard, and diverges by keeping the full Chrome extension host — zwire’s vim is a content script layered onto a real Chromium, not a modal application that replaced it, so **zpwrcrome** and its MV3 permissions still run. It shares Vivaldi’s and Arc’s ambition to reshape the whole browser, and diverges on ownership: Vivaldi and Arc are products you receive, while zwire is a rebrand and a patch series you apply to a snapshot Google builds, so nothing sits between you and the upstream engine except a shell script and, optionally, twenty-four readable patches. It shares the theme extension’s mechanism on the runtime path — it is an extension-and-flags rebrand — but pushes that mechanism far past where a theme stops, into content, behavior, and the internal pages. And it shares nothing important with the Electron wrapper, because the wrapper gave up the one thing zwire’s whole justification rests on: it is a real browser, with a real address bar, a real extension host, and a real Blink engine, not a single web view in a native window.

The runtime rebrand versus everyone who forks

The sharpest divergence is structural, and it is the two-tier design itself. Every neighbor that reshapes the native chrome — Vivaldi, Arc — does it by *maintaining a fork*, which means they own a divergence from upstream permanently and pay to carry it forward forever. zwire’s insight is that most of what people want from

a custom browser does not actually require the fork: colors, content, behavior, the new tab, the internal pages, the vim layer, the tiling, the terminal — all of it lives on the runtime path, on top of a stock snapshot that tracks upstream for free. The fork exists only for the residue that genuinely needs C++ — tab shape, the UI font, the native frame, the audio engine — and it is offered as an *option*, not imposed as the price of entry. No neighbor draws this line. The theme extension stops short of it; Vivaldi and Arc pay past it for everyone. *zwire* is the only one that put the line exactly where the capability boundary is and let you choose which side to stand on.

The cost *zwire* chose not to pay

Positioning is as much about refusals as features, and *zwire*'s defining refusal is the one Chapter 1 named: it does not write a browser. A from-scratch engine is a decade of work nobody finishes, and it buys independence from Google's engine at a price no small project can afford. *zwire* refuses that cost and inherits Blink's engine, security model, and update cadence instead — which is why it exists in kilobytes of shell and JavaScript rather than millions of lines of C++. It also refuses the Electron cost, which is the opposite trade: total window control in exchange for not being a browser. And on the daily-driver path it refuses the fork's cost — the recurring rebase tax of Chapter 21 — by making the fork optional. The through-line is that *zwire* is unusually clear-eyed about which costs buy something and which do not: it pays for a dedicated profile, an extension set, a native host, and (optionally) a patch series, and it declines to pay for an engine, a runtime, or a permanent fork it does not need.

Who *zwire* is for

The honest answer to “who is this for” falls out of the positioning. It is for someone who already lives in *tmux*, *vim*, and a powerline prompt and wants the browser to stop being the exception — who values the keyboard grammar enough to want it on every page. It is for someone who wants a real Chromium with real extensions, not a web view in a native shell, so the *WebView* wrappers are out. It is for someone who would rather own a rebrand of an upstream snapshot than consume a company's fork, so the update cadence stays Google's and the customization stays readable. And it is for someone willing to run a separate profile for their power tools, keeping their real logins in their real browser. It is not for someone who wants a supported product with a company behind it, or who needs the native-chrome HUD without being willing to build and rebase, or who wants their everyday and their experimental browser to be the same profile. The category “custom browser” holds all of these people; *zwire* is the answer for the specific one who wants the fleet's grammar on a Chromium they own.

Chapter 30 — Failure Modes and Recovery

Chapter 24 gave the degradation ladder — the principled account of what is fatal and what is best-effort. This chapter is the field guide beside it: the symptoms you will actually see, the cause behind each, and the fix, ordered roughly by how often you hit them. It is the chapter you turn to when `zwire` is misbehaving and you want the one command that sets it right, and it closes with the recovery kit — the small set of scripts that resolve almost everything.

The base is gone or wrong

The one fatal condition is a missing base, and it announces itself clearly: a fresh `zwire` on a machine with no base recorded exits with the exact `run: .../scripts/fetch-base.sh` message rather than failing opaquely. The fix is the command it printed. The subtler version is a *wrong* base — a snapshot that is too old for the `minimum_chrome_version: 127` the power-tool declares, or one that lost the `--load-extension` switch. The tell is that the browser launches but the extensions do not load or the power-tool refuses to; the fix is `fetch-base.sh` to a current snapshot, or `fetch-base.sh <revision>` to pin a known-good one. The base is the floor of the whole stack, so almost any “nothing works” symptom is worth checking against the base first: `base.path` and `base.version` at the top of the state directory are the entire contract, and if `base.path` points at a binary that is gone, everything above it fails in confusing ways until you re-fetch.

The rebrand reverted (macOS)

The most common cosmetic failure, and the one Chapter 24 warned would happen, is the Dock reverting to “Chromium” after a base upgrade. The cause is mechanical: `fetch-base.sh` unzips a fresh stock bundle over the patched one, so the name, the icon, the deleted `CFBundleIconName`, and the ad-hoc re-sign are all gone. The fix is `rebrand-macos.sh`, the one manual step an in-place upgrade must not forget — which is why `install.sh` chains it on a first install but leaves it to you on an upgrade. If the rebrand ran but the old icon lingers, the cause is one level further out: macOS’s LaunchServices cache is still serving the previous icon, and `bust.sh` forces it to refresh. The order to try is: re-run the rebrand first, and only reach for `bust.sh` if the name is right but the icon is stale.

The native host is silent

If the terminal will not open and the statusbar’s system stats are blank, the native host is not answering. This is not fatal — the browser runs fine without it, with one `stderr` line — but it disables the two features that depend on it. The launcher builds the host on launch when its manifest is missing or its target binary is gone, so the first fix is to relaunch and let it rebuild, or to force it with `FORCE=1`. If it still will not connect, the manifest is the thing to check: `com.zwire.hud.json` under the profile’s `NativeMessagingHosts/` must point `path` at a binary that exists and must list the extension’s origin in `allowed_origins`. On Windows the wiring is the registry rather than the profile dir, so a Windows host that is silent is a registry problem, not a file one. The daemon path has its own failure — a socket that will not open — and the cause there is almost always

permissions: the socket is `0600` inside a `0700` dir, so a socket owned by another user, or a stale socket file from a crashed daemon, blocks a new one until it is removed.

A submodule is missing

If a feature is simply absent — no power-tool, no shared toolkit behaviors — the likely cause is an uninitialized submodule. `zpwchrome` and `zgui-core` are submodules, and a clone without `--recurse-submodules` leaves their directories without a `manifest.json`, so the launcher skips them silently and loads what remains. The symptom is a browser that works but is missing a whole capability; the fix is `git submodule update --init --recursive`. This is best-effort by design — a missing submodule degrades rather than crashes — which is exactly why it can be easy to miss: nothing errors, the feature is just not there. If a submodule is present but at the wrong commit, the tell is subtler still, a feature that half-works or behaves like an older version; the fix is to sync the submodule to the pointer the superproject records.

Extensions did not load, or the wrong ones did

If the developer-mode banner appears but a specific extension is not running, the staging is the place to look. The launcher copies the loaded extensions into `~/.zwire/ext/` at launch rather than running them in place, so a stale copy there can shadow a change you made to the source. The fix is to clear the staged copy and let the launcher restage — the dev-loop scripts (Chapter 22) do this as part of a rebuild. If the *wrong* extensions loaded, or system Chrome's extensions somehow appeared, the cause is a profile confusion: `zwire` must be running against `~/.zwire/profile`, and if `$ZWIRE_STATE` or the profile path were overridden to point elsewhere, the browser is loading a different world. The `state-dir.sh` resolution and the `base.path` contract are the two things to verify when the browser is running but is not the browser you expected.

Schemes desynced

If the new tab is one scheme and the internal pages are another, the sync bus has a stale reader. The scheme lives in a native file the host owns, written atomically, and every surface reads it — the new-tab poller, the internal HUD, the fork's color mixer. A desync means one reader did not see the last write: usually a page that was open across the change and did not re-read, which a reload fixes, or a host that is not running so the pub/sub push never fired, which folds back into the silent-host case above. The mirrored `zb_scheme` storage key is the cross-script copy; if it and the native file disagree, the native file is the source of truth and the fix is to re-trigger a scheme write (set the scheme again) so every subscriber gets the push.

The fork will not rebase

On the fork path, the recurring failure is a patch that no longer applies after upstream moved, and Chapter 21's economics section is the frame for it. The `check-patch-series.sh` gate catches the structural drift — a patch on disk missing from `series`, or a `series` entry with no file — before anything applies. A patch that applies but is *wrong* after a rebase is the dangerous case the CI cannot catch, and the audio patches are the ones to re-verify by hand. The pragmatic recovery is the one the series file already models: if a patch has become fragile, back it out with a commented-out `series` line and a note, as `0008` was, rather than carrying a broken one forward. And the ultimate fallback is the two-tier design itself — if the fork will not build, the runtime rebrand still runs on a stock snapshot, so a broken rebase costs you the native-chrome HUD, not a working browser.

The recovery kit

Almost everything above resolves to one of a small set of scripts, and it is worth holding them as a kit. `fetch-base.sh` re-establishes the base, pinned or latest, and fixes the fatal case and the wrong-base case. `rebrand-macos.sh` re-applies the identity after any base swap, and `bust.sh` clears the LaunchServices cache when the icon is the only thing stale. `setup-native-host.sh` (or a relaunch, or `FORCE=1`) rebuilds and rewires the host when the terminal and stats go quiet. The dev-loop rebuild restages the extensions when a stale copy shadows a change. `git submodule update --init --recursive` restores a missing capability. And `nuke.sh` is the bottom of the ladder — a full reset of the state directory back to nothing, after which a single `install.sh` (or `fetch-base.sh` plus `rebrand-macos.sh`) rebuilds the world from the base up. The reason the kit is this small is the reason the whole browser is this small: zwire is mostly a shell script wrapped around a stock engine, so almost every failure is a missing file, a stale cache, or a reverted patch — and each has one command that puts it back.

Appendix A – File and Directory Map

A single reference for where each thing this book discussed actually lives in the [zwire](#) repository.

```

bin/zwire                                the launcher - resolve base, build ext list, ensure
├─ host, exec                             edition source (version), npm-script aliases
package.json                             zwire.icns, zwire.ico, icon.svg, icon-1024.png
branding/

scripts/
  fetch-base.sh                          download + pin the Chromium snapshot base
  install.sh                             PATH symlink + fetch + (macOS) rebrand
  rebrand-macos.sh                       in-place Dock name + icon + re-sign
  setup-native-host.sh                   build zwire-host + write profile-relative manifest
  localinstall.sh                        self-contained /Applications/zwire.app
  localinstall-linux.sh                  self-contained ~/.local/opt/zwire + .desktop
  localinstall-windows.ps1               self-contained %LOCALAPPDATA%\zwire + registry host
  dev / build / rebuild / clean          dev-loop helpers (no compile - assemble base)
  bust / nuke / test / cyberpunk         cache-bust · full reset · syntax gate · banner lib
  check-patch-series.sh                  CI: series file <-> patch files consistency

newtab/
  newtab.{html,css,js}                  the HUD new-tab extension (chrome_url_overrides)
  palette.js schemes.js                  page, style, logic (clock/tiles/omnibox/focus)
  scheme-sync.js tmux-pane.js            new-tab command palette + the 8-scheme table
  fonts/                                scheme follow via native file + tiling surface
                                          locally-vendored display faces (offline-correct)

extensions/
  hud-internal/                          the internal-HUD content-script suite
    manifest.json                        MV3: content-script order, commands, permissions
    background.js                        scheme relay + tab bus + term-open state + cmd seed
    theme.js schemes.js                  chrome:// WebUI skin + the 8-scheme table
    redirect.js rules/frame_bust.json    redirect chrome:// pages · strip X-Frame-Options
    zvim.js ztmux-config.js zpowerline.js vim keys · tmux overlay (ZGui.tmux) ·
├─ statusbar (ZGui.powerline)
  zpalette.js zfind.js                  global %%K palette · fuzzy find (Custom Highlight API)
  zkeys.js cmd-defaults.js              keybinding registry · seeded %%K keyword commands
  pages/
    extensions.js                       extension manager (management + developerPrivate)
    settings.js                         all Chrome prefs (settingsPrivate) + scheme/effects
    history/bookmarks/downloads          the three list pages (shared zg-list.js)
    sessions.js                         durable tmux-session CRUD (zb_tmux_sessions)
    version.js                          HUD chrome://version + live native-host probe
    keys.js commands.js                 keybinding editor · %%K keyword-command editor
    extshortcuts.js ci.js               extension shortcuts · GitHub Actions dashboard
    zg-boot.js zg-list.js fzf.js        shared shell · list factory · fuzzy matcher
  lib/term/                             xterm.js terminal (native-PTY bridged)
  native/zwire-host                     scheme + ui + sysinfo + PTY + fs/exec native host
├─ (Rust)
  native/zwire-host/src/                the Rust host: transport/proto/session/store/
                                          sysmon/pty/exec/fsops/jobs/watch/procs/osops/
                                          bus/peer/api (submodule, packaged path)
                                          shared fuzzy/palette/fx/fonts toolkit (submodule)
                                          the MV3 power-tool (submodule)

  lib/zgui-core/
  zpwrchrome/

theme/
  a colors-only Chrome theme - present, NOT loaded

fork/
  CHROMIUM_VERSION                      the source-build path (optional alternate base)
  args.gn                               pinned tag: 150.0.7871.46
  fetch.sh apply-patches.sh             unbranded release GN config
  build.sh package.sh                   depot_tools + checkout · apply/reverse series
  patches/0001..0024 + series           gn gen + autoninja · install as base + rebrand
                                          the twenty-four HUD patches, apply-clean at the tag

v{keys,ovl,pl,pop,pos,sys,term,win}.mjs CDP verification probes (port 9222)
.github/workflows/ci.yml                shellcheck + JSON validate + patch-series check
.github/workflows/pages.yml              publish docs/ to GitHub Pages

```

Appendix B — The State Directory (\$ZWIRE_STATE)

Everything zwire writes at runtime, in one place. The default is the OS application- data location — `~/Library/Application Support/com.menketotechnologies.zwire` on macOS (named by the reverse-DNS bundle id), `${XDG_CONFIG_HOME:--/.config}/zwire` on Linux — resolved by `scripts/state-dir.sh`; the old `~/ .zwire` dotdir and the brief bare-zwire macOS folder are legacy locations, auto-migrated once. `$ZWIRE_STATE` overrides it on every platform.

```
$ZWIRE_STATE/                                (default: OS app-data dir; ~/.zwire + bare zwire are
├── legacy, migrated)                        the extracted Chromium snapshot, or the packaged fork
├── base/                                    the extracted Chromium snapshot, or the packaged fork
├── (~325MB)
├── base.path                               absolute path to the resolved binary (launcher reads
├── this)
├── base.version                            "r<REV>" for a snapshot, "fork-<TAG>" for a compiled
├── fork
├── profile/                               the dedicated --user-data-dir
├── (bookmarks/history/sessions)
├── NativeMessagingHosts/
│   ├── com.zwire.hud.json                 the HUD native host manifest (path + allowed_origins)
│   └── Default/NativeMessagingHosts/    second location Chromium also checks
├── ext/                                   per-user writable copy of the loaded extensions (staged
├── at launch)
```

The extension-owned keys inside `profile/` that this book referenced:

```
zb_scheme      the mirrored active scheme (worker <-> content scripts)
zb_keys         user keybinding remaps { action: key }
zb_custom_cmds  the %K keyword-command rules
zb_sys          the streamed system-stats sample (statusbar reads this)
zb_term_open    whether the terminal overlay should re-open on the next page
zb_cmd          the storage command bus (wakes the worker for tab ops)
zb.tiles/zb.engine  new-tab tiles + search engine (page localStorage, not chrome.storage)
```

The two files at the top level — `base.path` and `base.version` — are the entire contract between the base layer and the launcher. The theme itself lives *outside* this per-install tree, in the fleet-wide `~/ .zwire/global.toml` (with the host's `hud-scheme/hud-light` text projections beside it for the fork's native color mixer) — the one channel through which the new-tab extension, the internal HUD, `zpwichrome`, the wider app suite, and (on the fork) the native chrome all agree on one look. `profile/` and `ext/` are the isolation boundary that keeps zwire a guest on the machine. Delete the whole tree and the next `zwire` launch fails with “no base browser installed” until you re-fetch — there is no hidden state anywhere else.

Appendix C – Keybinding Reference

The full default keymap from [zkeys.js](#), the single source of truth both [zvim.js](#) and the Keyboard settings page read. Every key here is remappable in the settings page (which writes [zb_keys](#)), except the fixed native prefix.

```
— Scroll —
j scroll down      k scroll up
h scroll left      l scroll right
d half-page down  u half-page up
G bottom of page  H top of document
M middle of document L bottom of document

— Tabs & history —
J previous tab    K next tab
x close tab      t new tab      r reload
[ history back   ] history forward

— Jump & hints —
f link hints (click)    F link hints (new tab)
g g-prefix: gg top · gt/gT tab next/prev · gi focus input
z z-prefix: zt/zz/zb position ref element top/center/bottom
y y-prefix: yy yank URL
m set mark m<x>          ` jump to mark `<x> (also ')

— Palette & find —
o command palette      : command palette
/ find in page         \ toggle vim off (Ctrl/⌘+\\ re-enables)
⌘K / Ctrl+K open palette    ⌘F / Ctrl+F open find

— Tmux overlay (prefix Ctrl-b, or ⌘b) —
% split right    " split down    o next pane    ; last (prev) pane
h select pane ←  j select pane ↓  k select pane ↑  l select pane →
H resize pane ←  J resize pane ↓  K resize pane ↑  L resize pane →
z zoom          x close pane      { swap ←        } swap →
O rotate        ! break to window q pane numbers  space cycle layout
F grid 4        G grid 8          I grid 16
c new window    n next window    p prev window    Tab last window
, rename window . move window    ' go to window    w window list
& kill window  m mark / swap    e sync panes     E toggle pane in sync
[ copy mode    ] paste buffer  = choose buffer
s sessions     S save session
r reload pane  t clock          : palette        d detach    ? help
0-9 select window    ↔↑↓ focus by direction
Ctrl/Alt + ↔↑↓ resize the active pane

— Fixed (native fork) —
Ctrl-b the tmux split prefix (rebindable in the Keyboard page)
```

Appendix D — The Eight-Scheme Palette

The full dark palettes from `schemes.js`. Every HUD surface styles itself in terms of these twenty CSS variables; each scheme also ships a matched `lightVars` set (omitted here for space) for light mode. Colors are the `vars` (dark) values.

```
cyberpunk  - Hot pink + cyan (default)
  accent #ff2a6d  cyan #05d9e8  magenta #d300c5  green #39ff14  red #ff073a
  text #e0f0ff / dim #7a8ba8 / muted #3d4f6a
  bg #05050a / #0a0a14 / card #0d0d1a / hover #12122a  border #1a1a3e

midnight   - Deep blue + electric purple
  accent #7c3aed  cyan #38bdf8  magenta #6366f1  green #34d399  red #f87171
  text #e2e8f0 / dim #94a3b8 / muted #475569
  bg #020617 / #0f172a / card #1e293b / hover #334155  border #334155

matrix     - Green phosphor terminal
  accent #00ff41  cyan #00ffcc  magenta #008f39  green #00ff41  red #ff3333
  text #d4ffd4 / dim #7ccc7c / muted #406640
  bg #000500 / #001a00 / card #002a00 / hover #003a00  border #004d00

ember      - Burnt orange + amber
  accent #f97316  cyan #fbbf24  magenta #dc2626  green #84cc16  red #ef4444
  text #fed7aa / dim #d97706 / muted #78350f
  bg #0a0503 / #1a0f05 / card #27150a / hover #3a1f0f  border #4a2c17

arctic     - Pale ice + steel blue
  accent #60a5fa  cyan #22d3ee  magenta #818cf8  green #2dd4bf  red #f87171
  text #e0f2fe / dim #7dd3fc / muted #0369a1
  bg #020617 / #0c1424 / card #0f1e33 / hover #1e2f4d  border #1e3a5f

crimson    - Rose-red accent + teal highlight
  accent #e11d48  cyan #2dd4bf  magenta #f43f5e  green #22c55e  red #ff073a
  text #ffe4e6 / dim #b08a92 / muted #6b4a52
  bg #0a0506 / #140a0c / card #1a0d10 / hover #2a1318  border #3e1a22

toxic      - Acid-lime accent + magenta
  accent #c6ff00  cyan #00e5ff  magenta #ff00aa  green #39ff14  red #ff073a
  text #e8ffd0 / dim #8a9a6a / muted #4a5a32
  bg #07090a / #0c0f0a / card #0f130c / hover #161b10  border #2a3a1a

vapor      - Vaporwave pastel pink + cyan
  accent #ff6ec7  cyan #72f1ff  magenta #c792ea  green #5af2b0  red #ff6b8b
  text #f0e6ff / dim #a99cc4 / muted #6a5f86
  bg #0d0814 / #140d1f / card #1a1228 / hover #241836  border #2e2142
```

The twenty keys in full order (`SCHEME_VAR_KEYS`): `--accent`, `--accent-light`, `--accent-glow`, `--cyan`, `--cyan-glow`, `--cyan-dim`, `--magenta`, `--magenta-glow`, `--green`, `--green-bg`, `--red`, `--text`, `--text-dim`, `--text-muted`, `--bg-primary`, `--bg-secondary`, `--bg-card`, `--bg-hover`, `--border`, `--border-glow`.

The light palettes (lightVars)

Each scheme ships a matched light variant under `lightVars`, and light mode swaps these in for the dark `vars` above. The light sets invert the depth relationship — `--bg-primary` becomes a near-white, `--bg-card` becomes `ffffff`, the text darkens — while keeping each scheme’s hue identity, with accents deepened for contrast on a light ground. The full light key colors, in the same layout as the dark table:

```
cyberpunk  accent #d6196e  cyan #0891b2  magenta #a300a3  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #f0f2f5 / #e4e7ec / card #ffffff / hover #f7f8fa  border #cbd5e1

midnight   accent #6d28d9  cyan #0284c7  magenta #4f46e5  green #047857  red #dc2626
text #0f172a / dim #475569 / muted #94a3b8
bg #f1f5f9 / #e2e8f0 / card #ffffff / hover #f8fafc  border #cbd5e1

matrix     accent #16a34a  cyan #059669  magenta #15803d  green #16a34a  red #dc2626
text #14532d / dim #4d7c0f / muted #86efac
bg #f0fdf4 / #dcfce7 / card #ffffff / hover #f7fee7  border #bbf7d0

ember      accent #c2410c  cyan #ca8a04  magenta #b91c1c  green #65a30d  red #dc2626
text #431407 / dim #9a3412 / muted #fdb74
bg #fff7ed / #ffedd5 / card #ffffff / hover #fef3c7  border #fed7aa

arctic     accent #2563eb  cyan #0891b2  magenta #4f46e5  green #0f766e  red #dc2626
text #0c4a6e / dim #0369a1 / muted #7dd3fc
bg #f0f9ff / #e0f2fe / card #ffffff / hover #f0f9ff  border #bae6fd

crimson    accent #be123c  cyan #0d9488  magenta #be185d  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #faf0f1 / #f2e4e6 / card #ffffff / hover #fdf6f7  border #e0c5cb

toxic      accent #5c8a00  cyan #0891b2  magenta #a3006e  green #15803d  red #dc2626
text #1e293b / dim #475569 / muted #94a3b8
bg #f3f7ec / #e7eedb / card #ffffff / hover #f8fbf2  border #d0dcc0

vapor      accent #d6469e  cyan #0e9bb0  magenta #8b5cf6  green #10a37f  red #e11d48
text #2a1e3a / dim #5a4a72 / muted #9a8ab0
bg #f7f0fb / #ede2f5 / card #ffffff / hover #faf4fd  border #ddc9ec
```

Two families collapse toward a shared light red: every scheme except vapor uses `#dc2626` for `--red` in light mode (vapor keeps a rose `#e11d48`), where the dark palettes each carried their own. That is the one place the light variants converge — a bright neutral red reads as “error” on any light background, so the schemes stop differentiating it.

Appendix E – Launch Flag Reference

Every flag `bin/zwire` passes to the base, and why. User arguments ("`$@"`") are appended after these, so any Chromium flag you pass overrides or adds to them.

<code>--user-data-dir="\$PROFILE"</code>	isolate to <code>~/.zwire/profile</code> (never system Chrome)
<code>--load-extension="\$LOAD"</code>	preload newtab + zpwrchrome + hud-internal (the switch unbranded Chromium keeps, branded Chrome dropped in 137) – only dirs with a <code>manifest.json</code>
<code>--extensions-on-chrome-urls</code>	let hud-internal content scripts run on <code>chrome://</code>
<code>--test-type</code>	suppress the automation-restriction warnings the above flags would otherwise raise
<code>--no-first-run</code>	skip Chromium's onboarding
<code>--no-default-browser-check</code>	skip the default-browser nag
<code>--homepage="chrome://newtab"</code>	the home button opens the HUD new tab
<code>--disable-features=NtpFooter</code>	drop the new-tab footer under the HUD page
<code>--enable-features=SplitViewHorizontal,SplitViewTabRestore</code>	side-by-side split view + restore-on-reopen
<code>"\$@"</code>	everything you typed, passed straight through (e.g. a URL, <code>--incognito</code> ,
<code>--remote-debugging-port=9222</code>)	

That is the whole launcher contract: nine HUD-defining flags, then your arguments. The launcher owns what makes `zwire` `zwire` and gets out of the way of every other Chromium flag — which is why `zwire --remote-debugging-port=9222` (Chapter 20), `zwire --incognito`, and `zwire https://github.com` all just work.

Appendix F – The Chrome WebUI Token Map

`theme.js` skins the `chrome://` pages that have no bespoke HUD replacement by overriding Chrome's own CSS custom properties, each with `!important`, so the WebUI components recolor without any markup change. It deliberately maps *only* palette and the mono font — dropping the grid lattice, scanlines, glow, and floating picker — so these pages inherit the theme with no HUD chrome on top. The full override map, token to HUD variable:

```

— Chrome design-system tokens (--color-sys-*) —————
--color-sys-surface                -> --bg-primary
--color-sys-surface-variant        -> --bg-card
--color-sys-surface-subtle         -> --bg-secondary
--color-sys-base                   -> --bg-primary
--color-sys-base-container         -> --bg-card
--color-sys-base-container-elevated -> --bg-hover
--color-sys-header                 -> --bg-secondary
--color-sys-header-container       -> --bg-secondary
--color-sys-on-surface             -> --text
--color-sys-on-surface-subtle      -> --text-dim
--color-sys-on-surface-variant     -> --text-dim
--color-sys-on-surface-secondary  -> --text-dim
--color-sys-primary                -> --cyan
--color-sys-on-primary             -> --bg-primary
--color-sys-primary-container      -> --bg-hover
--color-sys-on-primary-container   -> --accent
--color-sys-secondary              -> --accent
--color-sys-outline                -> --border
--color-sys-tonal-container        -> --bg-hover
--color-sys-tonal-outline          -> --border
--color-sys-neutral-container      -> --bg-card
--color-sys-inverse-surface        -> --text
--color-sys-state-hover-on-subtle  -> --bg-hover

— Fallback tokens (--cr-fallback-color-*) —————
--cr-fallback-color-base-container -> --bg-card
--cr-fallback-color-surface        -> --bg-primary
--cr-fallback-color-primary        -> --cyan
--cr-fallback-color-on-surface     -> --text
--cr-fallback-color-on-surface-subtle -> --text-dim
--cr-fallback-color-outline        -> --border
--cr-fallback-color-neutral-container -> --bg-card
--cr-fallback-color-tonal-container -> --bg-hover
--cr-fallback-color-divider        -> --border

— Component tokens (--cr-*) —————
--cr-card-background-color         -> --bg-card
--cr-primary-background-color      -> --bg-primary
--cr-secondary-background-color    -> --bg-secondary
--cr-primary-text-color            -> --text
--cr-secondary-text-color          -> --text-dim
--cr-toolbar-background-color      -> --bg-secondary
--cr-hover-background-color        -> --bg-hover
--cr-link-color                    -> --cyan
--cr-focus-outline-color           -> --accent
--cr-toggle-color                  -> --cyan
--cr-separator-line                -> 1px solid var(--border)

— Legacy Google tokens (--google-*) —————
--google-grey-900 -> --bg-primary  --google-grey-800 -> --bg-card
--google-grey-700 -> --bg-hover    --google-grey-500 -> --text-dim
--google-blue-300 -> --cyan         --google-blue-400 -> --cyan
--google-blue-500 -> --accent       --google-blue-600 -> --accent

— Also set —————
color-scheme: dark
font-family: "Share Tech Mono","Monaco",monospace (on the full element list)

```

The stylesheet is injected into `document.head` and recursively into every open shadow root, because Chrome's

WebUI is a tree of custom elements and a `:root`, `:host` rule must reach inside each shadow boundary to recolor the components within it. The scheme it reads is `zb_scheme` from `chrome.storage.local` (a content script cannot call the native host itself), falling back to a `zbhud-get` worker message, and it applies cyberpunk first to avoid a flash of the stock theme before the real scheme resolves.

Appendix G – The Native-Messaging Protocol

Every message the browser exchanges with the native host, framed as a 4-byte little-endian length followed by that many UTF-8 bytes of JSON. The **legacy path** is what zwire's browser side actually invokes — the scheme bridge, the sysinfo stream, and the PTY relay of Chapter 15. The **extended** commands are the larger local-machine agent **zwire-host** also exposes; zwire's front end does not call them, but they ship in the same binary and are reachable over its socket daemon.

```
— Legacy zwire path (what the browser uses) —————
{cmd:"get"}           -> {scheme, ui}   read the scheme + UI-pref files
{scheme:"<name>"}      validate vs 8 names, atomically write hud-scheme
{ui:{...}}            merge visual-effect prefs into hud-ui.json (atomic)
{cmd:"sysinfo_start"}  stream {sys:{...}} per interval until the port closes
{cmd:"pty_spawn"}      fork a login shell in a PTY (becomes a terminal port)
  {cmd:"pty_write", ...} keystrokes to the shell
  {cmd:"pty_resize", ...} TIOCSWINSZ the window size
  {cmd:"pty_kill"}      end the shell
  <- {ev:"output", b64:...} shell output, base64, 64KB chunks
  <- {ev:"exit"}        the shell died

— Extended agent (same binary, not called by zwire UI) —
hello / ping           {ok,host,version,os,arch,pid,caps:[...]}
kv_get/set/merge/del/keys  namespaced JSON key-value store (~/.<app>/kv/)
fs_read/write/append      file contents (64 MiB read cap)
fs_list/walk/stat         directory listing + bounded recursive crawl
fs_mkdir/rm              make (-p) / remove (recursive for dirs)
fs_watch / fs_tail       stream change events / follow a file (rotation-aware)
watch_stop / watch_list  drop / list active observers
exec                    run a program to completion -> {ok,code,stdout,stderr}
job_start/poll/list/result background jobs (+ desktop notify on completion)
ps / kill / which        process list / signal / PATH resolve
sub / unsub / pub        process-global publish-subscribe bus
open / clipboard_get/set  OS open + clipboard
notify / hostinfo         desktop notification / host facts
sysinfo_once / sysinfo_stop one-shot sample / stop a running stream
auth / peer_hello         token auth / peer handshake (TCP mesh)
peer_connect/peers/remote dial a peer / list peers / run a request remotely
```

The **sys** sample carries: **cpu**, **mem{u,t,p}**, **swap{u,t,p}**, **load[3]**, **uptime**, **disk{u,t,p}**, **net{up,down}**, **io{r,w}** (disk read/write rate), **temp**, **host**, **lip** (LAN IP), **pip** (public IP, refreshed sparingly), and **batt{p,c}** — the battery read natively per platform (**pmset** on macOS, **/sys/class/power_supply** on Linux, **GetSystemPowerStatus** on Windows), the disk-I/O and network figures as per-second rates over the elapsed interval. The stream cadence is a configurable **interval_ms**, with one-shot (**sysinfo_once**) and **sysinfo_stop** variants.

Appendix H – The Seeded Keyword Commands

The `⌘K` palette's keyword commands are seeded from `cmd-defaults.js` (`ZWIRE_CMD_DEFAULTS`) on first use and thereafter live in the user-editable `zb_custom_cmds` array, edited on the Commands page. New defaults added to the file appear automatically; a default you delete is remembered in `zb_cmds_removed` and never re-added. Each `url` entry substitutes your typed argument for `{q}`; the three `action` entries take no argument. The full seed set:

```
— AI & search —
cg ChatGPT          cl Claude          pp Perplexity
gm Gemini           wa WolframAlpha    kagi Kagi
bing Bing          def Define word    dd DevDocs

— Google —
gm Gmail            gd Google Drive    cal Google Calendar
tr Google Translate img Google Images  sch Google Scholar
fl Google Flights  wx Weather

— Code & dev —
gh GitHub           gist GitHub Gist    pr GitHub PRs
iss GitHub Issues  cop GitHub Copilot  gl GitLab
bb Bitbucket       grep grep.app       rex regex101
ciu Can I Use      bp Bundlephobia    npt npm trends
mvn Maven Central  hf Hugging Face    cp CodePen
csb CodeSandbox    repl Replit        aw Arch Wiki
uni Emoji / Unicode

— Cloud & tools —
aws AWS Console    gcp GCP Console    vc Vercel
cf Cloudflare      no Notion           fig Figma
lin Linear         trl Trello

— Comms —
li LinkedIn        sl Slack           dc Discord
tg Telegram        wsp WhatsApp

— Media & shopping —
imdb IMDb          nf Netflix         sp Spotify
ytm YouTube Music  tw Twitch          sc SoundCloud
eb eBay            etsy Etsy         abnb Airbnb
yelp Yelp

— Reading —
hn Hacker News     nyt New York Times arx arXiv

— Actions (no argument) —
rl Reload page     cu Copy page URL   cs Cycle color scheme
```

The list is intentionally a starting kit, not a fixed set: it is seeded once, yours to prune and extend, and the seeder's job is only to make sure a fresh profile has a sensible default without ever overwriting what you have

changed. (The worker carries a shorter first-run seed of the same shape; the `cmd-defaults.js` list is the fuller one that backfills the rest by id on the next load.)

Appendix I – The Storage Keys

The HUD's cross-script state lives in `chrome.storage.local` under a family of `zb_*` keys, plus a couple of page-local `localStorage` keys and the two native files. This is the whole set the book referenced, in one place — the shared memory the content scripts, the worker, and the HUD pages reconcile through:

```
— chrome.storage.local (shared across the extension) —
zb_scheme           the mirrored active scheme (worker <-> content scripts)
zb_sys              the streamed system-stats sample (statusbar reads this)
zb_tabs             live list of open tabs (palette reads it, worker writes it)
zb_exts             enabled extensions (palette Tweak/Manage entries)
zb_frecent          frecenty-scored history, top 30 (palette + new tab)
zb_shortcuts        every extension's keyboard shortcuts (via developerPrivate)
zb_cmd              the command bus – content scripts write, worker executes
zb_keys             user keybinding remaps { action: key }
zb_custom_cmds      the %%K keyword-command rules
zb_cmds_seeded      one-shot guard: the worker's first-run seed ran
zb_cmds_removed     tombstones for deleted default commands (never re-seed)
zb_status           whether the powerline statusbar is shown
zb_term_open        whether the terminal overlay should re-open on the next page
zb_tmux             statusbar readout of the in-page overlay (counts, prefix armed)
zb_tmux_state       the worker window-tiling engine's state (eviction recovery)
zb_tmux_sessions    durable named tmux layouts (the Sessions page)
zb_tmux_load        trigger the live overlay watches to attach a saved session
zb_tmux_prefix      the tmux prefix chord (Keyboard page)
zb_tmux_opts        tmux options, e.g. { timeout } (Keyboard page)
zb_ci               CI-monitor config { user, token, repolimit }
zb_ci_status        the CI tally { fail, running, ok, total, at }
zb_ci_cache         cached CI run data (the CI page)

— page localStorage (per-origin, not shared) —
zb.tiles            new-tab quick-launch tiles [{ label, url }]
zb.engine           new-tab search engine template with %s

— native files (~/.zwire, the cross-extension bus) —
hud-scheme          the active scheme (fork mixer + new-tab both read it)
hud-ui.json         shared visual-effect + light-mode prefs
```

The distinction that matters: `chrome.storage.local` is shared among the internal HUD's own scripts but *not* across extensions, and `localStorage` is not shared even across origins — so anything the new-tab extension or the power-tool must also see travels through the two native files, which are the only channel two extensions share. That is the whole reason `hud-scheme` and `hud-ui.json` exist as files rather than storage keys.

Appendix J – The Rust Host Modules

`zwire-host` is organized into fifteen small modules under `src/`, and the split is the map of the superset described in Chapter 23. For the reader who opens the crate, each module and its one job:

<code>main.rs</code>	entry point: pick the transport, run the session loop
<code>lib.rs</code>	the crate root; re-exports the api facade for embedders
<code>transport.rs</code>	three entry points – Chrome stdio, the NDJSON socket daemon, and a one-shot client call
<code>proto.rs</code>	framing (4-byte LE length + JSON; NDJSON for the socket), the 128-MiB message guard, id-tagged replies
<code>session.rs</code>	the per-connection router: dispatch, held PTYs/watchers/subs, teardown on drop
<code>store.rs</code>	file-backed state under <code>~/.<app>/</code> – the kv store and the legacy scheme/ui files, all atomic tmp-and-rename (no SQLite)
<code>sysmon.rs</code>	the system sample via the <code>`sysinfo`</code> crate (not <code>psutil</code>), streamed every interval
<code>pty.rs</code>	PTY terminals via <code>`portable-pty`</code> , multiplexed by id, base64 I/O
<code>exec.rs</code>	run a program to completion (base64 stdout/stderr + exit code)
<code>fsops.rs</code>	read/write/append/list/walk/stat/mkdir/rm, with size + count caps
<code>jobs.rs</code>	background jobs + completion notifications
<code>watch.rs</code>	poll-based <code>fs_watch</code> and rotation-aware <code>fs_tail</code> observers
<code>procs.rs</code>	<code>ps</code> / <code>kill</code> / <code>which</code> (reusing the <code>sysinfo</code> crate)
<code>osops.rs</code>	<code>open</code> / <code>clipboard</code> / <code>notify</code> / <code>hostinfo</code> (shelling to platform tools)
<code>bus.rs</code>	the process-global publish-subscribe broker
<code>peer.rs</code>	the TCP host-to-host mesh: auth, dial, bus federation, remote
<code>api.rs</code>	an in-process library facade (<code>exec/walk/read/write</code>) for embedders

The release profile is tuned for a small artifact — `opt-level="z"`, LTO, symbols stripped, `panic="abort"` — which is how a host that carries a kv store, a filesystem API, a PTY multiplexer, a job runner, and a TCP mesh still fits in roughly half a megabyte with no external runtime. `zwire`'s browser uses only `store.rs`, `sysmon.rs`, and `pty.rs`; the rest is the larger local agent the same binary offers any app that speaks its socket.

Appendix K – The Launcher and Build Scripts

The runtime path is a handful of shell scripts. Their contracts, in one place:

```
— bin/zwire – the launcher, in sequence —————
1 source state-dir.sh; STATE = $ZWIRE_STATE or the OS app-data dir
2 one-time migrate a leftover ~/.zwire into STATE (if no $ZWIRE_STATE)
3 base = executable $ZWIRE_BASE, else $STATE/base.path, else FAIL
  "zwire: no base browser installed – run: ../scripts/fetch-base.sh"
4 ext list = { newtab, extensions/zpwrchrome, extensions/hud-internal }
  filtered to dirs with a manifest.json (theme/ deliberately excluded)
5 read "path" from com.zwire.hud.json; if missing/dangling run
  setup-native-host.sh (never fatal – browser runs without the host)
6 exec base with the nine flags + "$@"

— the nine launch flags (Appendix E has the why) —————
--user-data-dir --load-extension --extensions-on-chrome-urls --test-type
--no-first-run --no-default-browser-check --homepage=chrome://newtab
--disable-features=NtpFooter
--enable-features=SplitViewHorizontal,SplitViewTabRestore
  (the Linux self-contained launcher adds a tenth: --no-sandbox)

— fetch-base.sh – host -> snapshot —————
Darwin/arm64 -> Mac_Arm    chrome-mac.zip    chrome-mac
Darwin/x86_64 -> Mac       chrome-mac.zip    chrome-mac
Linux/x86_64  -> Linux_x64  chrome-linux.zip  chrome-linux
MINGW/MSYS   -> Win_x64    chrome-win.zip    chrome-win
  revision = arg, else the platform's LAST_CHANGE
  writes base.path (abs binary) + base.version ("r<REV>")

— the self-contained install layouts —————
macOS    /Applications/zwire.app/Contents/{Resources/{browser,ext,native},MacOS/zwire}
Linux    ~/.local/opt/zwire/{browser,ext,native} + PATH link + .desktop
Windows  %LOCALAPPDATA%\zwire\{browser,ext,native} + zwire.cmd + registry host
  each stages extensions per-user (rsync, excluding _metadata) at launch
```

The three pinned extension ids appear identically in four scripts and each extension's own **key**: HUD [omcggnjfm b m p d l o f k l b p d d k h n f i b f h g g](#), zpwrchrome [h p p p d c h p n p h m i i j d e a n i b p c a d g k n m a j a](#), new-tab [g p o e p n e k o i p l h k e g j p o c n p e i j i e f g i e b](#). The native host name is `com.zwire.hud`, its manifest lists all three as `allowed_origins`, and the pinned fork tag is `150.0.7871.46`.

Appendix L – The CDP Verification Probes

The eight `v*.mjs` probes attach to a running `zwire` over the Chrome DevTools Protocol on `127.0.0.1:9222` (opened only when you pass `--remote-debugging-port=9222`). Each shares one bootstrap — `fetch /json/version`, open the `websocketDebuggerUrl`, `Target.attachToTarget` with `flatten:true`, `Runtime.enable`, then evaluate expressions with `returnByValue:true` — and prints booleans and counts that a human or a CI grep reads as pass or fail. What each verifies:

probe	target	asserts
<code>vkeys</code>	<code>createTarget keys.html</code>	<code>ZWIRE_KEYMAP</code> present · <code>chip/category</code> counts · writes <code>zb_keys.scrollDown='s'</code> + reads it back
<code>vterm</code>	<code>createTarget terminal.html</code>	<code>Terminal</code> global · <code>#terminalPane</code> · <code>.xterm</code> · waits 5s for <code>zsh</code> · rows of shell text appeared
<code>vovl</code>	<code>attach github/amazon/wiki</code>	<code>synth Ctrl+`</code> · <code>#terminalPane</code> becomes <code>.active</code> · <code>.xterm</code> rendered
<code>vpop</code>	<code>attach a stable page</code>	<code>Ctrl+`</code> · <code>.active</code> · waits 7s for the <code>SW->host</code> PTY shell · reads back visible shell text
<code>vwin</code>	<code>attach a stable page</code>	<code>Ctrl+`</code> opens <code>terminal.html</code> popup window · attach it · <code>.xterm</code> · <code>connectNative</code> reachable
<code>vsys</code>	<code>attach a stable page</code>	waits 5s for <code>zb_sys</code> · reads <code>CPU/MEM/NET/HOST/LAN/UP/BATT</code> segments · <code>.plr</code> chevron count · <code>clock</code>
<code>vpos</code>	<code>attach a stable page</code>	left-to-right segment order · <code>C-b</code> prefix sits left of the scheme segment
<code>vpl</code>	<code>attach a stable page</code>	the powerline <code>::</code> after chevron borders + the prefix block background render as intended

`vkeys` and `vterm` open an extension page directly; the other six attach to an existing `github/amazon/wikipedia` tab (and exit cleanly if none is open, since that is a setup gap, not a fault). Two probe lines are dead — `vterm` reaches for a non-existent `window.__zbTermInstance`, and `vovl` prints a placeholder instead of reading `zb_term_open` — noted here so the suite is not oversold.

Appendix M – The zpwrchrome Manifest

The power-tool's manifest, as the concrete proof of Chapter 2 (this book does not otherwise document `zpwrchrome`, which has its own):

```
manifest_version 3 · minimum_chrome_version 127 · module service worker
version 0.22.17 · <all_urls> host access · 6 content scripts · 54 commands

permissions (19):
  userScripts · declarativeNetRequestWithHostAccess · nativeMessaging ·
  webRequest · webRequestAuthProvider · tabs · tabGroups · sessions ·
  bookmarks · history · scripting · webNavigation · cookies · downloads ·
  contextMenus · notifications · clipboardWrite · alarms · storage

the four that require a genuine Chromium engine (Chapter 2):
  userScripts · declarativeNetRequestWithHostAccess · nativeMessaging · webRequest

feature surface (one toolbar icon): browserpass-compatible `pass` (fill/copy/
OTP/autofill via a Rust host over ~/.password-store) · segmented download
manager · MRU tab switcher (scenes/opener-tree/fzf history) · userscript
engine · full-page screenshot · tech detector · theme injector · lights-off
dimmer · JSON/XML viewers · UA switcher · find-in-all-tabs
```

Appendix N – The Fork Patch Targets

The twenty-four HUD patches, each apply-clean against 150.0.7871.46, and the native surface each reaches that a theme extension cannot:

```
0001 tab shape      tab_style_views.cc (GetPath)      trapezoid tabs, hud_slope 0.28
0002 UI colors    chrome_color_mixer.{cc,h}+mixers.cc HUD palette + 8 schemes on frame/
                                toolbar/tabs/omnibox + separator

0003 UI font      resource_bundle.cc (InitDefault...) Share Tech Mono, Monaco fallback
0004 toolbar line toolbar_view.cc (Init, kNormal) 2px cyan under-toolbar border
0005 omnibox      location_bar_view.cc (ComputeBorderRadius) clamp field radius to 2px
0006 product str. chrome/app/theme/chromium/BRANDING Chromium -> zwire, bundle id
0007 DevTools     3 front_end files          8 HUD schemes in Theme dropdown
0008 allow framing ancestor_throttle.cc      bypass X-Frame-Options / CSP
0009 ext focus     extension_keybinding_registry_views.cc hand focus to page for %K
0010 force newtab  search.cc (HandleNewTabURLRewrite) chrome://newtab -> zwire NTP
0011 pinned exts  toolbar_view.cc          pinned actions never overflow
0012 frame-src     navigation_request.cc     host CSP never blocks subframe
0013 restore sess. session_startup_pref.cc      default startup = last session
0014 no crash bubl session_crashed_bubble_view.cc      suppress "Restore pages?" bubble
0015 no nav block  extension_navigation_throttle.cc      allow NTP nav pre-registration
0016 ext priv API  _permission_features.json hud-internal:
    ~ developer/settingsPrivate
0017 no hold-quit app_controller_mac.mm        immediate %Q (no hold-to-quit)
0018 null-pol fix  extension_function_dispatcher.cc      gate is_sandboxed on IsActive
    ~ (crash fix)
0019 store script chrome_extensions_client.cc    content scripts run on Web Store
0020 menu colors  chrome_color_mixer.cc (final mixer) native menus -> HUD Sys tokens
0021 primitive col chrome_color_mixer.cc (final mixer) native primitives -> HUD Sys
    ~ tokens
0022 audio EQ      output_controller.{cc,h}+client.cc browser-wide output EQ (every
    ~ stream)
0023 audio capture tab_capture_api.cc + _api_features Audio page taps a tab, no prompt
0024 audio live   audio_service.mojom + service.{cc,h} live EQ config + output meters
```

The series file records each patch's apply-clean verification and keeps a commented-out 0008-tmux-split-prefix tombstone (the native two-pane split that ate the C-b prefix, superseded by the JS overlay); scripts/check-patch-series.sh gates the file against the patches on disk. fork/args.gn builds it unbranded (is_chrome_branded=false), official (is_official_build=true), with symbols, PGO, NaCl, and ThinLTO all off for a workstation-tolerable build time.

Appendix 0 — The Three Extension Manifests

zwire loads three unpacked extensions, each pinned to a stable id by an embedded [key](#). Their manifests side by side:

```
— newtab — "zwire New Tab" —————
manifest_version 3 · version 1.0.6 · id gpoepnekoiplhkegjpcnpeiiefgieb
chrome_url_overrides.newtab -> newtab.html
permissions: nativeMessaging · tabs · history
externally_connectable.ids: [ hud-internal ] (so the worker can open its palette)
web_accessible_resources: newtab/palette/schemes/scheme-sync/tmux-pane/fonts, <all_urls>

— hud-internal — "zwire HUD Internal" —————
manifest_version 3 · version 0.5.0 · id omcgnnjfbmpdlofklbpddkhnfibfhgg
permissions (15): storage · nativeMessaging · management · tabs · webNavigation ·
  history · downloads · bookmarks · sessions · alarms · notifications ·
  developerPrivate · settingsPrivate · system.display · declarativeNetRequest
host_permissions: <all_urls>
declarative_net_request: rules/frame_bust.json
background: background.js (classic service worker, not a module)
commands: open-palette = ⌘K / Ctrl+K
content_scripts: 7 blocks (Appendix / Chapter 7)
externally_connectable.ids: [ zpwrchrome ]
web_accessible_resources: pages/* · schemes.js · lib/term/*
  (no minimum_chrome_version, no chrome_url_overrides)

— zpwrchrome — the power-tool (submodule) —————
manifest_version 3 · version 0.22.17 · minimum_chrome_version 127
id hpppdchpnphmiijdeanibpcadgknmaja
permissions (19, incl. userScripts · declarativeNetRequestWithHostAccess ·
  nativeMessaging · webRequest) · host_permissions <all_urls>
background: module service worker · 6 content scripts · 54 commands (Alt+T)
options_ui: scripts-manager/manager.html · externally_connectable.ids: [ hud-internal ]
```

The asymmetry in [externally_connectable](#) is the cross-extension messaging graph: the new-tab extension accepts messages from the HUD worker (for ⌘K on the new tab), and both the HUD and the power-tool accept messages from each other for the scheme bridge — the only three extensions on the machine that speak to one another, wired by id, with the native host's [allowed_origins](#) completing the triangle so any of the three can reach [com.zwire.hud](#).

Appendix P – The Content-Script Injection Order

The `hud-internal` manifest registers seven content-script blocks; the order within a block is load-bearing because later scripts read globals earlier ones set. Each script also guards against double injection with a `window.__zb...Loaded` flag.

block	run_at	frames	scripts (in order)	defines
1	document_start	top	redirect.js	(redirect)
2	document_start	top	schemes.js, theme.js	window.ZWIRE_HUD
3	document_idle	top	schemes.js, fzf.js, zfind.js	ZBFzf, find bar
4	document_start	top	schemes.js, zgui util/fzf/command-palette, zkeys.js, cmd-defaults.js, zpalette.js, zvim.js	window.ZGui, ZWIRE_KEYMAP, palette, vim
5	document_idle	top	schemes.js, powerline.js, zpowerline.js	statusbar
6	document_idle	ALL	zterm-pane.js (+ top-frame ZGui.tmux & zterm-config.js)	tmux overlay
7	document_idle	top	xterm.js, terminal.js, zterm-boot.js	terminal

Block 1 matches only the five redirected `chrome://` pages; block 2 matches every other `chrome://` page (excluding the redirected set and the new-tab URLs); blocks 3–5 and 7 match `http/https/file/chrome://` (excluding the new-tab URLs); block 6 matches only `http/https/file` and is the sole `all_frames` block, so the tmux forwarder runs inside framed panes. `schemes.js` is re-listed in every block because each block is an isolated injection that needs the palette table present.

Appendix Q – The Fuzzy Matcher

`ZGui.fzf` (`fzf.js`) is the fzf-style matcher behind the palette, the find-bar fallback, and every fuzzy-filtered HUD page — extracted verbatim from Audio-Haxor so the whole suite ranks identically. Its scoring:

```
— weights (defaults, all tunable + localStorage-persisted) —
SCORE_MATCH          16    per matched character
SCORE_GAP_START      -3    first char of a gap between matches
SCORE_GAP_EXTENSION  -1    each further gap character
BONUS_BOUNDARY       9     match right after a non-word char (start of word)
BONUS_NON_WORD       8     match across a word/non-word class change
BONUS_CAMEL          7     lowercase -> UPPERCASE transition (camelCase)
BONUS_CONSECUTIVE    4     match adjacent to the previous match
BONUS_FIRST_CHAR     x2    the first matched char's bonus is doubled

— extended query sigils (space = AND, | = OR within a group) —
!term    negate (exclude matches)
'term    or term'  exact substring
^term    prefix
term$    suffix
term     fuzzy (subsequence)
```

The matcher tries every start position of the needle's first character, greedily takes the earliest subsequent matches, scores each run with match + position bonus + consecutive/gap adjustment, and keeps the best. Matched characters are wrapped in `<mark class="fzf-hl">` for the cyan emphasis. The name field of a row gets a `+500` field bonus so a hit in the label outranks a hit in the detail.

Appendix R – Visual Effects and Readline

The cyberpunk look and the terminal-style input editing both come from `zgui-core`, on two clean conventions. Effects (`ZGui.fx`): an effect is *on* when its body class is *absent*.

```
— effects: pref -> body class -> CSS technique —————
scanlines  no-scanlines    repeating-linear-gradient on body::after (+ animated beams)
vignette    no-vignette    radial-gradient ellipse on body::before (transparent->dark)
glow        no-neon-glow   pervasive text-shadow / box-shadow (off = none !important)
anim        no-anim        all animation/transition (off = none !important)
light       [data-theme=light] flips the palette (a root attribute, not a body class)
grid        two linear-gradient hairlines, 40px fixed cell (always on)
           prefs persist to zg-fx-* localStorage and to native hud-ui.json (cross-extension)

— readline (installReadline, on every text input) —————
^A start-of-line  ^E end-of-line    ^B back char    ^F forward char
^H backspace      ^D delete char    ^K kill to EOL  ^U kill to BOL
^W kill word back ^Y yank           ^T transpose    M-b/M-f word back/fwd
M-d kill word fwd (^K/^U/^W/M-d feed a shared kill-ring that ^Y pastes)
  also: autocorrect/autocapitalize/spellcheck forced off; ⌘F/Ctrl+F focuses the filter
```

Appendix S – The System-Stats Sample

Every tick of the sysinfo stream writes one `zb_sys` object; the statusbar renders one field per segment. The sample, and which host produces each field:

field	meaning	shape	both / note
cpu	CPU percent	number	both
mem	memory used/total/percent	{u,t,p}	both
swap	swap used/total/percent	{u,t,p}	both
load	load averages (3)	[a,b,c]	both
uptime	seconds since boot	number	both
disk	root-fs used/total/percent	{u,t,p}	both
net	throughput (bytes/s)	{up,down}	both (delta / interval)
io	disk I/O (bytes/s)	{r,w}	both (delta / interval)
temp	max sensor temperature (°C)	number	both (hidden if absent)
batt	battery percent + charging	{p,c}	both (macOS/Linux/Windows)
host	short hostname	string	both
lip	LAN IP (UDP-connect trick)	string	both
pip	public IP (ipify, every 120s)	string	both

The stream cadence is a configurable `interval_ms` (floored at 250 ms), with one-shot (`sysinfo_once`) and `sysinfo_stop` variants and a background-thread pump that keeps the connection free for other RPCs. The one-shot `sysinfo_once` double-refreshes so its `net/io` rates report a near-zero delta rather than a lifetime total mislabelled as a per-second rate.

Appendix T – The Palette Destinations

Beyond your tabs, history, schemes, and keyword commands, the `⌘K` palette carries a static catalog of destinations. The HUD pages and the `chrome://` diagnostics tail:

```
— HUD replacement pages —
Extensions · Settings · History · Bookmarks · CI runs · Shortcuts (keys) ·
Extension shortcuts · Custom commands · System info (version)

— chrome:// destinations (≈44) —
newtab · downloads · history · bookmarks · extensions · settings · version ·
system · flags · discards · net-internals/#dns · gpu · net-internals ·
password-manager · inspect · net-export · policy · components · management ·
accessibility · omnibox · media-internals · webrtc-internals ·
serviceworker-internals · indexeddb-internals · quota-internals ·
blob-internals · histograms · tracing · crashes · device-log · gcm-internals ·
sync-internals · process-internals · autofill-internals · download-internals ·
signin-internals · translate-internals · user-actions · ukm · predictors ·
memory-internals · webui-gallery · chrome://about (all pages)

— chrome://settings sub-pages (19) —
You & Google · Appearance · Autofill · Payments · Addresses · Privacy ·
Security · Cookies · Site settings · Clear browsing data · Performance ·
Search engine · Default browser · On startup · Languages · Downloads ·
Accessibility · System · Reset

— external —
Chrome Web Store · zwire app store (menketechnologies.github.io/app-store/)
```

The new-tab palette carries the same static catalog (it is a separate extension and must re-carry it), which is why the two palettes list identical destinations even though they reach `chrome.tabs` by different paths (Chapter 12).

Appendix U – The Command Bus

Content scripts cannot call the privileged tab, window, and session APIs, so they write a `zb_cmd` object to `chrome.storage.local`; the `storage.onChanged` event reliably wakes the MV3 worker, which performs the action. Every `zb_cmd` carries an action field `a`, a monotonic tag `n` (so a repeated identical command still registers as a change), and any parameters. The worker's dispatch:

<code>a</code>	<code>params</code>	<code>effect</code>
<code>ping</code>	<code>-</code>	wake + refresh <code>zb_tabs/zb_exts/zb_frecent/zb_shortcuts</code>
<code>open</code>	<code>url</code>	navigate the active tab, else create a tab
<code>openTab</code>	<code>url</code>	create a new tab
<code>activate</code>	<code>tabId</code>	switch to a tab + focus its window
<code>newTab</code>	<code>-</code>	create an empty tab
<code>newWindow</code>	<code>-</code>	create a new window
<code>duplicateTab</code>	<code>-</code>	duplicate the active tab
<code>reopenTab</code>	<code>-</code>	<code>sessions.restore</code> (reopen last closed)
<code>pinTab</code>	<code>-</code>	toggle the active tab's pinned state
<code>muteTab</code>	<code>-</code>	toggle the active tab's muted state
<code>closeTab</code>	<code>-</code>	close the active tab
<code>closeOthers</code>	<code>-</code>	close every non-active, non-pinned tab in the window
<code>nextTab</code>	<code>-</code>	cycle to the next tab (last-focused window)
<code>prevTab</code>	<code>-</code>	cycle to the previous tab
<code>tmux</code>	<code>sub, ...</code>	drive the worker's OS-window tiling engine (Chapter 8)

The vim script issues `prevTab/nextTab/closeTab/newTab/openTab` this way; the palette adds `duplicateTab`, `reopenTab`, `pinTab`, `muteTab`, `closeOthers`, `newWindow`, and `activate`. A parallel path — `runtime.onMessage` — carries the scheme set (`zbhud-scheme`), the scheme get (`zbhud-get`), a content-script open (`zbopen`), and a wake ping (`zbping`); these are request/response where the bus is fire-and-forget, and the worker uses whichever suits the reliability the caller needs.

Appendix V — The Frecency Model

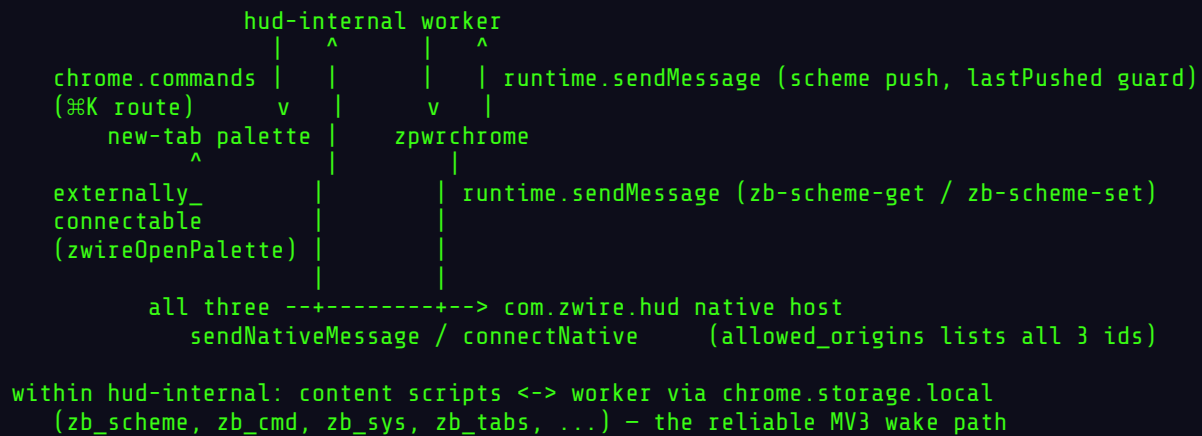
Three surfaces — the worker's `zb_frecent`, the new-tab palette, and the HUD pages' `zg-boot.js` — surface your “frequent + recent” history, and all three use the same formula, so a site ranks identically wherever it appears:

```
window    chrome.history.search over the last 90 days (startTime = now - 90d)
score      (visitCount + 2 * typedCount) / (1 + ageDays * 0.3)
           ageDays = (now - lastVisitTime) / 86_400_000
filter     drop URLs beginning with "chrome"
keep       the top 30 by score, descending
```

A typed visit counts double a clicked one (you meant to go there), and the divisor decays a site's score by roughly a third per day of age, so recency and frequency trade against each other rather than one dominating. The worker recomputes the list on every `history.onVisited`, so the palette's history rows are never stale.

Appendix W – The Extension Messaging Graph

Three extensions and one native host coordinate across boundaries that neither `chrome.storage` nor `localStorage` can cross. Who reaches whom:



The rule underneath the graph: storage is shared only *within* an extension, so every cross-extension value (the scheme, the UI prefs) travels through the native files, and every cross-extension *event* (open the palette, push a scheme) travels through `externally_connectable` or `runtime.sendMessage` between the specific ids each manifest allows.

Appendix X – The zgui-core Toolkit Surface

zwire loads a slice of the shared [zgui-core](#) toolkit; the modules and their public surface:

```
util.js  -> ZGui.util  escapeHtml · throttle · debounce · clamp · formatBytes ·
                                formatDuration · slugify · copyToClipboard · lineEdit ·
                                installReadline · focusFilterBar · dampenInput ·
                                installInputDefaults (auto-installs readline + input
                                defaults on load; binds ⌘F/Ctrl+F to the filter bar)
fzf.js   -> ZGui.fzf   fzfMatch · getMatchIndices · highlightWithIndices ·
                                searchScore · parseFzfQuery · setParam/defaults/bounds
command-palette.js -> ZGui.palette register · registerProvider · clear · open ·
                                close · isOpen · bindHotkey · create (three-tier ranking:
                                static+strong / secondary / fallback-last)
fx.js    -> ZGui.fx    set · toggle · get · all · load · buildToggles (the four
                                body.no-* effect classes)
crt.js   -> ZGui.crt / ZGui.neonGlow the legacy scanline + glow layers
fonts.css + webui/fonts/  Orbitron (display) + Share Tech Mono (body), SIL OFL
```

The guidance from [zgui-core](#)'s own docs is to consume it as a submodule and load the modules directly — never copy [webui/*](#) — which [zwire](#) follows for the content-script extension; the new-tab extension, a distinct origin, bundles its own copy of the [util/fzf/command-palette](#) trio under [lib/zgui/](#).

Appendix Y – The HUD Internal Pages

Each replacement page, the privileged API it stands on, and the state it reads or writes:

page	API	reads / writes
extensions	developerPrivate + management	enable/disable/reload/pack/site-access
settings	settingsPrivate	every Chrome pref + scheme/effects -> hud-ui
history	history.search	recency-sorted list (zg-list)
bookmarks	bookmarks.getTree	flattened folder-path table
downloads	downloads.search	open-on-click (native page owns the manager)
version	runtime + navigator + host	live com.zwire.hud probe
keys	ZWIRE_KEYMAP	writes zb_keys remaps + zb_tmux_prefix/opts
commands	–	writes zb_custom_cmds (+ zb_cmds_removed)
sessions	–	zb_tmux_sessions CRUD -> zb_tmux_load trigger
extshortcuts	developerPrivate	updateExtensionCommandKeybinding
ci	GitHub REST API	zb_ci config -> per-repo run accordions

All eleven mount through `zg-boot.js` (`ZBHUD.mount`), share the `zb-*` shell chrome, carry the same `⌘K` palette, and filter through the same `fzf.js` — one surface, not eleven.

Appendix Z – The Dev-Loop Scripts

The `scripts/` family assembles and resets the base; none of them compile the browser (the base is a prebuilt snapshot). Each sources `cyberpunk.sh` for its banner:

```
dev.sh      launch from the checkout, foreground (content scripts hot-reload)
build.sh    ensure a current rebranded base exists (fetch if missing, then rebrand)
rebuild.sh  clean + build
clean.sh    remove the /Applications wrapper + staged dist/ (keeps base + profile)
bust.sh     re-seal + re-register the bundle with LaunchServices (macOS only)
nuke.sh     wipe base + wrapper, re-fetch, rebrand, reinstall (keeps the profile)
test.sh     node --check the front-end (excl. node_modules + lib/zgui-core),
            then each extension's own pnpm test
install.sh  PATH symlink + fetch + (macOS) rebrand – runs from the checkout
localinstall*.sh  self-contained app: browser + ext + Rust host in one location
setup-native-host.sh  cargo build --release zwire-host + write the host manifest
fetch-base.sh      download + pin the Chromium snapshot base
check-patch-series.sh  CI gate: fork/patches/series <-> the *.patch files on disk
```

The destructive ones are careful about what they keep: `clean` and `nuke` never touch the profile, and `clean` does not even touch the 325 MB base — that is `nuke`'s job. The npm scripts in `package.json` alias the common ones (`pnpm dev`, `pnpm build`, `pnpm nuke`) for anyone who reaches for the package manager instead.

Appendix AA – Three Paths, End to End

The HUD's design is a set of multi-hop paths across isolation boundaries. Three of them, traced in full, tie the book together.

Switching the color scheme:

```
pick a scheme (palette / picker / Settings page)
-> runtime.sendMessage {zbhud-scheme} to the worker + storage.local zb_scheme
-> worker: sendNativeMessage {scheme} to com.zwire.hud
    -> host writes ~/.zwire/hud-scheme (atomic tmp + rename)
-> worker: pushToZpwr (runtime.sendMessage, lastPushed guard)
renderers reconcile to the file / the mirror:
content scripts + chrome:// skin    <- storage.local zb_scheme (observed)
new-tab extension                   <- polls hud-scheme every 1.5s
zpwchrome                           <- the runtime.sendMessage push
fork native color mixer              <- reads hud-scheme natively
```

Opening the command palette with ⌘K:

```
ordinary page    zpalette.js catches ⌘K in a capture listener -> opens
new-tab page     Chrome reserves ⌘K -> worker owns it as a chrome.commands
                  shortcut -> sendMessage(newtab, zwireOpenPalette) -> palette opens
inside a pane    the forwarder postMessages {palette:1} up -> top frame opens,
                  navigates the active pane instead of a stray tab
```

A keystroke in the terminal overlay:

```
xterm onData -> content-script port {pty_write} -> worker (zwire-pty, per-tab)
-> connectNative com.zwire.hud -> host os.write(master, bytes)
shell output -> host reads 64KB -> base64 {ev:output,b64} -> worker
-> (buffer if detached, else) content-script port -> atob -> xterm.write
navigation: content script disconnects, worker keeps sess.nat alive,
next page re-injects, re-attaches to the same tab's shell – session survives
```

Each path crosses at least one boundary a single page cannot cross on its own — a suspended worker, a second extension, a native process — and each is routed around its guard by the mechanism the earlier chapters named: the storage bus for the worker, [externally_connectable](#) for the other extension, the per-tab port for the PTY. That routing, repeated for every feature, is the whole of what makes a stack of extensions behave like one browser.

Appendix AB – Glossary

The recurring names, in one place.

```
— native files + host —————
hud-scheme      the active scheme; the cross-extension source of truth (a file)
hud-ui.json     shared visual-effect + light-mode prefs (a file)
com.zwire.hud   the native-messaging host manifest -> the Rust zwire-host binary
zwire-pty       the runtime.connect port name for the terminal's PTY relay

— storage keys (chrome.storage.local, Appendix I) —————
zb_scheme       mirror of the active scheme (worker <-> content scripts)
zb_cmd          the command bus (content script -> worker, Appendix U)
zb_sys          the streamed system-stats sample (statusbar, Appendix S)
zb_tabs/_exts/_frecency/_shortcuts  worker-maintained mirrors for the palette
zb_keys         keybinding remaps { action: key }
zb_custom_cmds  the %K keyword commands (Appendix H)
zb_term_open    whether the terminal re-opens on the next page
zb_tmux*        overlay readout, window-engine state, sessions, prefix, opts
zb_status       whether the powerline statusbar is shown
zb_ci*          CI-monitor config, status, cache

— globals (in the page / content scripts) —————
window.ZWIRE_HUD  the 8-scheme palette table (VAR_KEYS, ORDER, SCHEMES)
window.ZWIRE_KEYMAP the keybinding registry (Appendix C)
window.ZGui       the shared toolkit (fzf, palette, fx, util – Appendix X)
window.ZBHUD      the HUD-page shell (mount, palette, publishUi)

— the three extension ids —————
hud-internal      omcgnnjfbmpdlofklbpddkhnfibfhgg
zpwichrome        hpppdchpnphmiijdeanibpcadgknmaja
new-tab           gpoepnekoiplhkegjpcocnpeijiefgieb

— other terms —————
frecency          frequency+recency history score (Appendix V)
the fork          fork/, a compiled patched Chromium as an alternate base (Ch 21)
the rebrand       the runtime, no-compile skinning path (the default, Ch 5)
```

Appendix AC – The Statusbar Segments

The powerline statusbar (shared `ZGui.powerline` fed by `zpowerline.js`, top frame only, `position:fixed` bottom, 22px) is a left group of HUD state and a right group of live machine stats, split by a flex spacer, with true CSS-border powerline chevrons between blocks. Reading left to right:

```
— left: HUD state —————
ZW      the badge (cyan block, chevron)
C-b     the tmux prefix indicator (dim; flips to accent for 1.6s when armed)
<scheme> the active scheme's label
i·panes  per-window tmux readout (active emphasized, ± when syncing)
VIM     static mode marker
%K      clickable; opens the palette

— right: live machine (from zb_sys, Appendix 5) —————
seg      data-*      format
CPU      data-cpu     <n>%
MEM      data-mem     used/total   (humanized B/K/M/G/T, 1 decimal < 100)
SWP      data-swap     used/total
DSK      data-disk     <n>%
IO       data-io       R<rate> W<rate>      (bytes/s)
NET      data-net      ↑<rate> ↓<rate>      (bytes/s)
LD       data-load     a b c              (load averages)
UP       data-up       Xd Yh Zm          (humanized uptime)
°        data-temp     <n>°C              (segment hidden if no sensor)
BATT     data-batt     ⚡/battery + <n>%
LAN      data-lip      private IP
WAN      data-pip      public IP
@        data-host     short hostname
clock    data-clock    MM-DD HH:MM:SS      (own 1-second interval)
```

The chevrons are real triangles: an 11-pixel transparent top/bottom border plus a solid side border on a pseudo-element, left-of-spacer blocks pointing right and right-of-spacer blocks pointing left, consecutive blocks alternating two background shades. A stats refresh rewrites the `data-*` spans in place; a scheme change re-themes the bar from the same twenty variables as everything else.

Appendix AD — Grounding and Edition

Every claim in this book traces to a file in the `zwire` repository. Where the text says “the code comments say,” it is quoting the source verbatim; where it gives a number — 66 seeded commands, 44 `chrome://` destinations, a 0.28 tab slope, a 90-day frequency window, nine launch flags, twenty-four patches — that number was read out of the code, not recalled. Where the runtime path and the source fork differ, the book marks the boundary rather than rounding up; the honest gaps (the two dead CDP-probe lines) are named where they occur because a companion grounded in the code owes the reader its seams as much as its surfaces.

Two corrections this edition makes to earlier printings are worth flagging: the state directory default is the OS application-data location (with `~/.zwire` a legacy path that is migrated once, not the live one), and the Python reference host has been removed — the native host is now the Rust `zwire-host` alone, and where earlier printings speak of a “Python reference,” read that as the historical first implementation of the same `scheme/stats/PTY` path `zwire-host` now owns outright.

The edition string on the cover tracks `zwire/package.json`. `zwire` is fast-moving software, a MenkeTechnologies project built on the Chromium project’s engine; this book will move with the code, and where the two ever disagree, the code is right and the book is the bug.